KH정보교육원

SQL 활용

목 차

Part 1. 기본 SQL 작성하기

Chapter 1. DQL - SELECT 구문, 연산자 4

Chapter 2. 함수 30

Chapter 3. DML - INSERT, UPDATE, DELETE 구문 55

Chapter 4. TCL - TRANSACTION 구문 64

Part 2. 고급 SQL 작성하기

Chapter 5. 분석함수 69

Chapter 6. JOIN, SET 연산자, SUBQUERY 78

Chapter 7. DDL - TABLE 과 제약조건 105

Chapter 8. 공공데이터 테이블 이용하기 144

Part 1. 기본 SQL 작성하기

Chapter 1. DQL – SELECT 구문, 연산자

이번 장에서는 DB에서 가장 많이 사용하는 Data를 조회하는 명령어인 SELECT문과 WHERE절, ORDER BY절의 사용법을 이해하고, DB에서 기본적으로 사용하는 연산자 및 사용자 테이블 검색할 수 있는 명령어와 테이블 속성 정보를 확인하는 명령어를 배워보자.

1 SELECT 문

SQL문 중 가장 많이 쓰이며 테이블에 저장된 데이터를 조회하기 위한 명령어로 SELECT문에는 반드시 FROM 키워드가 따라와야 한다.

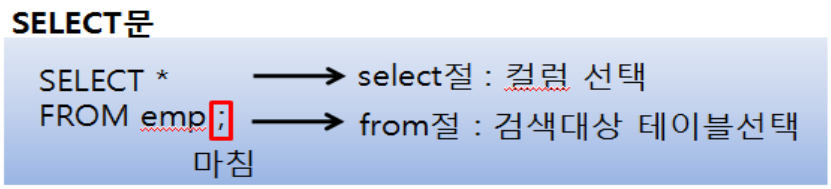
1.1 SELECT 형태

```
SELECT  {* | [ DISTINCT ] column [as alias] | 계산식 | 함수식 | … }
FROM table_name
[WHERE  컬럼명 비교연산자 비교값]
[ORDER BY 정렬할 컬럼명 정렬방식];
```

- DISTINCT : 중복 행 제거 옵션,
- * 또는 컬럼명 : 테이블 모든 컬럼 조회 하거나 해당 컬럼만 조회
- alias : 조회된 컬럼에 대해서 다른 이름을 부여할 때 사용, 컬럼명 뒤에 as는 생략 가능
- table_name : 조회 대상 테이블 명
- WHERE : 조건절로 비교한 결과에 일치하는 행만 검색
- ORDER BY : 정렬 순서를 정함, DESC(내림차순), ASC(오름차순, default로 생략 가능)

☞ 구문에서 [ ]는 생략이 가능한 구문, { }는 안의 내용 중 하나는 입력해야 한다는 의미

1.2 기본 SELECT 문



- SQL문은 그냥 엔터로 수행되지 않고 실행 문장이 끝나면 마침 표시인 ; 를 넣고 엔터를 입력해야 하며, SQLDeveloper 툴에서는 마침 표시 후 실행 버튼(F5 또는 Ctrl + Enter)를 실행해야 한다.
- 마침 표시를 하지 않을 경우 SQLDeveloper 툴의 경우 자동으로 실행시켜 주지만 SQLplus 에서는 실행되지 않는다.

1.2.1 특정 컬럼만 선택해서 조회하는 SELECT문

```
SELECT 컬럼이름1, 컬럼이름2, 컬럼이름3, …
FROM 테이블이름;
```

※ select 문에 작성된 컬럼 순서대로 출력이 된다.

테이블의 원하는 컬럼만 조회하고 싶을 때는 SELECT 뒤에 원하는 컬럼명을 작성하여 조회할 수가 있다. 원하는 컬럼이 한 개일 경우 하나만 쓰고 출력하면 되고 여러 개일 경우에는 ' , ' 로 구분하여 원하는 컬럼명을 적어주면 된다.

그러면 scott 계정을 이용하여 EMP 테이블에서 ENAME 만 출력해보자

쿼리문

```
SELECT ename
FROM emp ;
```

결과

	ENAME
1	SMITH
2	ALLEN
3	WARD
4	JONES
5	MARTIN
6	BLAKE

SELECT 문에 뒤에 컬러명 하나를 넣으면 하나만 출력되는 것을 확인할 수 있다.

이번에는 원하는 컬럼이 EMPNO, ENAME 이라고 가정하여 출력해 보자.

쿼리문

```
SELECT empno, ename
FROM emp ;
```

결과

	EMPNO	ENAME
1	7369	SMITH
2	7499	ALLEN
3	7521	WARD
4	7566	JONES
5	7654	MARTIN
6	7698	BLAKE
7	7782	CLARK

결과에서 보는 바와 같이 SELECT문 뒤에 원하는 컬럼이 여러 개 일 경우 ' , ' 로 구분하여 작성하면 원하는 컬럼이 조회 되는 것을 확인 할 수 있다.

1.2.2 모든 컬럼을 조회하는 SELECT문

```
SELECT *
FROM 테이블이름;
```

테이블의 모든 컬럼을 조회하기를 원할 때 모든 컬럼을 SELECT 뒤에 작성하면 되지만 프로그램을 구현하다 보면 수많은 테이블의 모든 컬럼을 다 기억 하기란 쉽지 않다. 그래서 모든 컬럼을 한번에 조회할 수 있는 특별한 문자를 만들어 사용하는데 이것이 바로 ' * ' 이다.
SELECT 뒤에 * 를 넣으면 선택된 테이블의 모든 컬럼을 출력하라는 의미이기 때문에 테이블의 모든 컬럼을 출력하고 싶을 때 SELECT 뒤에 컬럼명을 모두 적는 대신 * 를 넣으면 훨씬 효율적이고 간편할 것이다.

그러면 * 를 활용하여 EMP 테이블의 모든 데이터를 출력해 보자.

쿼리문

```
SELECT *
FROM emp ;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	80/12/17	800	(null)	20
2	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
3	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30
4	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
5	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
6	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30

결과에서 보는 바와 같이 SELECT문 뒤에 *를 넣으면 모든 컬럼의 데이터가 출력되는 것을 확인할 수 있다.

1.2.3 SQL문 작성 유의 사항

- SQL 문장은 대소문자를 구별하지 않는다
- SQL 문장은 한 줄 또는 여러 줄에 입력될 수 있다
- 일반적으로 키워드는 대문자로 입력한다
- 다른 모든 단어, 즉 테이블 이름, 열 이름은 소문자로 입력을 권장
- 가장 최근의 명령어 한 개가 SQL buffer 에 저장 된다
- SQL문 마지막 절의 끝에 ";" 를 기술하여 명령의 끝을 표시한다.

1.3 컬럼명에 별칭 붙이기 (예명 : Alias)

1. **SELECT** 컬럼명 별칭, 컬럼명2 별칭 **FROM** 테이블이름;
2. **SELECT** 컬럼명 **AS** 별칭, 컬럼명2 **AS** 별칭 **FROM** 테이블이름;
3. **SELECT** 컬럼명 "별칭", 컬럼명2 ″별칭″ **FROM** 테이블이름;

테이블 생성 시에 컬럼의 이름을 정하지만 간혹 데이터를 사용하다가 보면 컬럼을 다른 이름으로 사용하고 싶을 때가 있는데 이럴 때 컬럼 뒤에 AS 별칭 혹은 AS 없이 별칭을 기록하면 그 해당되는 이름으로 조회 결과가 출력된다.
scott 계정의 EMP 테이블의 ENAME컬럼을 이름으로 변경하고, SAL컬럼을 봉급이란 이름으로 변경하여 먼저 두가지 방법으로 출력해보자.

쿼리문

```
SELECT  ename 이름, SAL 봉급
FROM emp ;
```

결 과

	이름	봉급
1	SMITH	800
2	ALLEN	1600
3	WARD	1250
4	JONES	2975
5	MARTIN	1250
6	BLAKE	2850

쿼리문

```
SELECT  ename  AS 이름,  SAL  AS 봉급
FROM emp ;
```

결 과

	이름	봉급
1	SMITH	800
2	ALLEN	1600
3	WARD	1250
4	JONES	2975
5	MARTIN	1250
6	BLAKE	2850

위 결과와 같이 컬럼 뒤에 AS 또는 AS 없이 별칭을 써서 사용자가 원하는 컬럼명으로 변경하여 결과를 출력할 수 있다.
마지막 별칭을 부여하는 방법은 ″ ″ 을 사용하는 방법인데 이는 별칭에 띄어쓰기나 특수기호(*, $, _, #)을 넣기를 원하거나 대소문자 구분을 원하는 경우에 사용한다.
" " 를 사용하여 emp 테이블의 ename 을 “성 명” 으로 sal 을 “$_봉급” 으로 출력해 보자.

쿼리문

```
SELECT  ename "성 명",  SAL "$_봉급"
FROM emp ;
```

결 과

	성 명	$_봉급
1	SMITH	800
2	ALLEN	1600
3	WARD	1250
4	JONES	2975
5	MARTIN	1250
6	BLAKE	2850

이 별칭은 DB에 저장되어 있는 컬럼명이 완전히 바뀌는 것이 아니고 SELECT 문을 이용하여 결과를 조회할 때만 원하는 컬럼명으로 보여지는 것이다. 컬럼명을 완전히 바꾸려면 ALTER 라는 테이블 변경 구문을 써야 하는데 이는 테이블 생성, 변경, 삭제 장에서 공부하도록 하자.

1.4 테이블 관련 정보 확인 SQL문

데이터베이스를 활용하다 보면 자신의 사용하는 계정이 어떠한 테이블을 가지고 있고 그 테이블에 어떠한 컬럼과 제약조건이 있는지 확인하고 싶을 때가 있다. 이럴 때 사용하는 것이 TAB 테이블과 desc 구문이다.

1.4.1 TAB 테이블

TAB 테이블은 table의 약자로 접속한 계정이 소유하고 있는 테이블의 정보를 알려주는 데이터 딕셔너리이다.
데이터 딕셔너리는 Oracle 시스템에서 객체(테이블)에 대한 정보를 저장하는 곳으로 테이블에 대한 컬럼, 도메인, 제약조건에 대한 메타 데이터를 저장한다.

- scott 계정 테이블확인

테이블 확인 전 현재 계정이 scott 인지 확인한다.

쿼리문

```
show  user ;
```

결 과

```
USER이(가) "SCOTT"입니다.
```

계정을 확인했으면 SELECT 구문으로 tab 을 조회하여 scott 이 소유하고 있는 테이블을 확인한다.

쿼리문

```
SELECT * FROM tab;
```

결 과

	TNAME	TABTYPE	CLUSTERID
1	BONUS	TABLE	(null)
2	DEPT	TABLE	(null)
3	EMP	TABLE	(null)
4	SALGRADE	TABLE	(null)

tab을 이용하여 scott 계정에는 4 개의 테이블이 있다는 것을 확인할 수 있다.

- sys계정 테이블 확인

테이블 확인 전 계정변경  을 이용하여 sys 계정으로 다시 로그인 한다.

다시 로그인 후 사용자 계정 확인

쿼리문

```
show  user ;
```

결 과

USER이(가) "SYS"입니다.

쿼리문

```
SELECT  * FROM  tab;
```

결 과

	TNAME	TABTYPE	CLUSTERID
1	ACCESS$	TABLE	(null)
2	ALERT_QT	TABLE	(null)
3	ALL_ALL_TABLES	VIEW	(null)
4	ALL_APPLY	VIEW	(null)
5	ALL_APPLY_CHANGE_HANDLERS	VIEW	(null)
6	ALL_APPLY_CONFLICT_COLUMNS	VIEW	(null)
7	ALL_APPLY_DML_CONF_HANDLERS	VIEW	(null)
8	ALL_APPLY_DML_HANDLERS	VIEW	(null)
9	ALL_APPLY_ENQUEUE	VIEW	(null)
10	ALL_APPLY_ERROR	VIEW	(null)

sys 계정은 관리자 계정이기 때문에 많은 테이블이 있다는 것을 확인할 수 있다.

1.4.2 DESC(DESCRIBE)를 활용한 테이블 정보확인

```
DESC 테이블명 ;
```

DESC는 테이블에 대한 컬럼, 제약조건 등을 정보를 확인하는 명령어로 사용자 소유의 테이블 정보를 확인하기 위해 사용한다. 테이블에 대한 정보를 확인하려면 DESC 구문을 사용하면 된다.

emp 테이블에 대한 정보를 확인해 보자.

※ 테이블 명은 바로 전에 배운 tab 테이블을 이용하여 확인하면 된다.

쿼리문

```
DESC emp;
```

결 과

```
이름        널         유형
--------- -------- ------------
EMPNO     NOT NULL NUMBER
ENAME              VARCHAR2(10)
JOB                VARCHAR2(9)
MGR                NUMBER
HIREDATE           DATE
SAL                NUMBER
COMM               NUMBER
DEPTNO             NUMBER
```

DESC를 통해 출력해 본 결과 EMP 테이블의 각 컬럼에 대한 정보(제약조건, 데이터 자료형, 컬럼명)을 확인할 수 있다.
DB 작업시 테이블에 대한 컬럼명을 확인하거나 유형을 확인할 때 자주 쓰이는 명령어이다.

1.5 중복을 제거하는 DISTINCT

데이터를 조회할 때 값이 중복되는 행이 여러 개 조회되는 경우가 있는데, 값이 중복된 행을 한 개씩만 선택하고자 할 때 사용하는 키워드 이다.
DISTINCT 를 SELECT와 컬럼명 사이에 넣어 활용한다.

```
SELECT  DISTINCT 컬럼명, 컬럼명1, 컬럼명2
FROM 테이블이름 ;
```

emp 테이블에서 부서번호 deptno 를 그냥 조회하면 아래와 같이 출력된다.

쿼리문

```
SELECT deptno FROM emp;
```

결과

	DEPTNO
1	20
2	30
3	30
4	20
5	30
6	30
7	10
8	20
9	10
10	30
11	20

이를 DISTINCT를 활용하여 중복값에 대한 행을 하나씩만 출력되게 해보자

쿼리문

```
SELECT  DISTINCT  deptno
FROM  emp;
```

결 과

	DEPTNO
1	30
2	20
3	10

결과와 같이 출력된 레코드가 DISTINCT 를 사용하지 않았을 때는 중복값이 출력되어 11개 행의 데이터가 조회되었는데 DISTINCT 를 사용하니 중복 값이 제거되어 3개 행만 출력된 것을 확인할 수 있다. 데이터 조회시 중복 값을 제거할 때 유용하게 사용한다.

DISTINCT 뒤에 컬럼명을 2개 이상 기술하면, 그 컬럼명을 하나의 레코드로 인식하고, 중복을 제거한다.

쿼리문

```
SELECT  DISTINCT  DEPTNO, JOB
FROM EMP;
```

결 과

	DEPTNO	JOB
1	20	CLERK
2	30	SALESMAN
3	20	MANAGER
4	30	CLERK
5	30	SALEMAN
6	(null)	(null)
7	10	PRESIDENT
8	30	MANAGER
9	10	CLERK
10	10	MANAGER
11	20	ANALYST
12	(null)	MANAGER

DISTINCT 키워드를 추가함으로써 DEPTNO 컬럼값과 JOB 컬럼값을 하나의 레코드라 간주하고, 중복을 제거하였다. 이때 NULL 도 하나의 값으로 인식해서, 중복된 NULL 값은 하나의 NULL 로 처리된다.

1.6 컬럼값 계산하는 산술연산자

자료형이 숫자인 컬럼값들을 계산한 값을 조회하고 싶은 경우에 산술연산자를 사용한다. 산술연산자는 기본 수학에서 나오는 +, -, *, /가 있으며 우선순위는 수학과 동일하다. 사용시 유의할 점은 두 컬럼 모두 자료형이 숫자형(NUMBER)이나 날짜형(DATE) 이어야 한다. 그리고 날짜형의 경우에는 +, -만 사용 가능하다.

1. **SELECT** 컬럼명 + 컬럼명1 **FROM** 테이블명;
2. **SELECT** 컬럼명 - 컬럼명1 **FROM** 테이블명;
3. **SELECT** 컬럼명 * 컬럼명1 **FROM** 테이블명;
4. **SELECT** 컬럼명 / 컬럼명1 **FROM** 테이블명;

사원들의 연봉을 계산하여 출력해 보자.

쿼리문

```
SELECT ename, sal, sal * 12
FROM   emp;
```

결 과

	ENAME	SAL	SAL*12
1	SMITH	800	9600
2	ALLEN	1600	19200
3	WARD	1250	15000
4	JONES	2975	35700
5	MARTIN	1250	15000
6	BLAKE	2850	34200
7	CLARK	2450	29400
8	SCOTT	3000	36000
9	KING	5000	60000
10	TURNER	1500	18000

2 WHERE 구문

데이터를 조회할 때 사용자가 원하는 조건에 맞는 데이터만을 조회하고 싶을 때 사용하는 것이 WHERE구문이다. 기본 WHERE 구문과 WHERE 구문에 활용하는 연산자에 대해 알아보도록 하자.

2.1 기본 WHERE 절

```
SELECT 컬럼명, 컬럼명1
FROM 테이블명
WHERE 칼럼이름 비교연산자 조건값;
```

SELECT 문 FROM 절 다음에 위치하고 WHERE 절은 조건식을 표현하는데, 이 조건은 테이블에서 비교 또는 논리연산을 통해 행들이 골라지고 이 조건에 만족하는 데이터만 조회하여 출력한다.
WHERE 절에서는 비교, 논리연산을 통해 조건을 만들고 결과를 출력하기 때문에 비교, 논리 연산에 대한 이해가 중요하다.
아직 연산에 대해 공부하기 전이기 때문에 간단한 예제 실습해보고 넘어가자.
emp 테이블에서 부서번호(depno)가 10인 사원의 정보를 출력해 보자

쿼리문

```
SELECT * FROM emp
WHERE deptno = 10;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
2	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
3	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10

컬럼에 비교 연산자 '=' 을 사용하여 deptno = 10 이라는 조건을 만들었다. 프로그램언어에서는 '='이 대입 연산자이지만 DBMS 에서는 '=' 는 수학적 의미와 같은 '같다'는 의미라는 것을 기억하자.
'=' 는 같다는 의미이기 때문에 deptno 컬럼이 10 인 행을 찾아 출력하라는 의미이다.
출력결과를 보면 deptno가 10인 3개의 행만 출력된 것을 확인할 수 있다.

2.2 오라클 기본 연산자

2.2.1 비교 연산자

각 컬럼에 들어있는 값을 비교하여 그 결과가 일치하는 행을 조회해 주는 기능을 작성하기 위해 사용한다. 비교 연산은 수학에서의 의미와 비슷하지만 연산자의 모습이 약간 다르다. 숫자 데이터뿐만 아니라 문자와 날짜도 비교할 수 있다. 컬럼값 대 컬럼값으로 비교할 수 있고, 컬럼값 대 사용자가 지정한 값과 비교할 수도 있다.

연산자	설명
=	같다
>	크다
<	작다
>=	크거나 같다
<=	작거나 같다
<>, !=, ^=	같지 않다

비교 연산자는 수학에서 비교하는 방법과 유사하다. 이해하기 쉬운 개념이기 때문에 간단한 예제만 보고 넘어가도록 하자.

쿼리문

```
SELECT * FROM emp
WHERE comm > 500 ;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10

위의 명령 수행 결과 comm(Number형) 컬럼의 값이 500 초과인 데이터만 조회하여 출력하였다. 만일 500을 포함하려면 '>=' 를 적어주면 된다. 사용자가 지정한 숫자형값을 비교할 때는 비교연산자 뒤나 앞에 그냥 숫자만 적어주면 된다. 하지만 문자형이나 날짜형을 비교할 때는 값에 ' ' 를 붙여줘야 한다. 다음 예제를 보자.

쿼리문

```
SELECT * FROM emp
WHERE hiredate < '81/01/01' ;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	80/12/17	800	(null)	20

hiredate (고용일) 이 81년 01 월 01일 이전인 데이터를 조회하는 명령이다. 출력결과 지정일보다 빠른 날인 사원은 SMITH 1명 뿐이다.
날짜를 비교할 경우 날짜형식에 맞춰서 비교하여 값을 '년/월/일'로 넣어주면 된다.
문자형을 비교할 경우에도 ' ' 안에 비교할 문자를 넣어주면 된다.

쿼리문

```
SELECT * FROM emp
WHERE ename = 'WARD';
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30

문자 값은 '=' 연산자와 비교 시에는 대소문자를 구분하므로 값과 정확하게 일치하는 문자를 적어야 한다. 만일 데이터 값이 대문자인데 값을 소문자로 입력하면 아래 같이 정확한 결과가 나오지 않는다.

쿼리문

```
SELECT * FROM emp
WHERE ename = 'ward';
```

결 과

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO

문자형의 크기를 비교하면 알파벳 순서를 기준으로 높고, 낮음을 판단한다 .(대문자가 소문자가 보다 작음)
일치하지 않는 정보를 조회할 때는 네가지 방법이 있는데 '<>', '!=', '^='가 있으며 마지막으로 WHERE 뒤에 NOT을 쓰고 '=' 연산자를 사용하면 된다. 이 네 가지는 데이터가 같지 않은 데이터를 조회하라는 뜻이다. 아래 예제와 같이 사용하면 된다.

쿼리문

```
SELECT * FROM emp
1. WHERE deptno <> 30;
2. WHERE deptno != 30;
3. WHERE deptno ^= 30;
4. WHERE NOT deptno = 30;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	80/12/17	800	(null)	20
2	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
3	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
4	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
5	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
6	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
7	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
8	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10

세 가지를 모두 출력해 보면 동일한 결과값이 출력되는 것을 볼 수 있다. 다양한 비교식을 만들어서 조회하는 연습을 해보는 것이 비교연산자를 공부하는 데 많은 도움이 된다.

2.2.2 논리 연산자

비교 연산자나 기타 WHERE 에 조건구문이 두 개 이상일 경우 논리연산자를 이용하여 조건을 논리적으로 연결하여 조건에 맞는 값을 조회할 수 있게 한다. 예제를 보면서 설명하겠다.

연산자	설 명	비 고
조건A AND 조건B	두 가지 조건이 모두 만족되도록 결과 조회	교집합
조건A OR 조건B	두 가지 조건 중 하나라도 만족되도록 결과 조회	합집합
NOT 조건A	조건을 만족하지 않도록 결과 조회	여집합

EMP 테이블에서 sal 이 300 이상이고 deptno 가 30 인 사원을 찾아보자.

쿼리문

```
SELECT * FROM emp
WHERE sal >= 3000 AND deptno = 20 ;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
2	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20

sal >= 3000 이고 deptno = 20 인 두 개의 조건을 모두 만족해야 하므로 두 개의 조건식을 AND로 연결하여 WHERE 절을 작성하여 출력하면 두 조건을 모두 만족하는 두 개의 데이터가 조회되는 것을 확인할 수 있다.

만일 sal 이 3000 이거나 deptno 이 20 인 자료를 찾을 경우 즉 두 개의 조건 중 한 개만 만족해도 출력하려고 하면 아래와 같이 OR로 두 조건식을 묶으면 된다.

쿼리문

```
SELECT * FROM emp
WHERE sal >= 3000 OR deptno = 20 ;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	80/12/17	800	(null)	20
2	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
3	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
4	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
5	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
6	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20

OR로 조건식을 연결하면 두 개의 조건식 중 하나만 만족해도 그 데이터를 출력하기 때문에 데이터 조회결과가 위와 같이 sal 이 3000 이상이지만 deptno 가 20 이 아닌 사원도 출력되고 sal 이 3000 이상이 아니지만 deptno 가 20 인 사원이 출력이 된다.

논리연산자는 비교할 조건이 많으면 여러 개를 사용할 수 있다.
emp 테이블에서 sal 이 200 이상이고 deptno 가 20 이고 hiredate 가 82년 01월 01일 이전인 데이터를 조회해 보자.

쿼리문

```
SELECT * FROM emp
WHERE sal >= 2000 AND deptno = 20 AND hiredate < '82/01/01';
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
2	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20

3 개의 조건을 모두 만족해야 하기 때문에 AND 연산으로 조건을 결합하였다. 논리연산을 결합하는 것은 그렇게 어렵지 않을 것이다. 하지만 논리연산을 대입할 때 주의할 점은 AND 와 OR 가 쓰일 때 주의하여야 한다. 다음 예제를 보자.
emp 테이블에서 deptno 가 20 이고 sal 이 2000 이상이거나 hiredate 가 82년 01월 01일 이전인 사원을 조회해 보자.

쿼리문

```
SELECT * FROM emp
WHERE deptno = 20 AND sal >= 2000 OR hiredate < '82/01/01';
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	80/12/17	800	(null)	20
2	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
3	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30
4	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
5	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
6	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
7	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
8	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
9	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
10	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
11	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
12	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20

출력 결과를 보면 무언가 이상하다. 분명히 deptno 가 20인 데이터를 출력하려고 했는데 deptno 가 20 이 아닌 데이터가 출력되는 현상이 발생했다.
이유는 WHERE 절은 앞에서부터 순차적으로 논리연산을 진행하기 때문에 앞의 AND 연산을 수행 후

그 연산결과와 OR를 수행하여 dpetno 가 20 이고 sal 이 2000 이상이거나 hiredate 가 82년 01월 01일인 데이터를 출력하라는 의미가 된 것이다.
그렇다면 우리가 원하는 deptno 가 20 이고 sal 이 2000 이상이거나 hiredate 가 82년 01월 01일 이전인 사원은 어떻게 하면 출력할 수 있을까? 아래와 같이 '( )' 를 활용하여 논리 연산의 우선 순위를 사용자가 지정해 주면 된다. () 가 있으면 () 안에 있는 연산을 먼저 실행하기 때문이다.

쿼리문

```
SELECT * FROM emp
WHERE deptno = 20 AND (sal >= 2000 OR hiredate < '82/01/01');
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	80/12/17	800	(null)	20
2	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
3	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
4	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20

위 쿼리문은 OR연산을 먼저 실행하기 위해 OR문을 '( )' 에 넣고 이후에 AND 를 실행하기 때문에 우리가 원하는 데이터를 출력할 수 있다. AND 와 OR을 같이 사용할 경우에는 엉뚱한 데이터가 조회될 수 있으니 주의해서 작성하길 바란다.

2.3 WHERE 절에서 사용할 수 있는 기타 연산자

연산자	설 명
BETWEEN 조건 A AND 조건 B	a 와 b 사이의 데이터를 출력(a, b 포함)
IN (list)	list 의 값 중 어느 하나와 일치하는 데이터를 출력
LIKE	문자 형태로 일치하는 데이터를 출력(%, _사용)
IS NULL	NULL 값을 가진 데이터를 출력
NOT BETWEEN a AND b	a 와 b 사이에 있지 않은 데이터를 출력(a, b 포함하지 않음)
NOT IN (list)	list 의 값과 일치하지 않는 데이터를 출력
NOT LIKE	문자 형태와 일치하지 않는 데이터를 출력
IS NOT NULL	NULL 값을 갖지 않는 데이터를 출력

2.3.1 IN, NOT IN 연산자

비교할 값이 여러 개일 경우 사용하는 비교 연산자로 주로 나중에 공부할 조인이나 서브쿼리에서 많이 사용한다. 단일 값일 때 사용해도 상관은 없다.

emp 테이블에서 dpetno 가 20, 30 인 사원들의 정보를 조회해 보자.

쿼리문

```
SELECT * FROM emp
WHERE   deptno IN (20, 30);
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	80/12/17	800	(null)	20
2	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
3	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30
4	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
5	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
6	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
7	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
8	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
9	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
10	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
11	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20

조회 결과를 보면 AND 를 사용하지 않고 deptno 의 두 개값 (20, 30) 을 만족하는 사원 데이터가 조회되었다. 이렇게 컬럼 값과 비교할 대상이 여러 개일 경우 IN 을 쓰면 된다. 그리고 만일 복수의 값을 제외하고 싶을 때는 아래와 같이 IN 앞에 NOT 만 넣어주면 된다. 부정 쿼리문은 각자 쿼리문을 작성한 후 실행해 보자.

쿼리문

```
SELECT * FROM emp
WHERE   deptno NOT IN (20, 30);
```

실행해보면 deptno 값이 10인 사원만 조회되는 것을 확인할 수 있다.

2.3.2 BETWEEN, NOT BETWEEN 연산자

BETWEEN 연산자는 일정한 범위를 조회할 때 사용하는 연산자이다.
컬럼명 BETEWEEN 조건A AND 조건B 형식으로 조건A 에서 조건B 까지 포함되는 데이터를 조회하는 것이다.

empno 가 7698 에서 7902 까지의 사원을 조회해 보자.

쿼리문

```
SELECT * FROM emp
WHERE   empno BETWEEN 7698 AND 7902;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
2	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
3	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
4	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
5	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
6	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
7	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
8	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20

결과를 보면 범위에 조건A와 조건B가 포함되는 것을 알 수가 있다. 이를 기억할 수 있도록 하자. 그리고 이 BETWEEN 연산자는 AND 연산으로 바꿔서 아래와 같이 쿼리문으로 작성할 수 있다.

쿼리문

```
SELECT * FROM emp
WHERE   empno >= 7698 AND empno <=7902;
```

1980년이 아닌 해에 입사한 직원을 조회해 보자.

쿼리문

```
SELECT   *
FROM   EMP
--WHERE HIREDATE NOT BETWEEN '1980/01/01' AND '1980/12/31';
WHERE   NOT   HIREDATE   BETWEEN '1980/01/01' AND '1980/12/31';
```

2.3.3 LIKE, NOT LIKE 연산자

문자열 값이 지정하는 문자열패턴을 만족하면 TRUE 를 리턴하여 선택하도록 하는 연산자이다.
문자열패턴 지정을 위해 와일드카드 문자를 사용한다.

- **% (Percentage)** : 0개 이상의 임의의 문자열을 의미한다.
- **_ (Underscore) :** 문자 1개를 의미한다.

emp 테이블에서 이름에 K가 들어가는 사원의 empno 와 ename 을 조회해 보자.

쿼리문

```
SELECT empno, ename FROM emp
WHERE   ename LIKE '%K%';
```

결 과

	EMPNO	ENAME
1	7698	BLAKE
2	7782	CLARK
3	7839	KING

emp 테이블에서 이름에 K로 시작하는 사원의 empno 와 ename 을 조회해 보자.

쿼리문

```
SELECT empno, ename FROM emp
WHERE   ename LIKE 'K%';
```

결 과

	EM...	ENAME
1	7839	KING

emp 테이블에서 이름에 K 로 끝나는 사원의 empno 와 ename 을 조회해 보자.

쿼리문

```
SELECT empno, ename FROM emp
WHERE   ename LIKE '%K';
```

결 과

	EMPNO	ENAME
1	7782	CLARK

emp 테이블에서 이름에 두 번째 문자가 A인 사원의 empno 와 ename 을 조회해 보자.

쿼리문

```
SELECT empno, ename FROM emp
WHERE   ename LIKE '_A%';
```

결 과

	EMPNO	ENAME
1	7521	WARD
2	7654	MARTIN
3	7900	JAMES

- **Escape Option :** 와일드카드('%', '_') 문자를 기록 값으로 가진 데이터에서 기록값과 와일드카드 문자를 구분하기 위해 사용한다.

사원명에 '_' 문자 앞 글자가 두 글자인 직원 정보를 조회해 보자.

쿼리문

```
SELECT  *
FROM  EMP
WHERE  ENAME  LIKE  '__\_%'  ESCAPE  '\';
```

사원명에 마지막 글자가 '%P'인 직원 정보를 조회해 보자.

쿼리문

```
SELECT  *
FROM  EMP
WHERE  ENAME  LIKE  '%#%P'  ESCAPE  '#';
```

LIKE 연산자는 그렇게 어려운 개념이 아니기 때문에 한번씩만 연습하면 이해가 될 것이라고 생각된다. 혹시 이해가 되지 않았다면 스스로 몇 번 더 연습해 본다면 이해하는 것이 그렇게 어렵지 않을 것이다.
LIKE 부정문은 LIKE 앞에 NOT을 넣으면 되고 LIKE 문을 반대되는 데이터를 조회한다.
위 예제를 기준으로 LIKE 앞에 NOT을 넣어 실습해 보고 이해하도록 하자.

이름에 'A'가 없는 직원 정보를 조회해 보자

쿼리문

```
SELECT  *
FROM  EMP
--WHERE  ENAME  NOT  LIKE  '%A%';
WHERE  NOT  ENAME  LIKE  '%A%';
```

2.3.4 IS NULL, IS NOT NULL 연산자

데이터 값에는 NULL 값을 가질 수가 있는데 NULL 값이라는 것은 0이 아니고 빈 공간도 아니며, 미확정 / 알 수 없는 값을 의미한다. 그리고 NULL 값은 연산, 할당, 비교가 불가능하며 '=', 'IN' 연산자를 이용하면 조회되지 않는다. 위와 같은 NULL값의 특징은 기억할 수 있도록 하자.

emp 데이블에서 comm 데이터가 NULL 인 것을 조회해 보자.

쿼리문

```
SELECT * FROM emp
WHERE   comm IS NULL;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	80/12/17	800	(null)	20
2	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
3	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
4	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
5	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
6	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
7	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
8	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10

NULL 값을 조회할 때는 IS 라는 연산자를 사용한다는 것을 꼭 기억하자.
NULL이 아닌 데이터를 찾을 때에는 IS 다음에 NOT을 넣어 사용하면 된다.

쿼리문

```
SELECT * FROM emp
WHERE   comm IS NOT NULL;
```

지금까지 WHERE 절과 WHERE 절에서 사용하는 연산자에 대해서 알아봤다. 원하는 데이터를 조회할 때 WHERE절에 조건을 연산자를 통해 만들어 조회를 하기 때문에 매우 중요하다. 또한 앞으로도 WHERE절은 많이 사용하기 때문에 반드시 숙지해야 한다. 가장 좋은 방법은 다양한 방법으로 연산자를 활용하여 WHERE절을 만들어 보고 반복하는 것이다.

3 정렬을 위한 ORDER BY 절

ODER BY절은 특정컬럼의 데이터를 중심으로 오름차순이나 내림차순으로 정렬하는 기능을 하는 절이다. 쿼리문의 형식은 다음과 같다.

```
SELECT 컬럼명, 컬럼명1 FROM 테이블명
WHERE 칼럼이름 연산자 조건값
ODER BY 컬럼명 정렬방식(ASC, DESC);
```

ORDER BY 절은 쿼리문의 맨 끝에 위치하면 된다. 정렬의 기준이 될 컬럼명을 ORDER BY뒤에 넣고 정렬방식을 작성하면 된다. 정렬방식은 오름차순(ASC)과 내림차순(DESC) 정렬이 있다. 정렬방식을 명시하지 않으면 오름차순이 기본값이다

※ 오름차순 : 작은 값이 위에 출력되고 아래로 갈수록 점차 큰 값이 출력된다
※ 내림차순 : 큰 값이 위에 출력되고 아래로 갈수록 점차 작은 값이 출력된다.

각 데이터형별 기준은 다음과 같다.

구분	오름차순(ASC)	내림차순(DESC)
숫자	작은 값부터 정렬	큰 값부터 정렬
문자	사전 순서대로 정렬	사전 순서 반대로 정렬
날짜	빠른 날짜 순서로 정렬	늦은 날짜 순서로 정렬
NULL	가장 마지막에 나옴	가장 먼저 나옴

기본적인 내용을 확인하였으니 ORDER BY 절에 대한 예제를 보면서 이해해보자.

emp 테이블에서 sal 을 오름차순으로 정렬하여 조회해 보자.

쿼리문

```
SELECT * FROM emp
ORDER BY sal; // ASC넣어도 되고 생략해도 된다.
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	80/12/17	800	(null)	20
2	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
3	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
4	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30
5	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
6	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
7	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
8	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
9	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
10	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
11	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
12	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
13	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20

쿼리문을 실행결과 sal 컬럼이 오름차순으로 정렬된 것을 확인할 수 있다. 만일 sal 컬럼 기준 정렬후 sal 의 같은 값에 대해 다른 컬럼을 기준으로 정렬하고 싶다면 아래와 같이 쿼리문을 작성하면 된다.

쿼리문

```
SELECT * FROM emp
ORDER BY sal, ename;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	80/12/17	800	(null)	20
2	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
3	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
4	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
5	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30
6	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
7	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
8	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
9	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
10	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
11	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
12	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
13	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20

sal 값이 같은 ename 컬럼의 'FORD'와 'SCOTT'을 보면 먼저 한 실습에선 SCOTT이 12번 행에 있었는데 ORDER BY 절에 ename 에 대한 정렬을 추가한 결과 13번 행으로 두 행의 위치가 바뀐 것을 확인할 수 있다.

이것을 보면 ORDER BY 절에 ' , ' 로 구분하여 두 개의 정렬기준을 기입하는 경우 첫 번째 컬럼으로 먼저 정렬을 하고 중복된 값에 대해서는 두 번째 컬럼을 기준이 정렬이 된다는 것을 알 수 있다.

만일 첫 번째 정렬과 두 번째 정렬을 다르게 하고 싶다면 아래와 같이 쿼리문을 작성하면 된다.

쿼리문

```
SELECT * FROM emp
ORDER BY sal DESC, ename ASC;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
2	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
3	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
4	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
5	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
6	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
7	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
8	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
9	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
10	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
11	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30

sal 은 내림차순으로 정렬이 되고 ename 은 오름차순으로 정렬된 것을 확인할 수 있다. 이처럼 정렬을 할 때 각각의 컬럼에 대해 정렬기준을 별도로 지정해 줄 수 있다.

order by 를 적용할 컬럼의 표현은 컬럼명, 컬럼별칭, 컬럼순번 총 3가지로 표현할 수 있다.
기본으로 컬럼명을 정렬기준으로 사용한 위의 예제는 아래와 같이 컬럼의 나열순서를 통해서도 표현 가능하다. 6번째 컬럼 sal 과 2번째 컬럼 ename 을 ORDER BY절 조건으로 사용한다.

쿼리문

```
SELECT * FROM emp
ORDER BY 6 DESC, 2 ASC;
```

결 과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
2	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
3	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
4	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
5	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
6	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
7	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
8	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
9	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
10	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
11	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30

아래와 같이 별칭으로 정렬해보자.

쿼리문

```
SELECT empno 사원번호, ename 사원명, sal 월급, hiredate 입사일
FROM emp
ORDER BY 월급 DESC, 사원명 ASC;
```

결 과

	사원번호	사원명	월급	입사일
1	1010	BOB	(null)	(null)
2	1011	EDWARD	(null)	(null)
3	1006	JORDAN	5000	82/02/05
4	7839	KING	5000	81/11/17
5	7902	FORD	3000	81/10/03
6	7788	SCOTT	3000	82/10/09
7	7566	JONES	2975	81/04/02
8	7698	BLAKE	2850	81/04/01
9	7782	CLARK	2450	81/06/01
10	7499	ALLEN	1600	81/02/20
11	1003	MATT	1500	16/01/01
12	7844	TURNER	1500	81/09/08
13	7934	MILLER	1300	82/01/23
14	7654	MARTIN	1250	81/09/28
15	7521	WARD	1250	81/02/22
16	7876	ADAMS	1100	83/01/12
17	7900	JAMES	950	81/10/03
18	1001	ALICE	800	16/09/08
19	1002	MORRIS	800	16/09/08
20	7369	SMITH	800	16/12/19

위의 예제처럼 별칭을 사용하여 ORDER BY 절을 수행하였다.
별칭에는 한글, 영어 모두 사용 가능하다. 다만 띄어쓰기가 있을 경우 반드시 쌍따옴표(")로 묶어야 한다.

4 연결 연산자 ||

```
SELECT 컬럼명 || '문자 리터럴' || 컬럼명
FROM 테이블명;
```

SELECT 문 조회시에 컬럼 사이에 특정한 문자를 넣고 싶을 때 사용하는 연산자이다.
간단한 연산자이니 예제보고 이해하고 넘어가도록 하자.

SELECT 문으로 조회하여 결과가 'SMITH JOB is CLERK' 라고 조회하고 컬럼명은 EMPLOYEE 라고 출력해 보자.

쿼리문

```
SELECT ename || ''s JOB is ' || job AS EMPLOYEE
FROM emp
```

결 과

	EMPLOYEE
1	SMITH's JOB isCLERK
2	ALLEN's JOB isSALESMAN
3	WARD's JOB isSALESMAN
4	JONES's JOB isMANAGER
5	MARTIN's JOB isSALESMAN
6	BLAKE's JOB isMANAGER
7	CLARK's JOB isMANAGER
8	SCOTT's JOB isANALYST
9	KING's JOB isPRESIDENT
10	TURNER's JOB isSALESMAN

그렇게 어려운 구문이 아니니 쉽게 이해할 수 있을 것이라고 생각된다.
한가지 주의할 점은 ' ' 안의 문자열에 특수문자를 넣으려면 ' 를 더 추가해야 한다. 그래서 위 쿼리문에 ' 를 출력하기 위해 '' 를 넣었다.

쿼리문

```
SELECT ename || '의 급여는 ' || sal || ' 만원 입니다.'
FROM emp;
```

결 과

	ENAME||'의급여는'||SAL||'만원입니다.'
1	SMITH의 급여는 800 만원 입니다.
2	ALLEN의 급여는 1600 만원 입니다.
3	WARD의 급여는 1250 만원 입니다.
4	JONES의 급여는 2975 만원 입니다.
5	MARTIN의 급여는 1250 만원 입니다.
6	BLAKE의 급여는 2850 만원 입니다.
7	CLARK의 급여는 2450 만원 입니다.
8	SCOTT의 급여는 3000 만원 입니다.
9	KING의 급여는 5000 만원 입니다.
10	TURNER의 급여는 1500 만원 입니다.
11	ADAMS의 급여는 1100 만원 입니다.
12	JAMES의 급여는 950 만원 입니다.
13	FORD의 급여는 3000 만원 입니다.
14	MILLER의 급여는 1300 만원 입니다.

Chapter 2. 함수

1 DUAL 테이블

SYS 계정에서 제공하는 테이블로 함수나 계산식을 테이블 참조 없이 실행해보기 위해 FROM 절에 사용하는 더미(DUMMY) 테이블로 가상의 테이블로 생각하면 된다.
함수나 계산식의 결과를 한 행으로 볼 수 있게 해준다.
dual 테이블의 구조와 SELECT 문으로 조회 시에 아래와 같은 결과가 나온다.

쿼리문

```
DESC SYS.DUAL;
SELECT * FROM DUAL;
```

결 과

```
이름    널 유형
----- - ----------
DUMMY   VARCHAR2(1)
```

	DUMMY
1	X

dual 테이블에는 하나의 DUMMY 컬럼이 VARCHAR2(1) 형으로 제공되고 SELECT 문으로 조회를 하게 되면 X라는 값만 확인할 수 있다. DUAL 테이블이 유용한 건 사용자 테이블에서 산술연산을 수행하면 똑같은 값이 테이블에 있는 데이터 수 만큼 나오게 되지만 DUAL테이블은 단 한 개의 행만 조회되어 출력한다.

쿼리문

```
SELECT 20 * 30 FROM emp;
```

결 과

	10*20
1	200
2	200
3	200
4	200
5	200
6	200
7	200
8	200
9	200
10	200
11	200
12	200
13	200
14	200

쿼리문

```
SELECT 20 * 30 FROM dual;
```

결 과

	10*20
1	200

오라클에서 제공하는 함수의 출력 결과도 dual 테이블을 이용하면 결과만 쉽게 확인할 수 있다

쿼리문

```
SELECT MOD(5, 4) FROM dual; -- 5/4의 나머지를 구하는 함수
```

결 과

	MOD(5,4)
1	1

2 단일행 함수

여러 건의 데이터를 한번에 하나씩 데이터를 처리하는 함수로 테이블 조회시 각 행에 대한 처리 결과가 출력된다. 단일행 함수의 종류에는 문자함수, 숫자함수, 날짜함수, 형변환함수, 기타함수가 있다.

2.1 숫자 함수

수학적 계산식을 처리하기 위한 함수를 의미한다.

이름	의미	사용 예
ABS	주어진 숫자의 절대 값을 출력	ABS(-12) -> 12
ROUND	주어진 숫자를 반올림한 후 출력	ROUND(12.345, 2) -> 12.34
TRUNC	주어진 숫자를 버림 후 출력	TRUNC(12.345, 2) -> 12.34
MOD	주어진 숫자를 나누기한 후 나머지 값 출력	MOD(12, 10) -> 2
CEIL	주어진 숫자와 가장 근접한 큰 정수 출력	CEIL(12.345) -> 13
FLOOR	주어진 숫자와 가장 근접한 작은 정수 출력	FLOOR(12.345) -> 12
POWER	주어진 숫자1의 숫자2 제곱 출력	POWER(3, 2) -> 9

2.1.1 ABS : 절대값을 구하는 함수

-10의 절대값을 구해보자.

쿼리문

```
SELECT -10, ABS(-10) FROM dual;
```

결 과

	-10	ABS(-10)
1	-10	10

2.1.2 ROUND : 반올림한 결과를 반환하는 함수

- 반올림할 위치를 지정할 수 있다
- 자릿수를 지정하지 않으면 소수점 첫째 자리 반올림
- 음수로 자릿수를 지정할 경우 일->십->백 순의 자릿수에서 반올림

위의 세 가지 상황에 대해 함수로 출력해 보자.

쿼리문

```
SELECT   12.3456, ROUND(12.3456),
         ROUND(12.3456, 2), ROUND(12.3456, -1)
FROM dual;
```

결 과

	12.3456	ROUND(12.3456)	ROUND(12.3456,2)	ROUND(12.3456,-1)
1	12.3456	12	12.35	10

2.1.3 TRUNC : 버림을 한 결과를 반환하는 함수

- 버림할 자릿수를 지정할 수 있다
- 자릿수를 지정하지 않으면 소수점 첫째 자리에서 버림 (기본값이 0 임)
- 음수일 경우 일->십->백 순의 자릿수에서 버림

위의 4가지 상황에 대해 함수로 출력해 보자.

쿼리문

```
SELECT   12.3456, TRUNC(12.3456), TRUNC(12.3456, 2),
         TRUNC(12.3456, -1), TRUNC(12.3456, 0)
FROM dual;
```

결 과

	12.3456	TRUNC(12.3456)	TRUNC(12.3456,2)	TRUNC(12.3456,-1)	TRUNC(12.3456,0)
1	12.3456	12	12.34	10	12

2.1.4 MOD : 나누기한 후 나머지를 출력하는 함수

MOD(A, B)의 두 개의 매개변수(정수)를 전달해야 하는데 A는 대상 수, B는 나눌 수를 넣으면 된다.

21/5의 나머지를 구해보자.

쿼리문

```
SELECT   MDO(21, 5)   FROM   dual;
```

결 과

	MOD(21,5)
1	1

2.1.5 CEIL

CEIL(실수A) 넣으면 실수A 기준으로 가장 근접한 큰 값의 정수를 반환한다.

예제를 보자.

쿼리문

```
SELECT CEIL(12.3456) FROM dual;
```

결 과

	CEIL(12.3456)
1	13

2.1.6 FLOOR

FLOOR(실수A) 실수A 기준으로 가장 근접한 작은 정수 값을 반환하는 함수이다.

다음 예제를 보자.

쿼리문

```
SELECT CEIL(12.3456) FROM dual;
```

결 과

	FLOOR(12.3456)
1	12

2.1.7 POWER

POWER(정수A, 정수B) 정수A를 정수B만큼 제곱하는 함수이다.

예제를 보자.

쿼리문

```
SELECT POWER(3,4) FROM dual;
```

결 과

	POWER(3,4)
1	81

2.2 문자 함수

문자 데이터를 처리하기 위한 함수이다.

이름	의미
UPPER	대문자로 변환
LOWER	소문자로 변환
INITCAP	이니셜만 대문자로 변환
LENGTH	문자 길이 반환
LENGTHB	바이트 수 반환
INSTR	특정 문자의 위치 반환
SUBSTR	대상의 시작 위치부터 선택 개수만큼 추출
SUBSTRB	대상의 시작 위치부터 선택 바이트만큼 추출
LPAD/RPAD	빈칸만큼 특정문자로 채움
LTRIM/RTRIM	공백문자 삭제
TRIM	특정 문자 잘라내기

2.2.1 UPPER

대문자로 변환하는 함수로 UPPER(값 또는 컬럼명)을 넣으면 값 또는 해당 컬럼에 기록된 문자값이 모두 대문자로 반환된다.

예제를 보자.

쿼리문

```
SELECT UPPER('sam') FROM dual;
```

결 과

	UPPER('SAM')
1	SAM

쿼리문

```
SELECT UPPER(ename) FROM EMP;
```

2.2.2 LOWER

소문자로 변환하는 함수로 LOWER(값 또는 컬럼명)을 넣으면 값 또는 해당 컬럼에 기록된 문자열값이 모두 소문자로 반환된다.

예제를 보자.

쿼리문

```
SELECT LOWER('ORACLE') FROM dual;
```

결 과

	LOWER('ORACLE')
1	oracle

쿼리문

```
1 .SELECT ename FROM emp;
2. SELECT LOWER(ename) FROM dual;
```

결 과

1

	ENAME
1	SMITH
2	ALLEN
3	WARD
4	JONES
5	MARTIN
6	BLAKE

2

	LOWER(ENAME)
1	smith
2	allen
3	ward
4	jones
5	martin
6	blake

2.2.3 INITCAP

단어의 앞 글자만 대문자로 변환해주는 함수로 INITCAP(값 또는 컬럼명)을 넣으면 값 또는 해당 컬럼에 기록된 문자열 값의 첫 글자를 대문자로 변환하여 반환한다.

예제를 보자.

쿼리문

```
SELECT INITCAP('kim mal ddong') FROM dual;
```

결 과

	INITCAP('KIMMALDDONG')
1	Kim Mal Ddong

쿼리문

```
SELECT ename FROM emp;
SELECT INITCAP(ename) FROM emp;
```

2.2.4 LENGTH / LENGTHB

LENGTH는 LENGTH(문자열A 또는 문자형컬럼) 형식으로 사용하고, 문자열A의 길이, 컬럼에 기록된 문자열값의 길이를 반환하는 함수이다.

LENGTHB는 LENGTHB(문자열A 또는 문자형컬럼)형식으로 사용하고 문자열A 또는 컬럼에 기록된 문자열값의 바이트 수를 반환한다.

쿼리문

```
SELECT LENGTH('test11'), LENGTHB('test11') FROM dual;
```

결 과

	LENGTH('TEST11')	LENGTHB('TEST11')
1	6	6

쿼리문

```
SELECT name, LENGTH(name), LENGTHB(name) FROM test;
```

결 과

	NAME	LENGTH(NAME)	LENGTHB(NAME)
1	adem tailer	11	22
2	hong gil dong	13	26
3	lee soon sin	12	24
4	henry kim	9	18
5	ASDF	4	8
6	홍길동	3	6
7	이순신	3	6
8	임꺽정	3	6

조회결과를 보면 LENGTH는 글자수만을 숫자로 보여주고 LENGTHB는 BYTE수를 의미하는데 6~8까지는 한글이 포함되어 있어 글자수가 2Byte씩 차지하게 된다 그 결과값이 두배가 된다. 만일 name에 영

문 또는 숫자만 있는 경우에는 영문 또는 숫자는 1Byte만 차지 하므로 글자 수와 같게 출력된다.

2.2.5 INSTR / INSTRB

INSTR은 문자 또는 컬럼에 포함된 문자의 위치를 찾는 함수로 INST(문자 또는 컬럼, '검색문자',검색시작위치, 발견번째수)로 사용하는데 검색시작위치, 발변번째 수는 생략할 수 있고, 생략하게 되면 1로 값이 자동으로 들어가 첫 번째부터 검색하여 첫 번째 발견문자의 위치를 반환하고, 만약 인자값을 원하는 위치부터 시작하여 원하는 번째 문자를 검색할 때는 각 인자값에 값을 넣으면 된다.

쿼리문

```
SELECT INSTR('ORACLE WELCOME','O') FROM dual;
```

결 과

	INSTR('ORACLEWELCOME','O')
1	1

위 예제는 시작위치 및 발견번째수를 입력하지 않아 1번째 글자부터 검색하여 첫 번째 번 검색된 값의 위치를 반환하여 'ORACLEWELCOME'이라는 문자중 'O'RALCE의 O위치를 반환하여 1이라는 반환값이 나온 것이다.

쿼리문

```
SELECT INSTR('ORACLE WELCOME','O',2,1) FROM dual;
```

결 과

	INSTR('ORACLEWELCOME','O',2,1)
1	12

위 예제는 시작위치를 2 및 발견번째수를 1번으로 인자값을 넣었기 때문에 검색의 시작을 ORACLE에서 'R'에서 시작하여 첫 번째로 나오는 검색문자를 반환하는 것이라서 'WELCOME'에서의 O를 검색하여 12번째라는 값을 반환하는 것이다.

INSTRB는 바이트로 글자를 반환합니다. 한글로 할 때 한 글자에 2바이트로 보고 계산합니다. 위 예제를 INSTRB로 변경하여 테스트 해보고 결과값을 확인하면 알 수 있다..

2.2.6 SUBSTR/SUBSTRB

SUBSTR은 대상 문자열이나 컬럼의 자료에서 시작위치부터 선택 개수만큼의 문자를 반환하는 함수이다. SUBSTR(대상, 시작위치, 반환할문자수)의 형식으로 사용하고 결과는 해당 문자가 반환할 개수만큼 반환된다. SUBSTR은 문자수, SUBSTRB 함수는 바이트수로 동작한다.

쿼리문

```
SELECT SUBSTR('오라클 데이터베이스',3) FROM dual;
```

결 과

	INSTR('ORACLEWELCOME','O')
1	1

위 예제는 대상문자열에서, 시작위치 3번째 글자부터 선택개수(생략시 나머지 문자열 모두를 선택)만큼 반환한다.

쿼리문

```
SELECT SUBSTR('오라클 데이터베이스',3,5) FROM dual;
```

결 과

	SUBSTR('오라클데이터베이스',3,5)
1	클 데이터

위 예제는 대상문자열에서, 시작위치 3번째 글자부터 선택개수인 5글자를 반환한다.

쿼리문

```
SELECT SUBSTRB('오라클 데이터베이스',3) FROM dual;
```

결 과

	SUBSTRB('오라클데이터베이스',3)
1	라클 데이터베이스

위 예제는 대상문자열에서, 시작위치부터 세 번째 인자인 선택바이트수(생략시 나머지 문자열 모두를 선택)만큼 반환한다.

쿼리문

```
SELECT SUBSTRB('오라클 데이터베이스',4,6) FROM dual;
```

결 과

	SUBSTRB('오라클데이터베이스',4,6)
1	라클

위 예제는 대상문자열에서, 시작위치부터 세 번째 인자인 선택바이트수 직전까지 반환한다. Oracle 11g Express Edition의 경우 한글 한글자당 3바이트처리하므로 주의해야 한다. (다른 Standard Edition, Enterprise Edition의 경우는 한글 한글자당 2바이트 처리한다.)

2.2.7 LPAD/RPAD

기준 공간 칸수를 지정하고 빈칸만큼 특정문자로 채우는 함수

10칸을 기준으로하고 빈칸만큼 '+'로 채우기

쿼리문

```
SELECT LPAD('ORACLE', 10, '+') FROM dual;
```

결 과

	LPAD('ORACLE',10,'+')
1	++++ORACLE

위 예제는 첫번째 인자인 문자열의 개수(6)를 두 번째 인자인 공간칸수에서 뺀 차만큼 세번째인자 문자열로 채운 것이다. 공간칸수보다 첫 번째 인자 문자열의 크기가 크다면, 넘치는 문자열은 제거된다.

쿼리문

```
SELECT RPAD('ORACLE', 10, '+') FROM dual;
```

결 과

	RPAD('ORACLE',10,'+')
1	ORACLE++++

위 예제는 첫번째 인자인 문자열의 개수(6)를 두 번째 인자인 공간칸수에서 뺀 차만큼을 오른쪽부터 세 번째인자 문자열로 채운 것이다. 공간칸수보다 첫 번째 인자 문자열의 크기가 크다면, 넘치는 문자열은 제거된다.

2.2.8 LTRIM/RTRIM

왼쪽 또는 오른쪽의 공백문자를 제거해주는 함수

쿼리문

```
SELECT ' ORACLE ',
  LTRIM(' ORACLE '),      -- 왼쪽 공백 제거
  RTRIM(' ORACLE '),      -- 오른쪽 공백 제거
FROM dual;
```

결 과

	' ORACLE '	LTRIM(' ORACLE ')	RTRIM(' ORACLE ')
1	ORACLE	ORACLE	ORACLE

2.2.9 TRIM

양끝의 특정 문자를 제거하는 함수로, 특정 문자를 지정하지 않으면 양쪽 공백 제거한다.

양쪽 끝의 'a'문자 제거하여 조회

쿼리문

```
SELECT TRIM('a' FROM 'aaaOracleaaa') FROM dual;
```

결 과

	TRIM('A'FROM'AAAORACLEAAA')
1	Oracle

2.3 날짜 함수

문자 데이터를 처리하기 위한 함수

이름	의미
SYSDATE	함수가 아님. 시스템의 현재 날짜와 시간을 나타내는 값
MONTHS_BETWEEN	두 날짜 사이의 개월 수
ADD_MONTHS	주어진 날짜에 개월 수를 더함
NEXT_DAY	주어진 날짜 기준으로 다음날짜 출력
LAST_DAY	주어진 날짜가 속한 달의 마지막 날짜 출력

2.3.1 SYSDATE

함수가 아니며 시스템의 현재 날짜와 시간을 나타낸다

SYSDATE를 출력

쿼리문

```
SELECT SYSDATE FROM dual;
```

결 과

	SYSDATE
1	17/08/28

위와 같이 SYSDATE를 변환없이 출력하면 날짜만 출력하게 된다 시간도 함께 출력하기 위해서는 변환 함수를 이용해야 한다

SYSDATE를 변환함수 TO_CHAR를 이용해 시분초 까지 출력

쿼리문

```
SELECT TO_CHAR(SYSDATE, 'yyyy/mm/dd hh24:mi:ss') FROM dual;
```

결 과

	TO_CHAR(SYSDATE,'YYYY/MM/DDHH24:MI:SS')
1	2017/08/28 16:05:38

2.3.2 MONTHS_BETWEEN

두 날짜 사이의 개월 수

큰 날짜가 먼저 와야 양수 결과가 나오며 윤달(2월 29일)을 구분하지 못 한

두 날짜 간의 개월 수 차이를 출력

쿼리문

```
SELECT TO_CHAR(SYSDATE, 'yyyy/mm/dd hh24:mi:ss') FROM dual;
```

결 과

	MONTHS_BETWEEN('16/10/01','17/2/8')	MONTHS_BETWEEN('17/2/8','16/10/01')
1	-4.225806451612903225806451612903225806451	4.225806451612903225806451612903225806451

2.3.3 ADD_MONTHS

주어진 날짜에 개월 수를 더함

날짜 '17/1/18'에 2개월을 더한 날짜 출력

쿼리문

```
SELECT ADD_MONTHS('17/1/18', '2') FROM dual;
```

결 과

	ADD_MONTHS('17/1/18','2')
1	17/03/18

2.3.4 NEXT_DAY

주어진 날짜에서 가장 가까운 요일의 날짜를 반환

'17/1/9'는 월요일이므로 가장 최근의 목요일인 '17/1/12'을 출력

쿼리문

```
SELECT NEXT_DAY('17/8/28','금') FROM dual;
```

결 과

	NEXT_DAY('17/8/28','금')
1	17/09/01

2.3.5 LAST_DAY

주어진 날짜가 속한 달의 마지막 날짜를 반환

17년 2월의 마지막 날짜 출력

쿼리문

```
SELECT LAST_DAY('17/8/28') FROM dual;
```

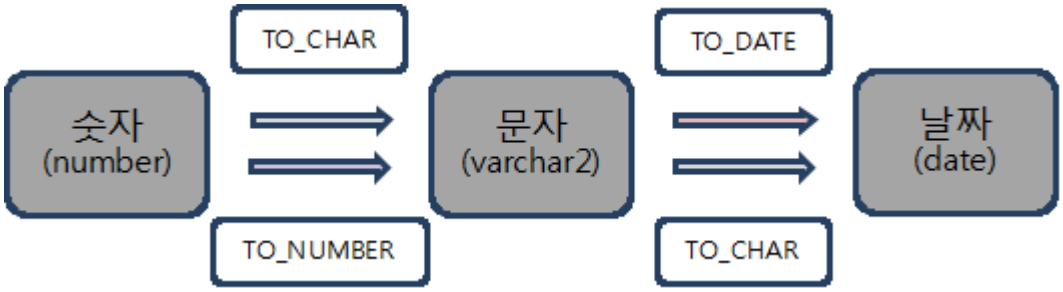
결 과

	LAST_DAY('17/8/28')
1	17/08/31

2.4 변환 함수

숫자, 문자, 날짜 데이터타입을 서로의 데이터타입으로 변환하는 함수

오라클의 데이터 타입 변환은 묵시적 형 변환(자동)과 명시적 형 변환(수동) 두 가지 방법이 있다



이름	의미
TO_CHAR	날짜형과 숫자형을 문자형으로 형 변환

TO_DATE	문자형을 날짜형으로 형 변환
TO_NUMBER	문자형을 숫자형으로 형 변환

2.4.1 TO_CHAR

DATE 타입, NUMBER 타입을 원하는 형식의 VARCHAR2 타입으로 변환한다

```
TO_CHAR(날짜, '원하는 형식')
```

숫자를 문자로 변환 시 사용하는 형식 요소

형식	의미
9	일반적인 숫자
0	앞의 빈자리를 0으로 채움
$	'$'(dollar) 표시를 나타냄
L	지역 통화 단위를 표시
.	소수점을 표시함
,	천단위를 표시함

날짜를 문자로 변환 시 사용하는 형식 요소

형식	의미
SCC	세기를 표시 S는 기원전(BC)
YEAR	연도를 알파벳(영어)로 표시
YYYY	4자리 연도로 표시
YY	끝 2자리 연도 표시
MONTH	월을 표시

MON	월의 약자로 표시(한글일 경우 MONTH와 다르지 않음)
MM	월을 2자리 숫자로 표시
DAY	날짜에 해당하는 요일 표시
DY	날짜에 해당하는 요일을 약자로 표시
DDD	연도 기준으로 몇 번째 날 인지 표시(1-366)
DD	월 기준으로 몇 번째 날 인지 표시(1-31)
D	주 기준으로 몇 번째 날 인지 표시(1-7), 요일을 나타냄(일요일부터 1로 시작)
HH, HH24	12시간 기준, 24시간 기준으로 시간을 표시
MI	분을 표시
SS	초를 표시
AM, PM	오전이나 오후를 표시

* 문자를 날짜로 변환할 때도 같은 형식을 사용한다

변환 함수 예제

표현	결과
TO_CHAR(SYSDATE,'YYYY/MM/DD HH24:MI:SS')	2017/01/12 10:47:03
TO_CHAR(350000, '$999,999'	$350,000
TO_CHAR(SYSDATE, 'YY/MM/DD')	17/01/06
TO_DATE('10 1월 17', 'YYYY MON DD')	17/01/10

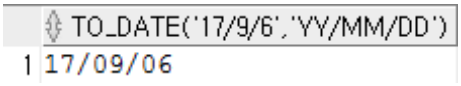
TO_NUMBER('1123')	1123

문자타입의 날짜를 날짜타입으로 변경해보자.

쿼리문

```
SELECT TO_DATE('17/9/6','YY/MM/DD') FROM dual;
```

결 과



2.5 기타 함수

이름	의미
NVL(exp1, exp2)	주어진 exp1이 NULL일 경우 exp2로 치환
NVL2(exp1, exp2, exp3)	exp1이 NULL이 아니면 exp2를 NULL이면 exp3로 치환
NULLIF(exp1, exp2)	exp1과 exp2의 값이 동일하면 NULL을 반환하고, 다를 경우 exp1을 반환
DECODE(exp1,exp2,exp3)	exp1이 exp2와 같으면 exp3를 출력

2.5.1 NVL

컬럼이 NULL일 경우 치환할 값을 지정하는 함수

emp 테이블(사원)의 comm 컬럼(성과금)의 NULL값을 0으로 치환

쿼리문

```
SELECT empno, ename, NVL(comm, 0) comm FROM emp;
```

결 과

	EMPNO	ENAME	COMM
1	7369	SMITH	0
2	7499	ALLEN	300
3	7521	WARD	200
4	7566	JONES	30
5	7654	MARTIN	300
6	7698	BLAKE	0
7	7782	CLARK	0
8	7788	SCOTT	0
9	7839	KING	3500
10	7844	TURNER	0
11	7876	ADAMS	0
12	7900	JAMES	0
13	7902	FORD	0
14	7934	MILLER	0

2.5.2 NVL2

NVL함수와 비슷하게 NULL인 경우를 판단하지만 NULL인지 아닌지에 따른 결과가 다르다

사원의 급여와 상여금을 NULL을 판단해 더한 결과 출력

쿼리문

```
SELECT empno, ename, nvl2(comm, sal+comm, sal) pay, sal, comm FROM emp;
```

결 과

	EMPNO	ENAME	PAY	SAL	COMM
1	7369	SMITH	800	800	0
2	7499	ALLEN	1900	1600	300
3	7521	WARD	1450	1250	200
4	7566	JONES	3005	2975	30
5	7654	MARTIN	1550	1250	300
6	7698	BLAKE	2850	2850	0
7	7782	CLARK	2450	2450	0
8	7788	SCOTT	3000	3000	0
9	7839	KING	8500	5000	3500
10	7844	TURNER	1500	1500	0
11	7876	ADAMS	1100	1100	0
12	7900	JAMES	950	950	0
13	7902	FORD	3000	3000	0
14	7934	MILLER	1300	1300	0

위 예제는 성과금컬럼(comm)이 null이 아니면, sal컬럼과 성과금을 더하고, null이면 sal컬럼값을 리턴한다.

2.5.3 NULLIF

두 값을 비교하여 동일한지 아닌지에 대한 결과 반환

두 값이 동일하면 NULL반환, 동일하지 않으면 첫 번째 값 반환

쿼리문

```
SELECT NULLIF(10, 10), NULLIF('A', 'B') FROM dual;
```

결 과

	NULLIF(10,10)	NULLIF('A','B')
1	(null)	A

위 예제에서 첫 번째 컬럼은 10과 10이 동일하므로, null을 리턴했고, 두번째 문자열 'A'와 'B'는 값이 다르므로, 첫 번째 인자값이 'A'를 리턴했다. 비교할 두 인자는 반드시 같은 데이터타입이어야한다.

2.5.4 DECODE

조건에 해당하는 값을 추출할 때 사용

조건이 TRUE일 경우와 FALSE일 경우 반환하는 값을 다르게 지정할 수 있으며 FALSE 부분을 연장하여 복잡한 조건을 줄 수 있다(프로그래밍 언어의 else if와 비슷)

쿼리문

```
SELECT DECODE(100, 100, 'EQUAL', 'UNEQUAL') FROM dual;
```

결 과

	DECODE(100,100,'EQUAL','UNEQUAL')
1	EQUAL

위 예제는 첫 번째 인자인 100과 두 번째 인자인 100이 동일하므로, 3번째 인자인 'EQUAL'문자열을 리턴하였다.

사원의 부서번호를 판단하는 조건을 지정해 부서명으로 치환

쿼리문

```
SELECT empno, ename, deptno,
        DECODE(deptno, 10, '회계팀', 20, '개발팀', 30, '영업팀', 40, '운영팀') deptname
FROM emp;
```

결 과

	EMPNO	ENAME	DEPTNO	DEPTNAME
1	7369	SMITH	20	개발팀
2	7499	ALLEN	30	영업팀
3	7521	WARD	30	영업팀
4	7566	JONES	20	개발팀
5	7654	MARTIN	30	영업팀
6	7698	BLAKE	30	영업팀
7	7782	CLARK	10	회계팀
8	7788	SCOTT	20	개발팀
9	7839	KING	10	회계팀
10	7844	TURNER	30	영업팀
11	7876	ADAMS	20	개발팀
12	7900	JAMES	30	영업팀
13	7902	FORD	20	개발팀
14	7934	MILLER	10	회계팀

위 예제는 각 행별로, deptno를 DECODE문의 10, 20, 30, 40과 비교해서 각각의 대응값인 '회계팀', '개발팀', '영업팀', '운영팀'을 리턴하도록 한 것이다.

2.5.5 CASE 문

일반함수 DECODE와 비슷한 처리를 할 수 있는 조건문 함수

DECODE함수가 제공할 수 없는 비교연산을 할 수 있다. DECODE함수를 이용해 비교연산을 하려면 비교연산을 하는 함수를 사용해야 하며 조건이 많아지면 DECODE함수의 내용이 길어져 알아보기 힘들어진다. CASE ~ WHEN THEN ~ END 로 구성한다

사원의 부서번호를 판단하는 조건을 지정해 부서명으로 치환

DECODE함수의 예제를 CASE문으로 표현한 것

쿼리문

```
SELECT DECODE(100, 100, 'EQUAL', 'UNEQUAL') FROM dual;
```

결 과

	EMPNO	ENAME	DEPTNO	DEPTNAME
1	7369	SMITH	20	개발팀
2	7499	ALLEN	30	영업팀
3	7521	WARD	30	영업팀
4	7566	JONES	20	개발팀
5	7654	MARTIN	30	영업팀
6	7698	BLAKE	30	영업팀
7	7782	CLARK	10	회계팀
8	7788	SCOTT	20	개발팀
9	7839	KING	10	회계팀
10	7844	TURNER	30	영업팀
11	7876	ADAMS	20	개발팀
12	7900	JAMES	30	영업팀
13	7902	FORD	20	개발팀
14	7934	MILLER	10	회계팀

3 그룹 함수

단일 행 함수는 행 마다 결과를 출력하지만 그룹 함수는 행들을 그룹화하여 하나의 도출된 결과를 출력한다

이름	의미
COUNT	명시한 컬럼의 행 개수 출력. 컬럼이름을 직접 명시할 경우 NULL을 제외한 행의 개수를 출력하고 * 사용 시 NULL을 포함한 개수 출력
SUM	NULL을 제외한 모든 행의 합 출력
AVG	NULL을 제외한 모든 행의 평균 출력
MAX	NULL을 제외한 모든 행에서 최대값 출력
MIN	NULL을 제외한 모든 행에서 최소값 출력

3.1 COUNT

행의 개수를 출력

쿼리문

```
SELECT COUNT(*), COUNT(empno), COUNT(mgr), COUNT(comm)
FROM emp;
```

결 과

	COUNT(*)	COUNT(EMPNO)	COUNT(MGR)	COUNT(COMM)
1	14	14	13	6

- COUNT(*) : emp 테이블의 전체 행은 12개
- COUNT(empno) : NULL값이 없는 empno컬럼의 수는 12개
- COUNT(mgr) : NULL값이 한 개 존재하는 mgr컬럼은 11개
- COUNT(comm) : NULL이 아닌 값이 4개인 comm 컬럼

3.2 SUM

행들의 합을 출력

사원들의 급여를 합산

쿼리문

```
SELECT SUM(sal) FROM emp;
```

결 과

	SUM(SAL)
1	29025

3.3 AVG

행들의 평균 출력

사원들의 급여 평균을 출력

쿼리문

```
SELECT AVG(sal) FROM emp;
```

결 과

	AVG(SAL)
1	2073.21428571428571428571428571428571428571

평균을 소수점 두 번째 자리에서 반올림하여 출력

쿼리문

```
SELECT ROUND(AVG(sal), 2) FROM emp;
```

결 과

	ROUND(AVG(SAL),2)
1	2073.21

위 예제는 소수점두번째자리에서 반올림한 결과를 리턴한다.

3.4 MAX

행 중에서 최고값 출력

사원들 중 제일 많은 급여 조회

쿼리문

```
SELECT MAX(sal) FROM emp;
```

결 과

	MAX(SAL)
1	5000

3.5 MIN

행들의 최소값 출력

사원들 중 제일 적은 급여 조회

쿼리문

```
SELECT MIN(sal) FROM emp;
```

결 과

	MIN(SAL)
1	800

4 데이터 그룹화 - GROUP BY 절

데이터 들을 원하는 그룹으로 나눌 때 사용

그룹을 나누고자 하는 컬럼명을 SELECT절과 GROUP BY 절 뒤에 추가하여 사용한다

직종 별 사원 수 조회

쿼리문

```
SELECT job, COUNT(job) FROM emp GROUP BY job;
```

결 과

	JOB	COUNT(JOB)
1	CLERK	4
2	SALESMAN	4
3	PRESIDENT	1
4	MANAGER	3
5	ANALYST	2

위 예제는 job컬럼의 속성값에 따라 그루핑한 후 각 그룹별 카운트를 리턴한다.

부서별 사원 수 조회

쿼리문

```
SELECT e.deptno, dname, COUNT(e.empno)
FROM emp e, dept d
WHERE e.deptno = d.deptno
GROUP BY e.deptno, dname;
```

결 과

	DEPTNO	DNAME	COUNT(E.EMPNO)
1	10	ACCOUNTING	3
2	20	RESEARCH	5
3	30	SALES	6

위 예제는 부서번호, 부서이름에 따라 그룹핑한후에 각부서별 사원수를 리턴한다.

5 그룹함수 데이터의 선별 - HAVING 절

GROUP BY 이후에 조건적으로 데이터를 선별하려면 어떻게 해야할까?

기본적으로 WHERE절에서는 그룹함수를 사용할 수 없다.

이를 해결하기 위해 GOURP BY 절과 함께 HAVING절을 사용한다

부서인원이 5명 이상인 부서 조회

쿼리문

```
SELECT e.deptno, dname, COUNT(e.empno)
FROM emp e, dept d
WHERE e.deptno = d.deptno
GROUP BY e.deptno, dname
HAVING COUNT(e.empno) > 5;
```

결 과

	DEPTNO	DN...	COUNT(E.EMPNO)
1	30	SALES	6

Chapter 3. DML - INSERT, UPDATE, DELETE 구문

1. DML(Data Manipulation Language), 데이터 조작어

데이터를 조회(SELECT), 삭제(DELETE), 입력(INSERT), 변경(UPDATE) 등으로 데이터를 조작하는데 사용하는 구문이다

2. INSERT

테이블에 새로운 행을 추가할 때 사용하는 SQL

테이블의 모든 컬럼에 자료를 입력할 경우 컬럼 목록을 기술하지 않아도 VALUES절 다음의 값들이 테이블의 기본 컬럼 순서대로 입력된다

INSERT 문법

```
INSERT INTO table_name( column1, column2, … )
VALUES ( data1, data2, … );
```

```
INSERT INTO table_name1( column1, column2, … )
SELECT column1, column2, …
FROM table_name2
WHERE 조건;
```

2.1. 기본 INSERT

2.1.1. emp 테이블 구조

```
DESC emp;
```

```
이름          널          유형
--------      --------    ------------
EMPNO         NOT NULL    NUMBER(4)
ENAME                     VARCHAR2(10)
JOB                       VARCHAR2(9)
MGR                       NUMBER(4)
```

```
HIREDATE                DATE
SAL                     NUMBER(7,2)
COMM                        NUMBER(7,2)
DEPTNO                  NUMBER(2)
```

emp 테이블에 데이터 INSERT

```
INSERT INTO emp VALUES (1001, 'ALICE', 'CLERK', 1003, '16/09/08', 800, NULL, 30);
INSERT INTO emp VALUES (1002, 'MORRIS', 'CLERK', 1003, '16/09/08', 800, NULL, 30);
INSERT INTO emp VALUES (1003, 'MATHEW', 'SALEMAN', NULL, '16/01/01', 1500, 100,
30);
INSERT INTO emp (empno, ename) VALUES (1010, 'BOB');
INSERT INTO emp (empno, ename, job) VALUES (1011, 'EDWARD', 'MANAGER');
```

INSERT한 데이터 확인

```
SELECT * FROM emp WHERE empno <2000;
```

```
EMPNO ENAME JOB MGR HIREDATE SAL COMM DEPTNO
---------- ---------- --------- ---------- -------- ---------- ---------- ----------
1001 ALICE CLERK 16/09/08 800 30
1002 MORRIS CLERK 1003 16/09/08 800 30
1003 MATHEW SALEMAN 16/01/01 1500 100 30
1010 BOB
1011 EDWARD MANAGER
```

2.2. 서브쿼리를 이용한 INSERT

INSERT INTO 절의 VALUES절을 서브 쿼리로 대신하는 것이 가능하다. 기존 테이블의 여러 행을 한번에 복사해서 다른 테이블에 입력할 수 있다. INSERT 문에 지정한 컬럼의 개수와 데이터 타입이 서브쿼리를 수행한 결과와 동일해야만 가능하다.

emp 테이블 구조와 같지만 데이터가 없는 emp_research 테이블을 생성

```
CREATE TABLE emp_research AS SELECT * FROM emp WHERE 1=0;
```

위 예제에서 WHERE절의 1=0 조건을 만족하는 행을 없으므로, 비어있는 테이블이 생성된다.

'RESEARCH' 부서 인원만을 이용해 emp_research에 INSERT

```
INSERT INTO emp_research (SELECT * FROM emp WHERE deptno=20);
SELECT * FROM emp_research;
```

```
EMPNO ENAME JOB MGR HIREDATE SAL COMM DEPTNO
---------- ---------- --------- ---------- -------- ---------- ---------- ----------
7369 SMITH CLERK 7902 80/12/17 800 20
7566 JONES MANAGER 7839 81/04/02 2975 20
7902 FORD ANALYST 7566 81/12/03 3000 20
```

2.3. INSERT ALL

WHEN 절에 지정한 조건을 만족하는 데이터를 테이블을 지정해 삽입

emp_hire, emp_sal 테이블 생성

```
CREATE TABLE emp_hire AS SELECT empno, ename, hiredate FROM emp where 1=0;
CREATE TABLE emp_sal AS SELECT empno, ename, sal FROM emp where 1=0;
```

위 예제에서 WHERE절의 1=0 조건을 만족하는 행을 없으므로, 비어있는 테이블이 생성된다.

emp_hire 테이블 구조

```
DESC emp_hire;
```

```
이름            널 유형
----------------------
EMPNO           NUMBER(4)
ENAME                   VARCHAR2(10)
HIREDATE        DATE
```

emp_sal 테이블 구조

```
DESC emp_sal;
```

```
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME                   VARCHAR2(10)
SAL             NUMBER(7,2)
```

emp_hire 테이블과 emp_sal 테이블에 각각 조건에 맞게 INSERT

```
INSERT ALL
```

```
WHEN HIREDATE > '2010/01/01'          -- 고용일이 '2010/01/01' 이후인 사원
THEN INTO emp_hire
VALUES(empno, ename, hiredate)
WHEN sal>=2000        -- 급여가 2000 이상인 사원
THEN INTO emp_sal
VALUES(empno, ename, sal)
SELECT empno, ename, hiredate, sal FROM emp;
```

```
SQL> SELECT * FROM emp_hire;
EMPNO      ENAME      HIREDATE
---------- ---------- --------
1002       MORRIS             16/09/08
1001       ALICE      16/09/08
1003       MATHEW             16/01/01
```

```
SQL> SELECT * FROM emp_sal;
EMPNO      ENAME      SAL
---------- ---------- ----------
7566       JONES      2975
7698       BLAKE      2850
7782       CLARK      2450
7839       KING       5000
7902       FORD       3000
```

위 예제에서 INSERT ALL구문의 가장 밑에 있는 서브쿼리가 대상 테이블이 된다. 그 중 WHEN~THEN 분기문에 따라 HIREDATE > '2010/01/01' 조건을 만족하는 행은 emp_hire 테이블에 INSERT되고, sal>=2000 조건을 만족하는 행은 emp_sal 테이블에 INSERT된다.

3. UPDATE

테이블의 행을 수정할 때 사용하는 SQL

WHERE 절을 사용하지 않으면 모든 행이 수정되므로 주의해야 한다

3.1. UPDATE 문법

```
UPDATE table_name
SET columne = 변경할값, column2 = 변경할값, …
WHERE 조건;
```

3.2. UPDATE 예제

emp 테이블에 데이터 INSERT

```
INSERT INTO emp VALUES (1001, 'ALICE', 'CLERK', 1003, '16/09/08', 800, NULL, 30);
INSERT INTO emp VALUES (1002, 'MORRIS', 'CLERK', 1003, '16/09/08', 800, NULL, 30);
INSERT INTO emp VALUES (1003, 'MATHEW', 'SALEMAN', NULL, '16/01/01', 1500, 100, 30);
INSERT INTO emp (empno, ename) VALUES (1010, 'BOB');
INSERT INTO emp (empno, ename, job) VALUES (1011, 'EDWARD', 'MANAGER');
```

'MATHEW' 사원의 이름을 'MATT'으로 수정

```
UPDATE emp
SET ename = 'MATT'
WHERE empno='1003'
```

```
EMPNO      ENAME  HIREDATE
---------- ---------- --------
1002       MORRIS      16/09/08
1001       ALICE  16/09/08
1003       MATT   16/01/01
```

모든 사원의 고용일을 오늘로 수정 (2017년 1월 8일에 작성)

```
UPDATE emp
SET hiredate=SYSDATE;
```

```
EMPNO      ENAME HIREDATE
---------- ---------- --------
1002       MORRIS     17/01/08
1001       ALICE      17/01/08
1003       MATT       17/01/08
```

3.3. 서브쿼리를 이용한 UPDATE

영업부와 회계부의 SAL를 10% 인상하는 쿼리를 작성해보자.

```
UPDATE emp
SET sal = sal + (sal * 0.1)
WHERE DEPTNO IN (SELECT DEPTNO FROM DEPT WHERE DNAME IN ('SALES', 'ACCOUNTING') );

--해당부서원 조회
SELECT * FROM EMP
WHERE DEPNTNO IN (SELECT DEPTNO FROM DEPT WHERE DNAME IN ('SALES', 'ACCOUNTING') );
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	1006	JORDAN	MANAGER	7782	82/02/05	5000	(null)	10
2	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
3	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
4	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
5	1003	MATT	SALEMAN	(null)	16/01/01	1500	100	30
6	1002	MORRIS	CLERK	1003	16/09/08	800	(null)	30
7	1001	ALICE	CLERK	1003	16/09/08	800	(null)	30
8	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
9	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
10	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
11	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
12	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30
13	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30

EMP테이블에서 DEPTNO가 SALES부서와 ACCOUNT부서의 DEPTNO인 10, 30과 동일한 행을 대상으로 10%의 월급을 인상하는 UPDATE문을 실행하였다.

4. DELETE

테이블의 행을 수정할 때 사용하는 SQL

WHERE절을 사용하지 않으면 모든 행이 삭제되므로 각별히 주의해야 한다

4.1. DELETE 문법

```
DELETE FROM table_name
WHERE 조건;
```

4.2. DELETE 예제

1. emp_sal 테이블의 급여가 3000미만인 사원들 삭제

```
DELETE FROM emp WHERE sal < 3000;
```

```
-- DELETE 전
EMPNO      ENAME      SAL
---------- ---------- ----------
7566       JONES      2975
7698       BLAKE      2850
7782       CLARK      2450
7839       KING       5000
7902       FORD       3000
```

```
-- DELETE 후
EMPNO      ENAME      SAL
---------- ---------- ----------
7839       KING       5000
7902       FORD       3000
```

5. MERGE

조건에 따라서 삽입, 갱신, 삭제 후 갱신 작업을 한번에 할 수 있는 구문
해당 행이 존재 하는 경우 UPDATE(DELETE 포함 가능)을 수행하고, 없는 행일 경우 INSERT를 수행
삽입/갱신에 대한 선택의 조건은 ON절로 표현한다

5.1. MERGE 문법

```
MERGE [ hint ]
INTO [ schema. ] { table | view } [ t_alias ]
USING { [ schema. ] { table | view }
| subquery
} [ t_alias ]
ON ( condition )
WHEN MATCHED THEN
UPDATE SET column = { expr | DEFAULT }
[, column = { expr | DEFAULT } ]...
[ DELETE where_clause ]
WHEN NOT MATCHED THEN
INSERT [ (column [, column ]...) ]
VALUES ({ expr [, expr ]... | DEFAULT })
```

INTO : 데이터가 INSERT되거나 UPDATE될 테이블 또는 뷰를 지정
USING : 비교할 소스 테이블 또는 뷰나 서브쿼리를 지정. INTO절 테이블과 동일해도 된다
ON : INSERT되거나 UPDATE될 조건. 조건을 만족하는 데이터가 존재하면 WHEN MATCHED 절을 수행하고 데이터가 존재하지 않으면 WHEN NOT MATCHED 절을 수행
WHEN MATCHED : ON 조건절에 해당하는 행이 존재할 경우 수행(UPDATE, DELETE 가능)
WHEN NOT MATCHED : ON 조건절에 해당하는 행이 존재하지 않을 경우 수행(INSERT)

부서번호가 10이나 20인 사원들을 이용해 emp_merge_ex 테이블 생성

```
CREATE TABLE emp_merge_ex
AS SELECT empno, ename, sal, deptno FROM emp WHERE deptno IN (10, 20);
```

```
SQL> SELECT * FROM emp_merge_ex;
EMPNO      ENAME  SAL    DEPTNO
---------- ------ ------ ----------
7369       SMITH  800    20
7566       JONES  2975   20
7782       CLARK  2450   10
7839       KING   5000   10
```

```
7902            FORD    3000    20
7934            MILLER  1300    10
```

emp 테이블의 부서번호가 20, 30인 사원들을 이용해 emp_merge_ex 테이블에 이미 존재하는 사원일 경우 급여를 10% 인상하고, 존재하지 않으면 급여가 1000보다 큰 사원들을 등록

```
MERGE INTO emp_merge_ex m
USING (
SELECT empno, ename, deptno, sal
FROM emp
WHERE deptno IN (20,30)
) e
ON ( m.empno = e.empno )
WHEN MATCHED THEN
UPDATE SET m.sal = ROUND(m.sal*1.1)
WHEN NOT MATCHED THEN
INSERT (m.empno, m.ename, m.sal, m.deptno)
VALUES (e.empno, e.ename, e.sal, e.deptno)
WHERE e.sal > 1000;
```

```
SQL> SELECT * FROM emp_merge_ex;
EMPNO       ENAME       SAL         DEPTNO
----------  ----------  ----------  ----------
1003        MATHEW                  1650      30
7369        SMITH       968         20
7499        ALLEN       1760        30
7521        WARD        1375        30

7566        JONES       3600        20

7654        MARTIN      1375        30

7698        BLAKE       3135        30
7782        CLARK       2450        10
7839        KING        5000        10
7844        TURNER                  1650      30
7902        FORD        3630        20
7934        MILLER      1300        10
```

Chapter 4. TCL - TRANSACTION 구문

1. TRANSACTION

데이터를 처리하는 한 단위

하나의 논리적인 작업 단위로써 성공하거나 실패하는 일련의 SQL문이다

데이터를 변경하는 DML문장으로 구성된다

① 트랜잭션의 시작

데이터를 조작하는 명령어(DML)가 실행될 때

② 트랜잭션의 종료

COMMIT이나 ROLLBACK 수행

DDL이나 DCL문장 실행

기계 장애 또는 시스템 충돌

deadlock 발생

사용자의 정상 종료

③ 자동 COMMIT 발생 상황

DDL, DCL문장의 완료

SQL*Plus의 정상 종료

④ 자동 ROLLBACK 발생 상황

SQL*Plus의 비정상 종료

시스템의 비정상적인 종료

1.1. COMMIT / ROLLBACK - DCL, TCL

COMMIT : 트랜잭션(변경사항)을 데이터베이스에 반영

ROLLBACK : 트랜잭션(변경사항)을 데이터베이스에 반영하지 않음

COMMIT 이전에는 데이터베이스에 영구적으로 변경사항이 적용되기 전이기 때문에 이전데이터로 복구가 가능하다. SELECT문을 이용하여 작업자는 변경한 사항을 확인할 수 있으며 다른 사용자는 변경사항을 확인할 수 없다. 변경된 행은 LOCK이 설정되어 있는 상태로 다른 사용자는 변경할 수 없는 상태가 된다. 이것으로 데이터베이스의 데이터 일관성을 제공한다.

COMMIT 이후에는 변경사항이 영구적으로 데이터베이스에 반영이 되며 이전 상태는 완전히 상실하게 된다. 작업자를 포함해 모든 사용자가 변경사항을 조회할 수 있으며 LOCK이 설정되어 있던 행의 LOCK이 해제된다.

1.2. AUTOCOMMIT

AUTOCOMMIT설정은 접속한 세션별로 변경할 수 있다.

```
--현재 AUTOCOMMIT상태 확인
SHOW AUTOCOMMIT;
```

```
autocommit OFF
```

```
--AUTOCOMMIT ON으로 세팅하기
SET AUTOCOMMIT ON;
```

```
autocommit IMMEDIATE
```

AUTOCOMIT 모드가 OFF더라도, DML문 실행 후 CREATE, ALTER, DROP , RENAME, TRUNCATE 등의 DDL문을 실행하면, 자동으로 COMMIT한다.

임의의 테이블 EMP01를 생성하고, DELETE문(DML)를 실행하되, COMMIT 전 상태에서 EMP02를 생성한다(DDL). 그리고, ROLLBACK을 실행한 결과를 알아보자.

```
--- EMP01 테이블 생성
CREATE TABLE EMP01
AS
SELECT * FROM EMP;

---EMP01 테이블의 특정행 삭제
DELETE FROM EMP01 WHERE EMPNO = 1234;
```

```
---EMP02테이블 생성
CREATE TABLE EMP02
AS
SELECT * FROM EMP;

---ROLLBACK실행
ROLLBACK;

--EMP01 테이블에서 삭제된 행 조회
SELECT * FROM EMP01 WHERE EMPNO = 1234;
--0행이 리턴됨.
```

위 예제처럼 DDL문 실행 시 자동으로 COMMIT 되었기 때문 DELETE 문의 실행 결과를 되돌릴 수 없다. 또 DDL문의 실행 결과와 상관없이 AUTOCOMMIT 된다. 예를 들어 CREATE TABLE 명령이 오류를 발생하고 끝나서, 원하는 DDL문의 결과를 얻지 못하였다고 해도, 이미 AUTOCOMMIT 처리된 이후이기 때문에 ROLLBACK은 불가하다.

1.3. SAVEPOINT / ROLLBACK TO

SAVEPOINT : 트랜잭션 작업을 사용자가 여러 개의 세그먼트로 분할하여, 부분적인 ROLLBACK 을 가능하게 한다.

ROLLBACK TO : 여러 SAVEPOINT 중에 특정 지점으로 돌아간다. 해당 SAVEPOINT 이후의 쿼리만 ROLLBACK 하는 결과를 얻을 수 있다.

```
SAVEPOINT
  INSERT INTO tbl1 (col1, col2, col3)
  VALUES (101, 102, 103);

  SAVEPOINT A;

  INSERT INTO tbl1 (col1, col2, col3)
  VALUES (201, 202, 203);

  SAVEPOINT B;
```

```
DELETE FROM tbl1
WHERE col1 IN (101, 201);

SELECT col1, col2, col3
FROM tbl1
WHERE col1 IN (101, 201);

-> 선택된 행이 없습니다.
```

ROLLBACK

ROLLBACK TO A;

-> SAVEPOINT A로 가서 그 이후 쿼리는 ROLLBACK한다.

ROLLBACK TO B;

-> SAVEPOINT B로 가서 그 이후 쿼리는 ROLLBACK한다.

Part 2. 고급 SQL 작성하기

Chapter 5. 분석함수

1. 분석함수의 정의

오라클 분석함수는 데이터를 분석하는 함수이다. 분석 함수를 사용하면, 쿼리 실행의 결과인 Result Set 을 대상으로 전체 그룹별이 아닌 소그룹별로 각 행에 대한 계산 값을 리턴 한다.

일반 그룹함수들과 다른 점은 분석함수는 분석함수용 그룹을 별도로 정의해서, 그 그룹을 대상으로 계산을 수행한다는 것이다. (일반 그룹함수는 그룹별로 계산을 수행한다)

이때 분석함수용 그룹을 오라클에서는 윈도우(window)라고 부른다.
분석 함수는 집계 결과를 각 행마다 보여준다.

```
SELECT deptno
    , empno
    , sal
    , SUM(sal) OVER(PARTITION BY deptno) s_sal
 FROM emp;
```

사용형식

```
분석함수 ([전달인자1, 전달인자2, 전달인자3])
OVER ( [쿼리 Partition 절]
[Order by 절]
[Window 절])
FROM 테이블명;
```

* **분석함수** : AVG, SUM, RANK, MAX, MIN, COUNT 등
* **전달인자** : 분석함수에 따라서 0~3개의 값이 올 수 있다
* **쿼리 Partition 절** : PARTITION BY 표현식
  PARTITION BY 로 시작하며, 표현식에 따라 그룹별로 단일 결과 셋으로 분리하는 역할
  즉, 분석함수의 계산대상 그룹을 지정함
* **ORDER BY 절** : PARTITION 절 뒤에 위치하며, 계산 대상 그룹에 대해 정렬 작업을 수행함
* **WINDOW 절** : 분석함수의 대상이 되는 데이터를 행 기준으로 범위를 더 세부적으로 정의하는데,
  PARTITION BY 에 의해 나누어진 기준 그룹에 또 다른 소그룹을 만듦.
  생략할 수 있음.

분석함수의 종류

- 순위함수 : RANK, DENSE_RANK, ROW_NUMBER, NTILE
- 집계함수 : SUM, MIN, MAX, AVG, COUNT
- 기타함수 : LEAD, LAG, FIRST_VALUE, LAST_VALUE, RATIO_TO_REPORT
- 분석함수(?) : KEEP, LISTAGG

1.1 등수 매기는 함수 : rank

-- 같은 등수가 있을 때는 다음 등수 값이 건너 뜀
-- 예: 1, 1, 3

급여에 순위를 매긴다면,

```
SELECT EMP_ID, EMP_NAME, SALARY,
RANK() OVER(ORDER BY SALARY DESC)  급여순위
FROM EMPLOYEE;
```

지정하는 값의 순위를 조회하고 싶다면, RANK(순위를 알고자하는 값) 사용

급여 2300000 이 급여가 많은 값에서 적은 값 순으로 정렬되었을 때 순위를 조회한다면,

```
SELECT RANK(2300000) WITHIN GROUP (ORDER BY SALARY DESC) AS  순위
FROM EMPLOYEE;
```

1.2 DENSE_RANK()

: 같은 순위일 때도 순위값이 건너뛰지 않음
-- 예 : 1, 2, 2, 3

-- 급여 전체에 순위를 매김

```
SELECT EMP_NAME, DEPT_ID, SALARY,
RANK() OVER (ORDER BY SALARY DESC) "순위1",
DENSE_RANK() OVER (ORDER BY SALARY DESC) "순위2",
DENSE_RANK() OVER (PARTITION BY DEPT_ID
ORDER BY SALARY DESC) "순위3"   -- 그룹 안에서의 순위
FROM EMPLOYEE
ORDER BY 2 DESC;
```

-- rank 를 이용한 top-N 분석 방법

```
SELECT *
FROM (SELECT EMP_NAME, SALARY,
RANK() OVER (ORDER BY SALARY DESC) 순위
FROM EMPLOYEE)
WHERE 순위 <= 5; -- 상위 5개의 정보 조회
```

-- 급여 적은순(내림차순) 11순위에 해당하는 정보 조회

```
SELECT *
FROM (SELECT EMP_NAME, SALARY,
RANK() OVER (ORDER BY SALARY DESC) 순위
FROM EMPLOYEE)
WHERE 순위 = 11;
```

1.3 CUME_DIST() : CUMulativE DISTribution

-- partition by 에 의해 나누어진 그룹별로 각 행을 order by 절에 명시된
-- 순서로 정렬한 후 그룹별 누적된 분산정도(상대적인 위치)를 구하는 함수임
-- 분산정도(상대적인 위치)는 구하고자 하는 값보다 작거나 같은 값을 가진
-- 행 수를 그룹 내의 총 행수로 나눈 것을 의미하며,
-- 0 < 결과 값 <= 1 범위의 값이 됨.
-

- 부서코드가 '50'인 직원의 이름, 급여, 누적분산 을 조회

```
SELECT EMP_NAME, SALARY,
ROUND(CUME_DIST() OVER (ORDER BY SALARY), 1) 누적분산
FROM EMPLOYEE
WHERE DEPT_ID = '50';
```

1.4 NTILE()

/* : partition을 bucket이라 불리는 그룹별로 나누고, partition 내의 각 행을 bucket 에 배치하는 함수임
- 예를 들어 partition 안에 100개의 행이 있다면, bucket 을 4개로 하면 1개의 bucket 당 25개의 행이 배분됨.
정확하게 분배되지 않을 때는 근사치로 배분한 후 남는 값에 대해 최초 bucket부터 한 개씩 배분함.*/

-- 급여를 4등급으로 분류

```
SELECT EMP_NAME, SALARY,
NTILE(4) OVER(ORDER BY SALARY)  등급
FROM EMPLOYEE;
```

1.5 ROW_NUMBER()

/*ROWNUM과는 관계가 없음.
각 PARTITION 내의 값들을 ORDER BY 절에 의해 정렬한 후 그 순서대로 순번을 부여함. */

-- 사번, 이름, 급여, 입사일, 순번

-- 단 순번은 급여가 많은 순으로, 같은 급여는 입사일이 빠른 사람부터

```
SELECT EMP_ID, EMP_NAME, SALARY, HIRE_DATE,
ROW_NUMBER() OVER (ORDER BY SALARY DESC,
HIRE_DATE ASC)  순번
FROM EMPLOYEE;
```

2. 집계함수

집계함수는 여러행 또는 테이블 전체 행으로부터 그룹별로 집계하여 결과를 반환한다.

```
SELECT deptno
     , SUM(sal) s_sal
  FROM emp
 GROUP BY deptno;
```

-- employee 테이블로 부터 부서코드가 '60'인 직원들의
-- 사번, 급여, 해당 부서그룹(윈도우라고 함)의 사번을 오름차순 정렬하고
-- 급여의 합계를 첫행부터 마지막행까지 구해서 win1 에,

```
SELECT EMP_ID, SALARY,
SUM(SALARY) OVER (PARTITION BY DEPT_ID
ORDER BY EMP_ID
ROWS BETWEEN UNBOUNDED PRECEDING
AND UNBOUNDED FOLLOWING) "win1",
-- rows : 부분그룹인 윈도우의 크기를 물리적인 단위로 행집합을 지정
-- unbounded preceding : 윈도우의 첫행
-- unbounded following : 윈도우의 마지막행
SUM(SALARY) OVER (PARTITION BY DEPT_ID ORDER BY EMP_ID
ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) "win2",
-- 윈도우의 시작행에서 현재 위치(current row) 까지의 합계를 구해서 win2에
SUM(SALARY) OVER (PARTITION BY DEPT_ID ORDER BY EMP_ID
ROWS BETWEEN CURRENT ROW AND
UNBOUNDED FOLLOWING) "win3"
-- 현재 위치에서 윈도우의 마지막행까지의 합계를 구해서 win3에
FROM EMPLOYEE
WHERE DEPT_ID = '60';
```

```
SELECT EMP_ID, SALARY,
SUM(SALARY) OVER (PARTITION BY DEPT_ID
ORDER BY EMP_ID
ROWS BETWEEN 1 PRECEDING
AND 1 FOLLOWING) "win1",
-- 1 preceding and 1 following
-- 현재 행을 중심으로 이전행과 다음행의 급여합계
SUM(SALARY) OVER (PARTITION BY DEPT_ID ORDER BY EMP_ID
ROWS BETWEEN 1 PRECEDING AND
CURRENT ROW) "win2",
-- 1 preceding and current row
-- 이전행과 현재행의 급여 합계
SUM(SALARY) OVER (PARTITION BY DEPT_ID ORDER BY EMP_ID
ROWS BETWEEN CURRENT ROW AND
1 FOLLOWING) "win3"
-- current row and 1 following
-- 현재 행과 다음행의 합계
FROM EMPLOYEE
WHERE DEPT_ID = '60';
```

2.1 RATIO_TO_REPORT

-- 해당 구간에서 차지하는 비율을 리턴하는 함수

-- 직원들의 총급여를 20,000,000원 증가 시킬 때, 기존 월급비율을 적용해서

-- 각 직원이 받게될 급여의 증가액은?

```
SELECT EMP_NAME, SALARY,
LPAD(TRUNC(RATIO_TO_REPORT(SALARY) OVER() * 100, 0), 5) || ' %'  비율,
TO_CHAR(TRUNC(RATIO_TO_REPORT(SALARY) OVER() * 20000000, 0), 'L00,999,999')
"추가로 받게될 급여"
FROM EMPLOYEE;
```

2.2 Lag(조회할 범위, 이전위치, 기준 현재위치)

: 지정하는 컬럼의 현재 행을 기준으로 이전 행(위)의 값을 조회함

```
SELECT EMP_NAME, DEPT_ID, SALARY,
LAG(SALARY, 1, 0) OVER (ORDER BY SALARY)  이전값,
-- 1 : 위의 행값, 0 : 이전행이 없으면 0 처리함
LAG(SALARY, 1, SALARY) OVER (ORDER BY SALARY) "조회2",
-- 이전행이 없으면 현재 행의 값을 출력
LAG(SALARY, 1, SALARY) OVER (PARTITION BY DEPT_ID
ORDER BY SALARY) "조회3"
-- 부서 그룹안에서의 이전 행값 출력
FROM EMPLOYEE;
```

2.3 Lead(조회할 범위, 다음행수, 0 또는 컬럼명)

: 다음 행의 값 조회

```
SELECT EMP_NAME, DEPT_ID, SALARY,
LEAD(SALARY, 1, 0) OVER (ORDER BY SALARY) 다음값,
-- 1 : 다음 행값, 0 : 다음행이 없으면 0 처리함
LEAD(SALARY, 1, SALARY) OVER (ORDER BY SALARY) "조회2",
-- 다음행이 없으면 현재 행의 값을 출력
LEAD(SALARY, 1, SALARY) OVER (PARTITION BY DEPT_ID
ORDER BY SALARY) "조회3"
-- 부서 그룹안에서의 다음 행값 출력
FROM EMPLOYEE;
```

2.4 FIRST_VALUE 함수

FIRST_VALUE 함수를 이용해 파티션별 윈도우에서 가장 먼저 나온 값을 구한다.
SQL Server 에서는 지원하지 않는 함수이다. MIN 함수를 활용하여 같은 결과를 얻을 수도 있다.

```
SELECT DEPTNO, ENAME, SAL
      , FIRST_VALUE(ENAME) OVER (PARTITION BY DEPTNO ORDER BY SAL DESC
ROWS UNBOUNDED PRECEDING) as DEPT_RICH
FROM EMP;
```

2.5 LAST_VALUE 함수

LAST_VALUE 함수를 이용해 파티션별 윈도우에서 가장 나중에 나온 값을 구한다.
SQL Server 에서는 지원하지 않는 함수이다. MAX 함수를 활용하여 같은 결과를 얻을 수도 있다.

```
SELECT DEPTNO, ENAME, SAL
      , LAST_VALUE(ENAME) OVER ( PARTITION BY DEPTNO ORDER BY SAL DESC
ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING) as DEPT_POOR
  FROM EMP;
```

Chapter 6. JOIN, SET 연산자, SUBQUERY

1 조인이란?

둘 이상의 테이블을 연결하여 데이터를 검색하는 방법
일반적으로 테이블의 식별 값인 Primary Key와 테이블 간 공통 값인 Foreign Key 값을 사용하여 조인한다

1.1 EQUI JOIN(등가조인)

조인 대상 테이블에서 공통적으로 컬럼의 값이 일치되는 행을 연결하여 결과를 생성
가장 일반적으로 사용하는 '='에 의한 조인

emp 테이블과 dept 테이블을 조인하여 직원들의 부서명을 조회하는 예제

쿼리문

```
SELECT e.empno, e.ename, d.dname
FROM emp e, dept d
WHERE e.deptno = d.deptno;
```

```
(ANSI)
SELECT e.empno, e.ename, d.dname
FROM emp e JOIN dept d
ON e.deptno = d.deptno;
```

결 과

	EMPNO	ENAME	DNAME
1	7369	SMITH	RESEARCH
2	7499	ALLEN	SALES
3	7521	WARD	SALES
4	7566	JONES	RESEARCH
5	7654	MARTIN	SALES
6	7698	BLAKE	SALES
7	7782	CLARK	ACCOUNTING
8	7788	SCOTT	RESEARCH
9	7839	KING	ACCOUNTING
10	7844	TURNER	SALES
11	7876	ADAMS	RESEARCH
12	7900	JAMES	SALES
13	7902	FORD	RESEARCH
14	7934	MILLER	ACCOUNTING

1.2 NON-EQUI JOIN(비등가조인)

동일 컬럼이 없이 다른 조건을 사용하여 조인할 때 사용
BETWEEN AND, IS NULL, IS NOT NULL, IN, NOT IN 을 사용
비등가조인은 자주 쓰이지 않는다

emp테이블의 연봉을 이용해 salgrade의 연봉등급을 조인하여 조회

쿼리문

```
SELECT e.ename, e.sal, s.grade
FROM emp e, salgrade s
WHERE e.sal BETWEEN s.losal AND s.hisal;
```

```
(ANSI)
SELECT e.ename, e.sal, s.grade
FROM emp e JOIN salgrade s
ON e.sal BETWEEN s.losal AND s.hisal;
```

결 과

	ENAME	SAL	GRADE
1	SMITH	800	1
2	ADAMS	1100	1
3	JAMES	950	1
4	WARD	1250	2
5	MARTIN	1250	2
6	MILLER	1300	2
7	ALLEN	1600	3
8	TURNER	1500	3
9	JONES	2975	4
10	BLAKE	2850	4
11	CLARK	2450	4
12	SCOTT	3000	4
13	FORD	3000	4
14	KING	5000	5

1.3 SELF-JOIN

하나의 테이블에서 자신의 테이블과 Equi Join이 발생하는 것이다
하나의 테이블에 대해서 두 개의 alias를 사용해 두 개의 테이블인 것처럼 조인한다

emp테이블에서 사원과 해당 사원의 매니저 조회

쿼리문

```
SELECT e.ename, m.ename "Manager"
FROM emp e, emp m
WHERE e.empno = m.mgr;
```

```
(ANSI)
SELECT e.ename, m.ename "Manager"
FROM emp e JOIN emp m
ON e.empno = m.mgr;
```

결 과

	ENAME	Manager
1	FORD	SMITH
2	BLAKE	ALLEN
3	BLAKE	WARD
4	KING	JONES
5	BLAKE	MARTIN
6	KING	BLAKE
7	KING	CLARK
8	JONES	SCOTT
9	BLAKE	TURNER
10	SCOTT	ADAMS
11	BLAKE	JAMES
12	JONES	FORD
13	CLARK	MILLER

1.4 INNER JOIN

일반적인 JOIN에서 ,(콤마)를 생략하고 INNER JOIN 절을 사용
WHERE절 대신에 ON절을 사용한다
INNER 키워드는 생략 가능

emp 테이블과 dept 테이블을 조인하여 직원들의 부서명을 조회하는 예제

쿼리문

```
SELECT e.ename, m.ename "Manager"
FROM emp e, emp m
WHERE e.empno = m.mgr;
```

(ANSI)

```
SELECT e.ename, m.ename "Manager"
FROM emp e JOIN emp m
ON e.empno = m.mgr;
```

결 과

	ENAME	Manager
1	FORD	SMITH
2	BLAKE	ALLEN
3	BLAKE	WARD
4	KING	JONES
5	BLAKE	MARTIN
6	KING	BLAKE
7	KING	CLARK
8	JONES	SCOTT
9	BLAKE	TURNER
10	SCOTT	ADAMS
11	BLAKE	JAMES
12	JONES	FORD
13	CLARK	MILLER

1.5 OUTER JOIN

Equi Join의 경우 한쪽 컬럼에 값이 없다면 해당 행은 조회되지 않는다
조인 조건을 만족하지 못 하는 행들을 같이 조회하기 위해 OUTER JOIN을 사용한다
OUTER JOIN의 연산자는 "(+)"
조인조건을 만족하는 값이 없어도 조회하려는 쪽에 "(+)"를 사용
(+)연산자는 한쪽에만 붙여서 사용 가능하다

emp테이블에 없는 deptno인 40에 대해서도 조인이 이루어진 것을 볼 수 있다
(+)연산자를 왼쪽에 두는 조건은 ANSI 방식의 RIGHT OUTER JOIN으로 표현 가능

쿼리문

```
SELECT e.ename, e.deptno, d.dname
FROM emp e, dept d
WHERE e.deptno(+) = d.deptno
ORDER BY e.deptno;
```

```
(ANSI)
SELECT e.ename, e.deptno, d.dname
FROM emp e RIGHT OUTER JOIN dept d
ON e.deptno = d.deptno
ORDER BY e.deptno;
```

결 과

	EN...	DEPTNO	DNAME
1	CLARK	10	ACCOUNTING
2	MILLER	10	ACCOUNTING
3	KING	10	ACCOUNTING
4	JONES	20	RESEARCH
5	SMITH	20	RESEARCH
6	SCOTT	20	RESEARCH
7	FORD	20	RESEARCH
8	ADAMS	20	RESEARCH
9	WARD	30	SALES
10	TURNER	30	SALES
11	ALLEN	30	SALES
12	JAMES	30	SALES
13	MARTIN	30	SALES
14	BLAKE	30	SALES
15	(null)	(null)	OPERATIONS

emp테이블을 SELF JOIN하여 mgr의 이름을 조회
mgr이 존재하지 않은 KING도 출력된 것을 볼 수 있다
(+)연산자를 오른쪽에 두는 조건은 LEFT OUTER JOIN으로 표현 가능

쿼리문

```
SELECT employee.ename employee, manager.ename manager
FROM emp employee, emp manager
WHERE employee.mgr = manager.empno(+);
```

```
(ANSI)
SELECT employee.ename employee, manager.ename manager
FROM emp employee LEFT OUTER JOIN emp manager
ON employee.mgr = manager.empno;
```

결 과

	EMPLOYEE	MANAGER
1	FORD	JONES
2	SCOTT	JONES
3	JAMES	BLAKE
4	TURNER	BLAKE
5	MARTIN	BLAKE
6	WARD	BLAKE
7	ALLEN	BLAKE
8	MILLER	CLARK
9	ADAMS	SCOTT
10	CLARK	KING
11	BLAKE	KING
12	JONES	KING
13	SMITH	FORD
14	KING	(null)

1.6 ANSI / ISO SQL 표준 OUTER JOIN

오라클9i 부터 ANSI/ISO SQL 표준 문법인 LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN 문법을 지원한다

오라클 (+)이용한 방법은 WHERE절을 이용해 조인 조건을 설정하지만 OUTER JOIN절을 이용한 조인에서는 ON절을 이용

1.6.1 LEFT OUTER JOIN

JOIN의 왼쪽에 오는 테이블을 모두 포함하여 조회
오른쪽에 오는 테이블에 NULL이 있어도 왼쪽 테이블의 모든 행이 출력된다

매니저가 없는 직원을 포함하여 직원과 매니저 조회

쿼리문

```
SELECT employee.ename employee, manager.ename manager
FROM emp employee, emp manager
WHERE employee.mgr = manager.empno(+);
```

```
(ANSI)
SELECT employee.ename employee, manager.ename manager
FROM emp employee LEFT OUTER JOIN emp manager
ON employee.mgr = manager.empno;
```

결 과

	EMPLOYEE	MANAGER
1	FORD	JONES
2	SCOTT	JONES
3	JAMES	BLAKE
4	TURNER	BLAKE
5	MARTIN	BLAKE
6	WARD	BLAKE
7	ALLEN	BLAKE
8	MILLER	CLARK
9	ADAMS	SCOTT
10	CLARK	KING
11	BLAKE	KING
12	JONES	KING
13	SMITH	FORD
14	KING	(null)

1.6.2 **RIGHT OUTER JOIN**

JOIN의 오른쪽에 오는 테이블을 모두 포함하여 조회
왼쪽에 오는 테이블에 NULL이 있어도 오른쪽 테이블의 모든 행이 출력된다

OPERATION부서에 해당하는 직원을 포함하여 조회

쿼리문

```
SELECT e.ename, e.deptno, d.deptno, d.dname
FROM emp e, dept d
WHERE e.deptno(+) = d.deptno
ORDER BY e.deptno;
```

```
(ANSI)
SELECT e.ename, e.deptno, d.deptno, d.dname
FROM emp e RIGHT OUTER JOIN dept d
ON e.deptno = d.deptno
ORDER BY e.deptno;
```

결 과

	ENAME	DEPTNO	DEPTNO_1	DNAME
1	CLARK	10	10	ACCOUNTING
2	MILLER	10	10	ACCOUNTING
3	KING	10	10	ACCOUNTING
4	JONES	20	20	RESEARCH
5	SMITH	20	20	RESEARCH
6	SCOTT	20	20	RESEARCH
7	FORD	20	20	RESEARCH
8	ADAMS	20	20	RESEARCH
9	WARD	30	30	SALES
10	TURNER	30	30	SALES
11	ALLEN	30	30	SALES
12	JAMES	30	30	SALES
13	MARTIN	30	30	SALES
14	BLAKE	30	30	SALES
15	(null)	(null)	40	OPERATIONS

1.6.3 FULL OUTER JOIN

양쪽 테이블 모두 OUTER JOIN 대상에 포함한다
어느 쪽이든 NULL을 포함하는 행이 있더라도 모두 출력한다

test1, test2 테이블의 no 컬럼은 각각 (10, 20,30), (10, 20, 40) 데이터를 가지고 있다

쿼리문

```
SELECT test1.NO no1, test2.NO no2
FROM test1 FULL OUTER JOIN test2
ON test1.NO = test2.NO;
```

결 과

	NO1	NO2
1	10	10
2	20	20
3	(null)	40
4	30	(null)

1.7 CROSS JOIN

두 테이블의 카테시안 곱을 얻을 때 사용

1.8 카테시안 곱(Cartesian Product)

두 개 이상의 테이블이 조인될 때 조건이 적절하지 않아 모든 데이터가 결과로 나오는 경우
만약 SELECT * FROM emp, dept;을 수행하면 컬럼 11개(emp컬럼 8개 + dept컬럼 3개)와 48개 행(emp행 12개 * dept행 4개)의 의미없는 데이터가 조회된다
카테시안 곱이 발생하는 상황을 피하려면 조인의 WHERE절에 공통되는 조건이 존재하도록 올바른 조건을 주어야 한다

다음 쿼리를 수행해 보면 모든 컬럼과 모든 행을 조합한 카테시안 곱의 결과를 볼 수 있다.

쿼리문

```
SELECT test1.NO no1, test2.NO no2 FROM test1 CROSS JOIN test2;
```

결 과

	NO1	NO2
1	10	10
2	10	20
3	10	40
4	20	10
5	20	20
6	20	40
7	30	10
8	30	20
9	30	40

위 예제는 TEST1테이블의 3개행 X TEST2테이블의 3개행, 총 9행을 리턴한다.

1.9 INNER JOIN 키워드

일반적인 JOIN에서 ,(콤마)를 생략하고 INNER JOIN 절을 사용
WHERE절 대신에 ON절을 사용한다
INNER 키워드는 생략 가능

emp 테이블과 dept 테이블을 조인하여 직원들의 부서명을 조회하는 예제

쿼리문

```
SELECT e.empno, e.ename, d.dname
FROM emp e, dept d
ON e.deptno = d.deptno;
```

```
(ANSI)
SELECT e.empno, e.ename, d.dname
FROM emp e INNER JOIN dept d
ON e.deptno = d.deptno;
```

결 과

	EMPNO	ENAME	DNAME
1	7369	SMITH	RESEARCH
2	7499	ALLEN	SALES
3	7521	WARD	SALES
4	7566	JONES	RESEARCH
5	7654	MARTIN	SALES
6	7698	BLAKE	SALES
7	7782	CLARK	ACCOUNTING
8	7788	SCOTT	RESEARCH
9	7839	KING	ACCOUNTING
10	7844	TURNER	SALES
11	7876	ADAMS	RESEARCH
12	7900	JAMES	SALES
13	7902	FORD	RESEARCH
14	7934	MILLER	ACCOUNTING

1.10 USING절 이용하기

위의 예제에서 동일한 컬럼명을 조인조건으로 제시할 경우에 한하여, ANSI에서는 USING키워드를 제공하고 있다.

```
SELECT e.empno, e.ename, d.dname
FROM emp e INNER JOIN dept d
ON e.deptno = d.deptno;
```

```
--USING키워드를 사용하기
SELECT e.empno, e.ename, d.dname
FROM emp e INNER JOIN dept d
USING (deptno);
```

결 과

	EMPNO	ENAME	DNAME
1	7369	SMITH	RESEARCH
2	7499	ALLEN	SALES
3	7521	WARD	SALES
4	7566	JONES	RESEARCH
5	7654	MARTIN	SALES
6	7698	BLAKE	SALES
7	7782	CLARK	ACCOUNTING
8	7788	SCOTT	RESEARCH
9	7839	KING	ACCOUNTING
10	7844	TURNER	SALES
11	7876	ADAMS	RESEARCH
12	7900	JAMES	SALES
13	7902	FORD	RESEARCH
14	7934	MILLER	ACCOUNTING

이 때, USING키워드에 제공되는 컬럼명은 테이블별칭이 없는 형태여야 합니다.

- **USING** (deptno) (O)
- **USING** (e.deptno) (X)

1.11 NATURAL JOIN

EQUI JOIN과 비슷하지만 WHERE 조건절 없이 조인한다
두 테이블의 동일한 이름의 컬럼을 모두 조인한다
조인 조건의 제어가 어려운 단점이 있다

emp 테이블과 dept 테이블을 조인하여 직원들의 부서명을 조회하는 예제

쿼리문

```
SELECT empno, ename, dname
FROM emp NATURAL JOIN dept;
```

결 과

	EMPNO	ENAME	DNAME
1	7369	SMITH	RESEARCH
2	7499	ALLEN	SALES
3	7521	WARD	SALES
4	7566	JONES	RESEARCH
5	7654	MARTIN	SALES
6	7698	BLAKE	SALES
7	7782	CLARK	ACCOUNTING
8	7788	SCOTT	RESEARCH
9	7839	KING	ACCOUNTING
10	7844	TURNER	SALES
11	7876	ADAMS	RESEARCH
12	7900	JAMES	SALES
13	7902	FORD	RESEARCH
14	7934	MILLER	ACCOUNTING

위 예제에서 emp테이블과 dept테이블은 deptno라는 컬럼을 공유한다. NATURAL JOIN을 통해

별도의 명시없이 **SELECT empno, ename, dname FROM emp JOIN dept ON emp.deptno = dept.deptno;**
와 동일한 쿼리결과를 얻을 수 있다.

2. 서브 쿼리, Sub Query

하나의 SQL 문장 내에 포함된 또 다른 SQL 문이 있는 쿼리문
전체 SQL 문장을 메인 쿼리라고 하며 메인 쿼리(Main Query or Outer Query)에 종속된 내부 쿼리를 서브 쿼리(Sub Query or Inner Query)라고 한다
서브 쿼리는 반드시 괄호 ( ) 안에 넣어 표현해야 한다

이름이 'KING'인 사원의 부서번호를 조회하는 서브쿼리와 그 결과로 부서명을 조회하는 메인 쿼리

쿼리문

```
SELECT dname FROM dept
WHERE deptno = ( SELECT deptno FROM emp WHERE ename = 'KING' );
```

결 과

	DNA...
1	ACCOUNTING

2.1. 단일 행 서브 쿼리

서브 쿼리의 결과가 1개의 행만 나오는 것
서브 쿼리의 결과가 1건만 나오고 이 결과를 메인 쿼리로 전달하여 전체적인 쿼리를 수행한다
메인 쿼리의 WHERE절에서 단일 행 연산자(=, <>, >, >=, <, <=)를 이용한다

평균 급여보다 많은 급여를 받는 사원 조회

쿼리문

```
SELECT empno, ename, sal FROM emp
WHERE sal >= (SELECT AVG(sal) FROM emp);
```

결 과

	EMPNO	ENAME	SAL
1	7566	JONES	2975
2	7698	BLAKE	2850
3	7782	CLARK	2450
4	7788	SCOTT	3000
5	7839	KING	5000
6	7902	FORD	3000

2.2. 다중 행 서브 쿼리

서브 쿼리의 결과가 두 건 이상 출력되는 것

단일 행 연산자는 사용할 수 없으며, 다중 행 연산자 (IN, NOT IN, ANY, ALL, EXISTS)만 사용 가능

2.2.1. IN 연산자

WHERE절의 IN 연산자와 같다

하나의 컬럼이 여러 개의 '=' 조건을 가지는 경우 사용

부서별 급여를 제일 많이 받는 사원 조회

쿼리문

```
SELECT dname, empno, ename, sal
FROM emp
JOIN dept USING(deptno)
WHERE sal IN (SELECT MAX(sal)
              FROM emp
              GROUP BY deptno);
```

결 과

	DNAME	EMPNO	ENAME	SAL
1	SALES	7698	BLAKE	2850
2	RESEARCH	7788	SCOTT	3000
3	ACCOUNTING	7839	KING	5000
4	RESEARCH	7902	FORD	3000

위 예제는 우선, emp 테이블과 dept 테이블의 공통 컬럼이 deptno 를 이용해 조인한다.

WHERE 절 서브쿼리에서 emp 테이블에서 부서번호(deptno)별 월급 컬럼(sal)의 최대값을 구하는데, 다중행(복수개, 부서번호별)을 반환이다. 이때, IN을 사용하여, 메인 쿼리에서 WHERE절의 비교할 컬럼인 월급 컬럼(sal)값이 이 값 중에 하나에 해당되는 경우만 출력된다.

2.2.2. ANY 연산자

메인 쿼리의 비교 조건이 서브 쿼리의 여러 검색 결과 중 하나 이상 만족되면 반환한다.

'SALESMAN' 들의 급여를 조회하여 ANY 연산자로 메인 쿼리와 비교하여 출력되도록 해 보자.
'SALESMAN' 들의 급여 중 최소값보다 많은 급여를 받는 사원들이 출력된다

쿼리문

```
SELECT empno, ename, sal
FROM emp
WHERE sal > ANY (SELECT sal
                 FROM emp
                 WHERE JOB = 'SALESMAN');
```

결 과

	EMPNO	ENAME	SAL
1	7839	KING	5000
2	7902	FORD	3000
3	7788	SCOTT	3000
4	7566	JONES	2975
5	7698	BLAKE	2850
6	7782	CLARK	2450
7	7499	ALLEN	1600
8	7844	TURNER	1500
9	7934	MILLER	1300

'SALESMAN'들의 급여와 같은 급여를 받는 사원을 조회해 보자.

쿼리문

```
SELECT empno, ename, sal
FROM emp
WHERE sal = ANY (SELECT sal
                 FROM emp
                 WHERE JOB = 'SALESMAN');
```

결 과

	EMPNO	ENAME	SAL
1	7499	ALLEN	1600
2	7654	MARTIN	1250
3	7521	WARD	1250
4	7844	TURNER	1500

2.2.3. ALL 연산자

메인 쿼리의 비교 조건이 서브 쿼리의 여러 검색 결과와 모든 값이 일치하면 반환한다.

'MANAGER' 사원들의 급여들 보다 높은 급여를 받는 사원을 조회해 보자
'MANAGER' 사원들 중 최고 급여인 2975 보다 많은 급여를 받는 사원들이 조회된다

쿼리문

```
SELECT empno, ename, sal
FROM emp
WHERE sal > ALL (SELECT sal
                 FROM emp
                 WHERE JOB = 'MANAGER');
```

결 과

	EMPNO	ENAME	SAL
1	7788	SCOTT	3000
2	7902	FORD	3000
3	7839	KING	5000

2.2.4. EXISTS 연산자

서브쿼리의 데이터가 존재하는 지 여부를 먼저 따져 존재하는 값들만 결과로 출력한다.

관리자로 등록되어 있는 사원들을 조회해 보자.

쿼리문

```
SELECT empno, ename, sal
FROM emp e
WHERE EXISTS (SELECT empno
              FROM emp
              WHERE e.empno = mgr);
```

결 과

	EMPNO	ENAME	SAL
1	7902	FORD	3000
2	7698	BLAKE	2850
3	7839	KING	5000
4	7566	JONES	2975
5	7788	SCOTT	3000
6	7782	CLARK	2450

2.3. 다중 열 서브 쿼리

서브 쿼리의 결과가 두 개 이상의 컬럼으로 반환되어 메인 쿼리에 전달하는 쿼리이다.

부서 번호가 30인 사원들의 급여와 부서번호를 묶어 메인 쿼리로 전달해 보자.

쿼리문

```
SELECT empno, ename, sal, deptno
FROM emp
WHERE (deptno, sal) IN (SELECT deptno, sal
                        FROM emp
                        WHERE deptno = 30);
```

결 과

	EMPNO	ENAME	SAL	DEPTNO
1	7499	ALLEN	1600	30
2	7654	MARTIN	1250	30
3	7521	WARD	1250	30
4	7698	BLAKE	2850	30
5	7844	TURNER	1500	30
6	7900	JAMES	950	30

2.4. 스칼라 서브쿼리

SELECT문에서 쓰이는 단일 행 서브쿼리를 스칼라 서브쿼리라고 한다.

```
SELECT ENAME, DEPTNO, SAL,
        (SELECT TRUNC(AVG(SAL)) -- 소속 부서의 급여 평균값 1개
          FROM EMP
          WHERE DEPTNO = E.DEPTNO)  AS  AVGDEPTSAL
FROM EMP E;
```

위 예제에서 EMP 테이블의 매 행마다 부서번호가 각 행의 부서번호와 동일한 사원들의 SAL 평균을 구해서 리턴 하고 있다.

부서 위치가 NEW YORK 인 경우에 본사로, 그 외부서는 분점으로 리턴 하는 쿼리를 CASE문을 통해 실행해보자.

```
SELECT EMPNO, ENAME,
        CASE WHEN DEPTNO = (SELECT DEPTNO
                            FROM DEPT
                            WHERE LOC = 'NEW YORK')
             THEN '본사'
             ELSE '분점'
        END AS 소속
FROM EMP E
ORDER BY 소속 DESC;
```

위 예제에서 EMP 테이블을 순회하면서 사원의 부서번호가 LOC가 NEW YORK인 부서의 부서번호와 비교한 뒤 같으면, '본사'를 그 외에는 '분점'을 출력하고 있다. 이와 같이 대응되는 단 하나의 리턴 값을 가지는 서브쿼리를 스칼라 서브쿼리라 한다.

2.5. 상[호연]관 [서브]쿼리

메인 쿼리의 특정 컬럼을 조인 조건으로 사용하는 서브 쿼리이다.

SELECT 컬럼에 상관쿼리를 사용해서 사원의 부서이름을 구해 보자.

쿼리문

```
SELECT ENAME, JOB,
        (SELECT DNAME
          FROM DEPT
          WHERE DEPTNO = E.DEPTNO)   DNAME
FROM EMP E;
```

결 과

	ENAME	JOB	DNAME
1	SMITH	CLERK	RESEARCH
2	ALLEN	SALESMAN	SALES
3	WARD	SALESMAN	SALES
4	JONES	MANAGER	RESEARCH
5	MARTIN	SALESMAN	SALES
6	BLAKE	MANAGER	SALES
7	CLARK	MANAGER	ACCOUNTING
8	SCOTT	ANALYST	RESEARCH
9	KING	PRESIDENT	ACCOUNTING
10	TURNER	SALESMAN	SALES
11	ADAMS	CLERK	RESEARCH
12	JAMES	CLERK	SALES
13	FORD	ANALYST	RESEARCH
14	MILLER	CLERK	ACCOUNTING

위 예제는 EMP 테이블에 없는 컬럼인 부서이름을 조회하고 있다. 다만, 메인 쿼리에서 조인하지 않고, 한 행이 출력될 때, 해당 사원의 부서번호를 조인 조건으로 DEPT 테이블을 상관쿼리로 호출한다. 이때, 메인 쿼리의 EMP 테이블을 ALIAS로 호출하고 있는 것에 주의해야 한다.

WHERE 절에 상관쿼리를 사용해서 SALES 부서 사원 구하기 - 1

쿼리문

```
SELECT ENAME, JOB
FROM EMP E
WHERE DEPTNO = (SELECT DEPTNO
                FROM DEPT
                WHERE DEPTNO = E.DEPTNO AND DNAME= 'SALES' );
```

결 과

	ENAME	JOB
1	ALLEN	SALESMAN
2	WARD	SALESMAN
3	MARTIN	SALESMAN
4	BLAKE	MANAGER
5	TURNER	SALESMAN
6	JAMES	CLERK

위 예제는 매 행마다 EMP 테이블에서 부서번호와 DEPT 테이블의 부서번호가 동일하면서, SALES 부서인 경우만 상관쿼리로 조회하고 있다. E.DEPTNO 처럼 메인 쿼리의 EMP 테이블의 부서번호 컬럼을 ALIAS를 이용한다.

WHERE 절에 상관쿼리를 사용해서 SALES 부서 사원구하기 - 2

쿼리문

```
SELECT ENAME, JOB
FROM EMP E
WHERE EXISTS (SELECT 1 FROM DEPT
                    WHERE DEPTNO = E.DEPTNO AND DNAME = 'SALES');
```

결 과

	ENAME	JOB
1	ALLEN	SALESMAN
2	WARD	SALESMAN
3	MARTIN	SALESMAN
4	BLAKE	MANAGER
5	TURNER	SALESMAN
6	JAMES	CLERK

위 예제는 위와 동일한 결과를 WEHRE절 EXISTS를 사용한다. 이때 행이 존재하기만 하면, EXISTS를 만족하므로, 서브쿼리 SELECT행을 1로 처리하였다.

2.6 From절 서브 쿼리

메인 쿼리의 FROM 절을 서브 쿼리로 이용하는 방법으로, 인라인 뷰라고도 한다.

급여가 부서번호 20인 부서의 평균보다 높고 사원을 관리하는 'MANAGER' 로써 20 부서에 속하지 않은 사원을 조회해 보자.

쿼리문

```
SELECT b.empno, b.ename, b.job, b.sal, b.deptno
FROM (SELECT empno
        FROM emp
        WHERE sal > (SELECT AVG(sal)
                        FROM emp
                        WHERE deptno = 20)
        ) a, emp b
WHERE a.empno = b.empno
AND b.mgr is NOT NULL
AND b.deptno != 20;
```

결 과

	EMPNO	ENAME	JOB	SAL	DEPTNO
1	7566	JONES	MANAGER	2975	20
2	7698	BLAKE	MANAGER	2850	30
3	7782	CLARK	MANAGER	2450	10
4	7788	SCOTT	ANALYST	3000	20
5	7839	KING	PRESIDENT	5000	10
6	7902	FORD	ANALYST	3000	20

2.7 ORDER BY절 서브 쿼리

현재 조회한 컬럼이 아닌 다른 컬럼값으로 정렬하고자 한다면, ORDER BY 에 서브커리를 사용하면 된다.

```
SELECT EMPNO, ENAME, DEPTNO, HIREDATE
FROM EMP E
ORDER BY (SELECT DNAME
FROM DEPT
WHERE DEPTNO = E.DEPTNO) DESC;
```

결 과

	EMPNO	ENAME	DEPTNO	HIREDATE
1	1011	EDWARD	(null)	(null)
2	1010	BOB	(null)	(null)
3	7521	WARD	30	81/02/22
4	7499	ALLEN	30	81/02/20
5	1003	MATT	30	16/01/01
6	1002	MORRIS	30	16/09/08
7	1001	ALICE	30	16/09/08
8	7900	JAMES	30	81/10/03
9	7844	TURNER	30	81/09/08
10	7698	BLAKE	30	81/04/01
11	7654	MARTIN	30	81/09/28
12	7788	SCOTT	20	82/10/09
13	7902	FORD	20	81/10/03
14	7566	JONES	20	81/04/02
15	7369	SMITH	20	16/12/19
16	7876	ADAMS	20	83/01/12
17	7839	KING	10	81/11/17
18	7782	CLARK	10	81/06/01
19	7934	MILLER	10	82/01/23
20	1006	JORDAN	10	82/02/05

3. 집합 연산자

두 개 이상의 쿼리 결과를 하나로 결합하는 연산자이다.
여러 개의 SELECT문을 하나로 연결한다
집합 연산자로 결합되는 결과의 컬럼은 데이터타입이 동일해야 한다

3.1. UNION

여러 개의 SQL문의 결과에 대한 합집합을 반환한다.
중복행을 하나의 행으로 보여준다

영업사원과 관리자를 따로 질의하여 UNION으로 결합해 보자.

쿼리문

```
SELECT empno, ename, JOB FROM emp WHERE JOB = 'SALESMAN'
UNION
SELECT empno, ename, JOB FROM emp WHERE JOB = 'MANAGER';
```

결 과

	EMPNO	ENAME	JOB
1	7499	ALLEN	SALESMAN
2	7521	WARD	SALESMAN
3	7566	JONES	MANAGER
4	7654	MARTIN	SALESMAN
5	7698	BLAKE	MANAGER
6	7782	CLARK	MANAGER
7	7844	TURNER	SALESMAN

3.2. UNION ALL

여러 개의 SQL문의 결과에 대한 합집합을 반환한다.
UINION 연산자와 다르게 중복행도 모두 출력한다

급여가 1000 미만인 직원, 2000 미만인 직원을 조회하여 UNION ALL 로 결합해 보자.
중복 결과를 포함하여 출력한다.

쿼리문

```
SELECT empno, ename, sal FROM emp WHERE sal < 1000
UNION ALL
SELECT empno, ename, sal FROM emp WHERE sal < 2000
ORDER BY empno;
```

결 과

	EMPNO	ENAME	SAL
1	7369	SMITH	800
2	7369	SMITH	800
3	7499	ALLEN	1600
4	7521	WARD	1250
5	7654	MARTIN	1250
6	7844	TURNER	1500
7	7876	ADAMS	1100
8	7900	JAMES	950
9	7900	JAMES	950
10	7934	MILLER	1300

3.3. INTERSECT

여러 개의 SQL문의 결과에 대한 교집합을 반환한다.
중복 행은 하나의 결과로 보여준다
EXISTS 또는 IN 서브쿼리를 이용한 SQL문으로 변경 가능하다

급여가 1000 초과인 직원을 조회하고 2000 미만인 직원을 각각 조회하여 INTERSECT로 결합해 보자.
급여가 1000 초과 2000 미만인 직원들만 조회한다.

쿼리문

```
SELECT empno, ename, sal FROM emp WHERE sal > 1000
INTERSECT
SELECT empno, ename, sal FROM emp WHERE sal < 2000;
```

결 과

	EMPNO	ENAME	SAL
1	7499	ALLEN	1600
2	7521	WARD	1250
3	7654	MARTIN	1250
4	7844	TURNER	1500
5	7876	ADAMS	1100
6	7934	MILLER	1300

3.4. MINUS

여러 개의 SQL문의 결과에 대한 차집합을 반환한다.
중복 행은 하나의 결과로 보여준다
NOT EXISTS 또는 NOT IN 서브쿼리를 이용한 SQL문으로 변경 가능하다

사원 전체 결과에서 MINUS 연산자를 이용해 급여가 2000 초과인 사원 제외하여 조회해 보자.

쿼리문

```
SELECT empno, ename, sal FROM emp
MINUS
SELECT empno, ename, sal FROM emp WHERE sal > 2000;
```

결 과

	EMPNO	ENAME	SAL
1	7369	SMITH	800
2	7499	ALLEN	1600
3	7521	WARD	1250
4	7654	MARTIN	1250
5	7844	TURNER	1500
6	7876	ADAMS	1100
7	7900	JAMES	950
8	7934	MILLER	1300

Chapter 7. DDL - TABLE 과 제약조건

1. 테이블

데이터베이스의 기본 데이터 저장 단위인 테이블은 사용자가 접근 가능한 데이터를 보유하며 레코드(행)와 컬럼(열)으로 구성된다
시스템 내에서 독립적으로 사용되길 원하는 데이터 집합을 표현하며 데이터 집합 간의 관계를 표현 할 수 있다. 테이블 간의 관계를 표현하는 데 외래 키(Foreign Key)를 사용한다.
DDL을 이용해 테이블을 생성(CREATE), 변경(ALTER), 제거(DROP)하는 등의 일을 수행할 수 있다

1.1. CREATE TABLE

CREATE TABLE을 이용해 테이블을 만들 때에는 각 컬럼들을 괄호 ( )로 묶어 지정
컬럼 뒤에 해당 컬럼의 데이터타입을 반드시 지정해야 한다
각 컬럼의 구분은 콤마(,)로 구분하며 CREATE TABLE 구문의 마지막은 세미콜론(;)으로 끝난다
테이블 내의 컬럼들은 같은 이름을 가질 수 없다

CREATE TABLE 문법

```
CREATE TABLE table_name (
column1 자료형1
, column2 자료형2
, …
);
```

```
CREATE TABLE test (
        empno NUMBER(4),
        ename VARCHAR2(30),
        sal NUMBER(7, 2)
);
```

사용자가 소유하는 모든 테이블 조회

```
SQL> SELECT table_name FROM USER_TABLES;
```

```
TABLE_NAME
------------------
DEPT
EMP
BONUS
SALGRADE
TEST
```

1.2. 데이터 타입

1.2.1. CHAR

고정 길이 문자 데이터 타입이다.
VARCHAR2 와 동일한 형태의 데이터를 저장할 수 있다
입력된 데이터의 크기와 무관하게 고정된 길이만큼 저장 공간을 차지한다
저장 공간의 크기를 명시해야 하며, 최소 크기는 1 이다.
예) char(10)

1.2.2. VARCHAR2

가변 길이 문자 데이터 타입이다.
실제 입력된 데이터의 크기만큼 저장 공간을 차지한다
최대 크기를 명시해야 하며, 최소 크기는 1 이다.
예) varchar2(30)

1.2.3. NUMBER

숫자를 나타내기 위한 데이터 타입이다.
음수, 0, 양수를 저장할 수 있다
연산이 필요한 컬럼에 NUMBER 타입을 지정한다
가변길이 데이터 타입이다
소수점이 지정되지 않았는데 입력되거나, 지정된 소수점 자릿수 이상 입력되면 반올림되어 저장된다.

NUMBER 타입의 3가지 방법

NUMBER	최고 38자리까지의 숫자 저장 가능
NUMBER(w)	W 자리까지의 수치로 최대 38 자리까지 가능 (유효숫자가 38자리)
NUMBER(w, d)	W 는 전체 자릿수, d 는 소수점 이하 자릿수를 지정 W 의 길이는 d 의 길이를 포함

1.2.4. DATE

날짜와 시간을 저장하는 날짜형 데이터 타입이다.
BC 4712년 1월 1일 ~ AD 4712년 12월 31일까지의 날짜를 저장할 수 있다. 날짜와 시간의 연산이 빈번한 경우 사용한다. 세기, 년도, 월, 일, 시간, 분, 초 데이터를 포함한다
SYSDATE는 현재 날짜와 시간을 제공한다. (DATE타입)

1.2.5. TIMESTAMP

DATE형 의 확장된 형태로 밀리초 단위까지 표현 가능한 데이터 타입이다.

1.2.6. LONG

가변 길이 문자형 데이터 타입이다.
최대 크기는 2GB 이며, 테이블에 한 개의 컬럼만 생성 가능하다.
SELECT의 결과로 데이터가 직접 조회된다

1.2.7. CLOB

가변 길이 문자형 데이터 타입이다.
최대 크기는 4GB 이며, 테이블에 여러 개의 컬럼을 생성할 수 있다
SELECT의 결과로 데이터의 위치가 조회된다

1.2.8. RAW, LONG RAW

이미지나 사운드를 저장할 수 있는 데이터 타입이다.
이진 데이터를 저장하며 16진수 형태로 표현된다
VARCHAR2 데이터 타입과 유사하지만 문자 변환을 수행할 수 없다

1.2.9. BFILE

대용량 이진 데이터를 파일 형태로 저장하는 데이터 타입이다.
최대 크기는 4GB 이다.

1.3. 명명 규칙

테이블명과 컬럼명을 사용자가 부여할 때 지켜야 하는 규칙이다

- 문자로 시작한다.
- 1~30 자까지 가능하다.
- A~Z의 대소문자, 0~9의 숫자, 특수기호(_, $, #) 만 가능하다.
- 오라클의 예약어나 다른 객체명과 중복 불가하다.
- 공백을 허용하지 않는다

1.4. 제약 조건

테이블에 제약사항을 걸어 부적절한 데이터가 입력되는 것을 막기 위한 것이다
데이터 무결성을 유지하기 위해 사용자가 지정하는 성질이다
CREATE TABLE 구문에서 지정할 수 있으며, 테이블 생성 후에 ALTER TABLE table_name ADD CONSTRAINTS 명령어를 이용하여 추가하는 것도 가능하다

테이블 생성시 제약조건 생성하기

컬럼 레벨에서 작성

```
  --컬럼명 없이 제약조건 생성
  --제약사항의 이름을 지정하지 않은 경우 SYS_xxxxxxx 와 같은 이름의 제약사항으로 생성된다

  CREATE TABLE emp01(
        empno NUMBER PRIMARY KEY
  , ename VARCHAR2(20) NOT NULL
  , job VARCHAR2(9) UNIQUE
  , deptno NUMBER(2) REFERENCES dept(deptno)
  );

  --USER_CONSTRAINTS 테이블에서 확인
  SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
  FROM user_constraints
  WHERE table_name='EMP01';
```

결과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C007047	C	EMP01	(null)
2	SYS_C007048	P	EMP01	(null)
3	SYS_C007049	U	EMP01	(null)
4	SYS_C007050	R	EMP01	SYS_C007046

```
--컬럼명 추가해서 제약조건 생성

--자료형 다음에 [CONSTRAINTS 제약조건명 제약조건]의 순으로 추가한다.


CREATE TABLE emp02(
        empno NUMBER CONSTRAINTS EMP02_EMPNO_PK PRIMARY KEY
, ename VARCHAR2(20) CONSTRAINTS EMP02_ENAME_NN NOT NULL
, job VARCHAR2(9) CONSTRAINTS EMP02_JOB_UK UNIQUE
, deptno NUMBER(2) CONSTRAINTS EMP02_DEPTNO_FK REFERENCES dept(deptno)
);


--USER_CONSTRAINTS 테이블에서 확인
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
FROM user_constraints
WHERE table_name='EMP02';
```

결과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	EMP02_ENAME_NN	C	EMP02	(null)
2	EMP02_EMPNO_PK	P	EMP02	(null)
3	EMP02_JOB_UK	U	EMP02	(null)
4	EMP02_DEPTNO_FK	R	EMP02	SYS_C007046

테이블 레벨에서 작성

NOT NULL 제약조건은 테이블레벨에서 작성할 수 없다. 테이블생성시 컬럼레벨에서 작성하거나, 테이블 생성 이후 ALTER TABLE table_name MODIFY문을 통해서 추가해야 한다.

```
--컬럼명 없이 제약조건 생성
--제약사항의 이름을 지정하지 않은 경우 SYS_xxxxxxx 와 같은 이름의 제약사항으로 생성된다

CREATE TABLE emp03(
        empno NUMBER
, ename VARCHAR2(20) NOT NULL
, job VARCHAR2(9)
, deptno NUMBER(2)
, PRIMARY KEY(empno)
, UNIQUE(job)
, FOREIGN KEY(deptno) REFERENCES dept(deptno)
);

--USER_CONSTRAINTS 테이블에서 확인
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME

FROM user_constraints

WHERE table_name='EMP03';
```

결과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C007055	C	EMP03	(null)
2	SYS_C007056	P	EMP03	(null)
3	SYS_C007057	U	EMP03	(null)
4	SYS_C007058	R	EMP03	SYS_C007046

```
--제약조건명 추가해서 제약조건 생성

CREATE TABLE emp04(
        empno NUMBER
, ename VARCHAR2(20) CONSTRAINTS EMP02_ENAME_NN NOT NULL
, job VARCHAR2(9)
, deptno NUMBER(2)
, CONSTRAINTS EMP02_EMPNO_PK PRIMARY KEY

, CONSTRAINTS EMP02_JOB_UK UNIQUE

, CONSTRAINTS EMP02_DEPTNO_FK REFERENCES dept(deptno)

);

--USER_CONSTRAINTS 테이블에서 확인
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME

FROM user_constraints

WHERE table_name='EMP04';
```

결과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	EMP04_ENAME_NN	C	EMP04	(null)
2	EMP04_EMPNO_PK	P	EMP04	(null)
3	EMP04_JOB_UK	U	EMP04	(null)
4	EMP04_DEPTNO_FK	R	EMP04	SYS_C007046

1.4.1. NOT NULL

해당 컬럼에 NULL값이 입력될 수 없도록 한다

```
-- 제약사항의 이름을 지정하지 않고 NOT NULL 제약을 적용
CREATE TABLE test (
        data varchar2(20) NOT NULL
);
```

```
-- 제약사항의 이름을 test_nn_data로 지정하고 NOT NULL 제약을 적용
CREATE TABLE test (
        DATA VARCHAR2(20) CONSTRAINT test_nn_data NOT NULL
);
```

* 제약사항의 이름을 지정하지 않은 경우 SYS_xxxxxxx 와 같은 이름의 제약사항으로 생성된다

1.4.2. UNIQUE

데이터의 유일성을 보장하는 제약 조건이다.
해당 컬럼에 중복 데이터가 존재할 수 없게 되며 자동 인덱스가 생성된다

```
CREATE TABLE test (
        data1 varchar2(20) UNIQUE,
        data2 varchar2(20)
);
```

```
-- 테이블 생성 후에 data2 컬럼에 UNIQUE 제약사항 변경
ALTER TABLE TEST ADD CONSTRAINT test_uk_data2 UNIQUE (data2);
```

1.4.3. CHECK

컬럼의 값을 특정 조건 범위로 제한한다.

```
CREATE TABLE test (
        data1 NUMBER,
        data2 NUMBER CHECK (data2 >=1 AND data2 <= 99)
);
```

```
-- 테이블 생성 후에 data2 컬럼에 UNIQUE 제약사항 변경
ALTER TABLE TEST ADD CONSTRAINT test_ck_data1 CHECK data1 IN (1000, 2000, 3000, 4000);
```

1.4.4. PRIMARY KEY

기본키는 UNIQUE와 NOT NULL 제약사항을 모두 포함한다
기본키는 행을 대표하는 컬럼으로 외래키들이 참조할 수 있는 키값의 자격을 가지고 있다
자동으로 인덱스를 생성하며 기본키 제약 조건의 이름과 같은 이름을 가진다

```
CREATE TABLE TEST (
        no NUMBER CONSTRAINT test_pk_no PRIMARY KEY,
        data VARCHAR2(30)
);
```

```
-- 테이블 생성 후에 data 컬럼에 기본키 제약사항을 생성할 경우
ALTER TABLE TEST ADD CONSTRAINT test_uk_data PRIMARY KEY (no);
```

1.4.5. FOREIGN KEY

기본키를 참조하는 컬럼 또는 컬럼의 집합을 뜻한다
참조하고 있는 기본키의 데이터타입과 일치해야 하며 외래키에 참조되고 있는 기본키는 삭제할 수 없는 제약이 걸린다

```
-- 기본키를 가지는 test테이블 생성
CREATE TABLE TEST (no NUMBER CONSTRAINT test_pk_no PRIMARY KEY, data VARCHAR2(30));
```

```
-- test2 테이블 생성
CREATE TABLE TEST2 (no NUMBER );
```

```
-- test 테이블의 기본키 no 컬럼을 참조하는 외래키 제약사항 설정
ALTER TABLE TEST2 ADD CONSTRAINT test2_fk_no FOREIGN KEY (no) REFERENCES test(no);
```

ON DELETE [CASCADE/SET NULL] 연산자를 통해 기본키가 삭제될 경우 같이 삭제된다.
외래키로 지정된 기본키를 삭제하는 경우, 외래키의 참조무결성에 의해 기본키를 삭제할 수 없다. 그 번거로움을 덜기 위해서 외래키를 설정할 때, 기본키 삭제시 따라 삭제 또는 NULLABLE로 제약조건을 변경할 수 있다.

📁 ON DELETE CASCADE : 부모테이블기본키(parentTbl의 name) 삭제시, 자식테이블의 name컬럼 제약조건도 따라 삭제.

```
CREATE TABLE test1 (
no NUMBER(4) PRIMARY KEY,
name VARCHAR2(10) CONSTRAINT test4_fk REFERENCES parentTbl(name) ON DELETE CASCADE
) ;
```

📁 ON DELETE SET NULL : 부모테이블기본키(parentTbl의 name) 삭제시, 자식테이블의 name컬럼 제약조건은 NULLABLE로 변경.

```
CREATE TABLE test1 (
no NUMBER(4) PRIMARY KEY,
name VARCHAR2(10) CONSTRAINT test4_fk REFERENCES parentTbl(name) ON DELETE SET NULL
) ;
```

1.4.6. 제약사항 확인

- **제약사항 정보를 확인할 수 있는 데이터사전**

USER_CONS_COLUMNS : 컬럼에 할당된 제약 조건 조회
USER_CONSTRAINTS : 유저가 소유한 모든 제약 조건 조회

- **사용자의 모든 제약사항 확인**

```
SQL> SELECT * FROM USER_CONSTRAINTS;
```

- **사용자의 모든 제약사항을 보기 편하게 출력하는 SQL**

```
SQL> SELECT
        SUBSTR(A.COLUMN_NAME,1,15) COLUMN_NAME,
        DECODE(B.CONSTRAINT_TYPE,
                'P','PRIMARY KEY',
                'U','UNIQUE KEY',
                'C','CHECK OR NOT NULL',
                'R','FOREIGN KEY'
        ) CONSTRAINT_TYPE,
        A.CONSTRAINT_NAME
FROM USER_CONS_COLUMNS A, USER_CONSTRAINTS B
WHERE A.TABLE_NAME = UPPER('&table_name')
        AND A.TABLE_NAME = B.TABLE_NAME
        AND A.CONSTRAINT_NAME = B.CONSTRAINT_NAME
ORDER BY 1;
```

2. ALTER TABLE

테이블 구조를 변경하기 위해서 사용하는 DDL 구문이다.
컬럼과 제약조건에 대하여, ADD(추가) / RENAME(이름변경) / MODIFY(수정) / DROP(삭제)를 수행할 수 있다.

2.1. 컬럼 추가 / 이름변경 / 수정 / 삭제

2.1.1 ALTER TABLE ~ ADD 컬럼

기존 테이블에 새로운 컬럼을 추가하는 명령 구문이다.
새로운 컬럼은 테이블 맨 마지막에 추가된다
원하는 위치에 컬럼을 지정해서 넣을 수 없다
데이터가 존재하는 로우(행)에는 컬럼이 추가되지만 컬럼 값은 NULL 값으로 입력된다

test 테이블에 job 컬럼 추가하기

```
SQL> DESC test;
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME                   VARCHAR2(30)
SAL             NUMBER(7,2)
SQL> ALTER TABLE test ADD (job VARCHAR(9));
Table TEST이(가) 변경되었습니다.
SQL> DESC test;
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME                   VARCHAR2(30)
SAL             NUMBER(7,2)
JOB             VARCHAR2(9)
```

2.1.2 ALTER TABLE ~ RENAME 컬럼

컬럼명을 바꿀 때 사용하는 명령 구문이다.

test 테이블의 ename 컬럼명을 empname 으로 변경

```
SQL> DESC test;
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME                   VARCHAR2(30)
SQL> ALTER TABLE test RENAME COLUMN ename to empname;
Table TEST이(가) 변경되었습니다.
SQL> DESC test;
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
EMPNAME         VARCHAR2(30)
```

2.1.3 ALTER TABLE ~ MODIFY 컬럼

테이블에 이미 존재하는 컬럼을 수정하는 명령 구문이다.

해당 컬럼에 데이터 유무에 따라 동작이 다르다

데이터 유무에 따른 컬럼 변경

	자료가 없는 경우	자료가 있는 경우
데이터 타입 변경	변경 가능	변경 불가 (조건부)
컬럼의 크기 변경	변경 가능	작은 크기로는 불가

* 자료가 있는 경우 데이터 타입이 가능한 경우는 CHAR와 VARCHAR2 뿐이다. 이 때, 데이터 손실이 없도록 기록된 최대값을 고려해서 변경하여야 한다.

test 테이블의 job 컬럼의 크기 변경

```
SQL> ALTER TABLE test MODIFY (job VARCHAR(30));
Table TEST이(가) 변경되었습니다.
SQL> DESC test;
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME                   VARCHAR2(30)
SAL             NUMBER(7,2)
JOB             VARCHAR2(30)
```

컬럼의 DEFAULT 값을 변경하는 경우, 변경시점부터 기본값이 적용된다.
기존데이터들은 영향을 받지 않는다.

```
SQL> ALTER TABLE EMP MODIFY COMM NUMBER DEFAULT 0;
Table EMP이(가) 변경되었습니다.
SQL> INSERT INTO EMP_COPY (EMPNO, ENAME, JOB, DEPTNO) VALUES (9999,'JANE','SALESMAN',30);
1행이 추가되었습니다.
SQL> SELECT * FROM EMP_COPY;
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
11	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
12	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
13	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
14	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
15	1001	ALICE	CLERK	1003	16/09/08	800	(null)	30
16	1002	MORRIS	CLERK	1003	16/09/08	800	(null)	30
17	1003	MATT	SALEMAN	(null)	16/01/01	1500	100	30
18	1010	BOB	(null)	(null)	(null)	(null)	(null)	(null)
19	1011	EDWARD	MANAGER	(null)	(null)	(null)	(null)	(null)
20	1006	JORDAN	MANAGER	7782	82/02/05	5000	(null)	10
21	1234	JONATHAN	(null)	(null)	(null)	(null)	(null)	30
22	9999	JANE	SALESMAN	(null)	(null)	(null)	0	30

위와 같이 데이터 추가시 COMM컬럼값을 주지 않았는데, 0이라는 기본값이 지정되었다. 이미 존재하던 다른 행의 COMM컬럼값에는 변함이 없다.

2.1.4 ALTER TABLE ~ DROP 컬럼

테이블에 존재하는 컬럼을 삭제하는 명령 구문이다.

test 테이블의 sal 컬럼 삭제

```
SQL> ALTER TABLE test DROP COLUMN sal;
Table TEST이(가) 변경되었습니다.
SQL> DESC test;
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME           VARCHAR2(30)
JOB             VARCHAR2(30)
```

2.2. 제약조건 추가 / 이름변경 / 수정 / 삭제

2.2.1 ALTER TABLE ~ ADD 제약조건

기존 테이블에 새로운 제약조건을 추가하는 명령어

```
--형식
ALTER TABLE table_name ADD [CONSTRAINT constraint_name] constraint_type (column_name);
```

제약조건이름없이 제약조건을 생성한다면, CONSTAINT키워드는 생략가능하다. 제약조건의 이름을 지정하지 않은 경우 SYS_xxxxxxx 와 같은 이름의 제약사항으로 생성된다

test 테이블에 제약조건 추가하기

쿼리문

```
ALTER TABLE EMP ADD CONSTRAINTS EMP_EMPNO_PK PRIMARY KEY(EMPNO);
```

```
Table EMP이(가) 변경되었습니다.
```

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS
WHERE TABLE_NAME='EMP'
```

결 과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C006997	C	EMP	(null)
2	EMP_EMPNO_PK	P	EMP	(null)

2.2.2 ALTER TABLE ~ RENAME 제약조건

제약조건명을 바꿀 때 사용하는 명령어

쿼리문

```
ALTER TABLE EMP RENAME CONSTRAINTS EMP_EMPNO_PK TO EMP_EMPNO_PK_NEW;
```

```
Table EMP이(가) 변경되었습니다.
```

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS
WHERE TABLE_NAME='EMP'
```

결 과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C006997	C	EMP	(null)
2	EMP_EMPNO_PK_NEW	P	EMP	(null)

2.2.3 ALTER TABLE ~ MODIFY 제약조건

NOT NULL을 제외한 제약조건은 수정이 불가하다. 삭제(DROP)후 다시 생성(ADD)해야 한다.
NOT NULL 제약조건은 ADD가 아닌 MODIFY명령어를 사용해서 추가해야 한다.
NULLABLE(기본값)에서 NOT NULL로 변경하는 것으로 이해해야 한다.

쿼리문

```
ALTER TABLE EMP MODIFY ENAME CONSTRAINTS EMP_ENAME_NN NOT NULL;
```

```
Table EMP이(가) 변경되었습니다.
```

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS
WHERE TABLE_NAME='EMP'
```

결 과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C006997	C	EMP	(null)
2	EMP_EMPNO_PK	P	EMP	(null)
3	EMP_ENAME_NN	C	EMP	(null)

2.2.4 ALTER TABLE ~ DROP 제약조건

테이블에 존재하는 제약조건을 삭제하는 명령어

```
--형식
ALTER TABLE table_name DROP [CONSTRAINT constraint_name];
```

기본적으로 제약조건명을 제시해서 삭제해야 한다. 제약조건명은 USER_CONSTRAINTS테이블에서 조회가능하므로, 정확한 제약조건명을 조회후 실행하도록 하자. 하지만 아래와 같이도 삭제가능하다.

```
ALTER TABLE table_name DROP PRIMARY KEY;
```

쿼리문

```
ALTER TABLE EMP DROP CONSTRAINTS EMP_ENAME_NN;
```

```
Table EMP이(가) 변경되었습니다.
```

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS
WHERE TABLE_NAME='EMP'
```

결 과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C006997	C	EMP	(null)
2	EMP_EMPNO_PK	P	EMP	(null)

3.3 ALTER TABLE ~ SET UNUSED

컬럼을 삭제하지 않지만 논리적으로 컬럼의 사용을 제한하는 옵션(비활성) 이다.
컬럼을 삭제하려는 테이블에 여러 사용자가 접근할 수 있기 때문에 무조건 삭제해버리는 것은 위험할 수 있다. 컬럼을 삭제할 경우 컬럼에 해당하는 데이터도 같이 삭제 해야하므로 데이터가 많을 경우 오랜 시간이 소요될 수 있다. 또한 삭제 작업을 할 경우 데이터 무결성을 위하여 다른 사용자의 접근을 막을 수 있다(LOCK 발생). 우선 SET UNUSED를 이용하여 비활성시킨 후에 작업량이 적은 시간대에 DROP을 하도록 한다.
단, ALTER TABLE ~ SET UNUSED 구문을 이용해 컬럼을 논리적으로 사용할 수 없게 만들 경우에도 DROP과 마찬가지로 데이터를 복구할 수 없다

test 테이블의 job 컬럼을 비활성

```
SQL> DESC test;
```

```
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME                   VARCHAR2(30)
JOB             VARCHAR2(30)
```

```
SQL> ALTER TABLE test SET UNUSED(job);
```

```
Table TEST이(가) 변경되었습니다.
```

```
SQL> DESC test;
```

```
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME                   VARCHAR2(30)
```

SET UNUSED 작업을 처리한 job 컬럼이 보이지 않는 걸 확인할 수 있다

3.4 ALTER TABLE ~ RENAME TO 새테이블명

테이블명을 바꿀 때 사용하는 명령어

test 테이블명을 test_ddl로 변경

```
SQL> SELECT table_name FROM tabs WHERE table_name LIKE '%TEST%';
TABLE_NAME
---------------
TEST
SQL> ALTER TABLE test RENAME TO test_ddl;
Table TEST이(가) 변경되었습니다.
SQL> SELECT table_name FROM tabs WHERE table_name LIKE '%TEST%';
TABLE_NAME
---------------
TEST_DDL
```

4. TRUNCATE TABLE

사용하던 테이블의 모든 행을 제거하기 위한 명령어
DROP TABLE 명령을 수행하면 테이블 존재 자체가 없어지는 것과는 달리 TRUNCATE TABLE은 테이블 기본 구조를 유지하면서 모든 데이터를 삭제한다

- test_ddl 테이블에 TRUNCATE TABLE 명령 수행

```
SQL> TRUNCATE TABLE test_ddl;
Table TEST_DDL이(가) 잘렸습니다.
```

5. DROP TABLE

테이블을 삭제하는 명령어
테이블에 존재하던 데이터는 삭제되며 테이블도 삭제된다
삭제 후 되돌릴 수 없으니 주의하여야 한다

test_ddl 테이블에 TRUNCAT TABLE 명령 수행

```
SQL> DROP TABLE test_ddl;
Table TEST_DDL이(가) 삭제되었습니다.
```

5.1 DELETE, DROP, TRUNCATE의 차이

DELETE, DROP TABLE, TRUNCATE TABLE을 수행하면 모두 테이블의 데이터가 삭제되는 상황이지만 차이점이 있다

	종류	ROLLBACK 가능여부	비고
DELETE	DML	가능	• 데이터가 지워지지만 테이블의 용량이 줄어들지 않는다. • TRIGGER가 걸려있다면 각 행이 삭제될 때마다 실행된다. • 이전에 할당되었던 영역은 삭제되어 빈 TABLE이나 CLUSTER에 그대로 남아 있게 된다.
TRUNCATE TABLE	DDL	불가능	• 데이터가 지워지며 테이블의 용량이 줄어든다. • TABLE과 관련된 구조(CONSTRAINT, TRIGGER 등)과 권한에 영향을 주지 않는다. • TABLE에 걸려 있는 TRIGGER는 실행되지 않는다.
DROP TABLE	DDL	불가능	• 데이터뿐만 아니라 테이블 자체가 삭제된다. • 모든 관련된 INDEX, CONSTRAINT, TRIGGER도 삭제된다.

6. 인덱스

'색인'이라고도 불리며 데이터베이스 객체 중 하나로, 테이블에 인덱스를 작성
인덱스의 역할은 검색속도의 향상에 있다
테이블과는 별개의 독립된 객체로 작성되지만 테이블에 의존적이다
기본키나 유일키와 같은 제약 조건이 지정되면 따로 인덱스를 생성하지 않더라도 해당 키에 대한 인덱스가 자동으로 생성된다

6.1. 인덱스의 장점

검색 속도가 빨라진다
시스템의 부하를 줄여 전체적인 성능을 향상시킨다

6.2. 인덱스의 단점

인덱스 생성에 시간이 필요하다
인덱스를 위한 추가적인 공간이 필요하다
검색 외 데이터 변경(INSERT / UPDATE / DELETE)가 빈번히 일어날 경우 성능이 저하될 수 있다

6.3. 인덱스가 필요한 경우

- 테이블의 행의 수가 많을 경우
- WHERE문에 특정 컬럼이 많이 사용될 때
- 검색 결과가 전체 데이터의 2~4%정도일 때
- JOIN에 자주 사용되는 컬럼
- NULL을 포함하는 행이 많은 컬럼일 경우

6.4. 인덱스가 불필요한 경우

- 데이터가 적은(수천 건 미만) 경우
- 조회 보다 삽입, 수정, 삭제 처리가 많은 테이블일 경우
- 조회 결과가 전체 행의 15% 이상 조회할 것으로 예상되는 경우

6.5. 인덱스 생성

6.5.1. 고유 인덱스(Unique Index)

중복되지 않는 데이터를 인덱스의 키 값으로 지정하는 인덱스로, 성능이 좋다
인덱스를 생성한 컬럼에 대해서 중복되는 값을 테이블에 삽입하려고 하면 에러 발생

단일 컬럼 고유 인덱스 생성

```
CREATE UNIQUE INDEX index_name ON table_name(column_name);
```

6.5.2. 비고유 인덱스(Non-Unique Index)

중복되는 데이터를 인덱스의 키 값으로 지정하는 인덱스

단일 컬럼 비고유 인덱스 생성

```
CREATE INDEX index_name ON table_name(column_name);
```

6.5.3. 함수 기반 인덱스

인덱스를 사용하려면 WHERE 절의 조건을 다른 형태로 가공해서는 안 된다
만약 WHERE절의 조건으로 'sal + 100'과 같이 가공된 형태로 사용해야 하고 인덱스를 생성하고 싶다면 함수 기반 인덱스를 생성해야 한다
인덱스의 생성으로 검색 속도를 향상되긴 하지만 데이터베이스 성능향상을 위한 근본적인 해결방안이 아니다. 또한 쿼리 조건이 바뀌면 인덱스를 다시 생성해야하는 단점이 있다

함수 기반 인덱스 생성

```
CREATE INDEX index_name ON table_name(column_name 가공 형태);
```

함수 기반 인덱스 예

```
CREATE INDEX idx_sal_add_bonus ON emp(sal + 100);
```

6.5.4. 결합 인덱스(Composite Index)

두 개 이상의 컬럼을 합쳐서 인덱스 생성
WHERE 절의 조건 컬럼이 2개 이상 AND로 연결되어 사용되는 경우 많이 사용

결합 인덱스 생성

```
CREATE INDEX index_name ON table_name(column_name1, column_name2, … );
```

6.5.5. BITMAP 인덱스

데이터 값의 종류가 적고 동일한 데이터가 많을 경우 사용
예) 성별 컬럼 : 남, 여

데이터의 변경량이 적고 값의 종류가 적을 때 좋은 인덱스
데이터가 어디에 존재하는지 정보를 BIT로 표시한다
BITMAP INDEX 생성 이후 컬럼 값이 새로 하나 더 생긴다면 인덱스를 전부 수정해 줘야 한다

BITMAP 인덱스 생성

```
CREATE BITMAP INDEX index_name ON table_name(column_name);
```

6.6. 인덱스 제거

```
DROP INDEX index_name;
```

6.7. 인덱스 재 생성

대량의 DML 작업을 수행하면 인덱스의 성능이 저하되므로 재설정을 해주는 것이 좋다
꾸준한 관리가 필요하다

```
ALTER INDEX index_name REBUILD;
```

6.8. 인덱스 조회

USER_INDEXES 뷰를 이용해 인덱스 정보를 조회할 수 있다

```
SELECT table_name, index_name, index_type
FROM user_indexes
where table_name = '테이블명';
```

7. 뷰

물리적인 테이블에 근거한 논리적인 가상의 테이블로, 데이터가 없고 테이블에 대한 SQL만 저장되어 있다. 사용자가 VIEW에 접근할 경우 SQL이 수행되면서 그 결과를 가져온다
여러 테이블에 대한 검색결과를 뷰 하나로 검색할 수 있다
복잡하고 긴 쿼리문을 뷰로 정의하면 접근을 단순화할 수 있다

7.1. 뷰 문법

```
CREATE [ OR REPLACE ] [ FORCE | NOFORCE ] VIEW view_name
AS subquery
[ WITH CHECK OPTION [CONSTRAINT 제약조건] ]
[ WITH READ ONLY ]
```

1. CREATE : 뷰 생성
2. CREATE OR REPLACE : 뷰를 생성하지만 같은 이름의 뷰가 있으면 질의가 변경된다
3. FORCE : 기본 테이블 유무에 관계없이 뷰를 생성
4. WITH CHECK OPTION : 제약조건에 맞는 데이터만 입력되거나 변경될 수 있다
5. WITH READ ONLY : SELECT만 가능한 뷰 생성

7.2. 뷰 정보 조회

USER_VIEWS 를 통해 뷰에 대한 정보를 조회 할 수 있다

```
SELECT view_name, text
FROM user_view
WHERE view_name = '뷰이름';
```

7.3. NOFORCE/FORCE

뷰는 일반적으로 기본테이블이 존재한다는 가정하에 쿼리문을 작성하지만, FORCE 옵션을 이용해서 간혹 존재하지 않는 테이블을 대상으로 뷰를 생성할 수도 있다.
기본값은 NOFORCE이고, 이는 생략 가능하다.

```
CREATE OR REPLACE FORCE VIEW HUMAN_RESOURCE_VIEW
AS
SELECT EMPNO, ENMAE, DEPTNO
FROM HUMAN_RESOURCE;
```

```
경고: 컴파일 오류와 함께 뷰가 생성되었습니다.
```

사용자가 소유한 뷰를 조회해본다

```
SELECT * FROM user_views;
```

	VIEW_NAME	TEXT
1	HUMAN_RESOURCE_VIEW	SELECT EMPNO, ENAME, DEPTNOFROM HUMAN_RESOURCE_VIEW

7.4. VIEW를 통한 DML문 실행

뷰는 가상테이블이지만, 기본테이블을 반영한다. 뷰에서 SELECT문을 통한 데이터 조회외에도, INSERT/UPDATE/DELETE의 DML문을 실행할 수 있다. 뷰에 대한 DML문은 기본테이블에 적용된다.

EMP테이블의 카피본인 EMP_COPY테이블을 만들고, EMP_COPY테이블에 대한 뷰를 생성, DML문을 실행해보자.

```
--- EMP_COPY 테이블 생성
CREATE TABLE EMP_COPY
AS
SELECT * FROM EMP;

-- VIEW생성
CREATE VIEW EMP_COPY_VIEW
AS
SELECT EMPNO, ENAME, DEPTNO
FROM EMP_COPY;

-- EMP_COPY_VIEW에 INSERT문 실행
INSERT INTO EMP_COPY_VIEW VALUES(1234,'JONATHAN',30);

--- EMP_COPY_VIEW 조회
SELECT * FROM EMP_COPY_VIEW;
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
10	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
11	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
12	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
13	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
14	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
15	1001	ALICE	CLERK	1003	16/09/08	800	(null)	30
16	1002	MORRIS	CLERK	1003	16/09/08	800	(null)	30
17	1003	MATT	SALEMAN	(null)	16/01/01	1500	100	30
18	1010	BOB	(null)	(null)	(null)	(null)	(null)	(null)
19	1011	EDWARD	MANAGER	(null)	(null)	(null)	(null)	(null)
20	1006	JORDAN	MANAGER	7782	82/02/05	5000	(null)	10
21	1234	JONATHAN	(null)	(null)	(null)	(null)	(null)	30

기본테이블인 EMP_COPY을 조회해보자.

```
---EMP_COPY 조회

SELECT * FROM EMP_COPY;
```

결과

	EMPNO	ENAME	DEPTNO
4	7566	JONES	20
5	7654	MARTIN	30
6	7698	BLAKE	30
7	7782	CLARK	10
8	7788	SCOTT	20
9	7839	KING	10
10	7844	TURNER	30
11	7876	ADAMS	20
12	7900	JAMES	30
13	7902	FORD	20
14	7934	MILLER	10
15	1001	ALICE	30
16	1002	MORRIS	30
17	1003	MATT	30
18	1010	BOB	(null)
19	1011	EDWARD	(null)
20	1006	JORDAN	10
21	1234	JONATHAN	30

새로 추가된 JANATHAN 사원의 부서번호를 20으로 변경하는 UPDATE문을 실행해보자.

```
-- EMP_COPY_VIEW에 UPDATE문 실행
UPDATE EMP_COPY_VIEW SET DEPTNO=20 WHERE EMPNO=1234;

--- EMP_COPY_VIEW 조회
SELECT * FROM EMP_COPY_VIEW WHERE EMPNO=1234;
```

결과

	EMPNO	ENAME	DEPTNO
1	1234	JONATHAN	20

기본테이블인 EMP_COPY을 조회해보자.

```
---EMP_COPY 조회

SELECT * FROM EMP_COPY WHERE EMPNO=1234;
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	1234	JONATHAN	(null)	(null)	(null)	(null)	(null)	20

JONATHAN 사원 레코드를 삭제해보자.

```
-- EMP_COPY_VIEW에 DELETE문실행
DELETE FROM EMP_COPY_VIEW WHERE EMPNO=1234;

--- EMP_COPY_VIEW 조회
SELECT * FROM EMP_COPY_VIEW WHERE EMPNO=1234;

---EMP_COPY 조회

SELECT * FROM EMP_COPY WHERE EMPNO=1234;



---EMP_COPY 조회

SELECT * FROM EMP_COPY WHERE EMPNO=1234;
```

EMP_COPY테이블에서 JONATHAN사원이 삭제되고, 이를 참조하는 뷰인 EMP_COPY_VIEW에서도 JONATHAN 사원이 보이지 않는다.

7.5. WITH CHECK OPTION

뷰에 대한 DML문을 실행할 때 뷰생성시 조건제시에 사용된 컬럼값을 변경하지 못하도록 한다.

부서번호가 30인 사원들만을 열람하는 뷰 EMP_VIEW_30을 WITH CHECK OPTION으로 생성하고,

```
-- VIEW생성
CREATE VIEW EMP_VIEW_30
AS
SELECT EMPNO, ENAME, DEPTNO
FROM EMP_COPY
WHERE DEPTNO = 30
WITH CHECK OPTION;

--- EMP_VIEW_30조회
SELECT * FROM EMP_VIEW_30;
```

결과

	EMPNO	ENAME	DEPTNO
1	7499	ALLEN	30
2	7521	WARD	30
3	7654	MARTIN	30
4	7698	BLAKE	30
5	7844	TURNER	30
6	7900	JAMES	30
7	1001	ALICE	30
8	1002	MORRIS	30
9	1003	MATT	30
10	1234	JONATHAN	30

JONATHAN 사원의 부서번호를 20으로 변경해보자.

```
UPDATE EMP_VIEW_30 SET DEPTNO=20
WHERE EMPNO=1234;
```

오류 보고 -

ORA-01402: view WITH CHECK OPTION where-clause violation

위와 같이 WITH CHECK OPTION설정으로 뷰 생성시 조건으로 제시된 컬럼을 변경할 수 없게 된다.

7.6. WITH READ ONLY

뷰에 가진 모든 컬럼에 대하여 UPDATE를 할 수 없도록 방지하는 것이다. WITH CHECK OPTION 이 뷰 생성시 조건 컬럼에 대해서만 UPDATE를 방지하는 것과 달리, 뷰의 모든 컬럼의 정보를 수정할 수 없게 된다.

```
-- VIEW생성
CREATE VIEW EMP_VIEW_30
AS
SELECT EMPNO, ENAME, DEPTNO
FROM EMP_COPY
WHERE DEPTNO = 30
WITH READ ONLY;

--- JONATHAN사원 이름 변경
UPDATE EMP_VIEW_30 SET ENAME='JAKE'

WHERE EMPNO=1234;
```

SQL 오류: ORA-42399: cannot perform a DML operation on a read-only view

42399.0000 - "cannot perform a DML operation on a read-only view"

7.7. TOP-N 구하기

TOP-N 분석은 연봉이 높은 상위 5명의 사원을 구한다거나, 입사일이 빠른 10명의 사원을 구하는 데 사용된다.

ROWNUM은 오라클에서 내부적으로 부여하는 컬럼이다. 쿼리문에서 SELECT 된 순서대로 부여된다.

```
SELECT ROWNUM, EMPNO, ENAME, SAL
FROM EMP;
```

결과

	ROWNUM	EMPNO	ENAME	SAL
1	1	7369	SMITH	800
2	2	7499	ALLEN	1600
3	3	7521	WARD	1250
4	4	7566	JONES	2975
5	5	7654	MARTIN	1250
6	6	7698	BLAKE	2850
7	7	7782	CLARK	2450
8	8	7788	SCOTT	3000
9	9	7839	KING	5000
10	10	7844	TURNER	1500
11	11	7876	ADAMS	1100
12	12	7900	JAMES	950
13	13	7902	FORD	3000
14	14	7934	MILLER	1300
15	15	1001	ALICE	800
16	16	1002	MORRIS	800
17	17	1003	MATT	1500
18	18	1010	BOB	(null)
19	19	1011	EDWARD	(null)
20	20	1006	JORDAN	5000

위 예제와 같이 별도의 ORDER BY 절을 기술하지 않는다면, 내부적으로 ROWNUM의 순서에 따라 정렬하게 된다.

ROWNUM은 내부적으로 이미 부여받은 값이기 때문에 ORDER BY 절을 통해 순서를 바꾸더라도 값은 변하지 않는다.

```
SELECT ROWNUM, EMPNO, ENAME, SAL
FROM EMP
ORDER BY SAL DESC;
```

결과

	ROWNUM	EMPNO	ENAME	SAL
1	20	1006	JORDAN	5000
2	9	7839	KING	5000
3	13	7902	FORD	3000
4	8	7788	SCOTT	3000
5	4	7566	JONES	2975
6	6	7698	BLAKE	2850
7	7	7782	CLARK	2450
8	2	7499	ALLEN	1600
9	10	7844	TURNER	1500
10	17	1003	MATT	1500
11	14	7934	MILLER	1300
12	3	7521	WARD	1250
13	5	7654	MARTIN	1250
14	11	7876	ADAMS	1100
15	12	7900	JAMES	950
16	16	1002	MORRIS	800
17	1	7369	SMITH	800
18	15	1001	ALICE	800
19	19	1011	EDWARD	(null)
20	18	1010	BOB	(null)

뷰와 ROWNUM을 이용해 TOP-N 구하기

```
-- VIEW생성
CREATE OR REPLACE VIEW EMP_VIEW_SAL
AS
SELECT EMPNO, ENAME, NVL(SAL,0) SAL
FROM EMP
ORDER BY SAL DESC;
--- EMP_VIEW_SAL  조회
SELECT ROWNUM, EMPNO, ENAME, SAL
FROM EMP_VIEW_SAL;
```

결과

	ROWNUM	EMPNO	ENAME	SAL
1	1	1006	JORDAN	5000
2	2	7839	KING	5000
3	3	7902	FORD	3000
4	4	7788	SCOTT	3000
5	5	7566	JONES	2975
6	6	7698	BLAKE	2850
7	7	7782	CLARK	2450
8	8	7499	ALLEN	1600
9	9	7844	TURNER	1500
10	10	1003	MATT	1500
11	11	7934	MILLER	1300
12	12	7521	WARD	1250
13	13	7654	MARTIN	1250
14	14	7876	ADAMS	1100
15	15	7900	JAMES	950
16	16	1002	MORRIS	800
17	17	7369	SMITH	800
18	18	1001	ALICE	800
19	19	1011	EDWARD	0
20	20	1010	BOB	0

위 예제에서 뷰를 생성하면서, ROWNUM 값을 새롭게 부여 받았다. 이를 이용해서 월급이 많은 상위 5명을 구해보자.

```
SELECT ROWNUM, EMPNO, ENAME, SAL
FROM EMP_VIEW_SAL
WHERE ROWNUM <=5;
```

결과

	ROWNUM	EMPNO	ENAME	SAL
1	1	7839	KING	5000
2	2	1006	JORDAN	5000
3	3	7788	SCOTT	3000
4	4	7902	FORD	3000
5	5	7566	JONES	2975

위 예제에서 WHERE절에 ROWNUM이 5이하인 사원만 추려내어 출력하였다.

인라인뷰로 TOP-N구하기

위 예제를 인라인뷰를 통해서 구해보자. 인라인뷰는 1회성의 서브쿼리이다. 보통 FROM절에서 사용하며, 테이블처럼 취급하면 된다.

형식

```
SELECT ...
FROM ... ( SELECT ...
FROM ...) --서브쿼리
); --메인쿼리
```

```
SELECT ROWNUM, EMPNO, ENAME, SAL
FROM ( SELECT EMPNO, ENAME, NVL(SAL,0) SAL
FROM EMP
ORDER BY SAL DESC )
WHERE ROWNUM <= 5;
```

결과

	ROWNUM	EMPNO	ENAME	SAL
1	1	7839	KING	5000
2	2	1006	JORDAN	5000
3	3	7788	SCOTT	3000
4	4	7902	FORD	3000
5	5	7566	JONES	2975

뷰를 명시적으로 생성하지 않고, 인라인뷰를 활용하여 월급 상위 5명의 명단을 구할 수 있다. 메인쿼리 안쪽에 인라인뷰를 정의하고, 이때 생성된 ROWNUM 을 이용하여 상위 5명을 추려낸다.

8. 시퀀스

숫자를 자동으로 생성하는 객체이다. 행을 구분하기 위한 고유값으로 기본키를 사용한다. 기본키는 중복되지 않은 유일한 값을 가져야 하는데 직접 값을 생성해내는 것이 사용자의 부담이 될 수 있다. 이럴 경우 시퀀스를 이용해 기본키로 지정을 한다.

8.1. 시퀀스 생성

```
CREATE SEQUENCE sequence_name
[ START WITH n ]
[ INCREMENT BY n ]
[ MAXVALUE n | NOMAXVALUE ]
[ MINVALUE n | NOMINVALUE ]
[ CYCLE | NOCYCLE ]
[ CACHE | NOCACHE ]
```

- START WITH : 시퀀스 시작 값
- INCREMENT BY : 시퀀스 증가 값
- MAXVALUE : 시퀀스 최대 값
- MINVALUE : 시퀀스 최소 값
- CYCLE : 최대값 도달 시 순환 여부, NOCYCLE은 증가완료시 에러를 유발한다.
- CACHE : 메모리상에서 시퀀스값을 관리(CACHE)할지 여부를 정한다. 기본값은 20, 원하는 숫자만큼 미리 만들어 캐시로 상주시킨다. NOCACHE설정은 메모리상에서 시퀀스값을 관리하지 않는다. 캐싱을 하는 것이 퍼포먼스상으로 우위에 있지만, 연속된 번호를 얻어야 한다면 NOCACHE가 바람직하다. (캐싱이 AGING-OUT되거나, DB의 비정상적종료로 채번이 불연속적으로 될수 있다. 물론 기본키상에 번호가 중복될 경우는 없다.)

8.2. 시퀀스 활용

시퀀스 생성 및 확인

```
CREATE SEQUENCE seq_empno
START WITH 1000
INCREMENT BY 1
CACHE 20;
```

```
SELECT seq_empno.NEXTVAL FROM dual;
SELECT seq_empno.NEXTVAL FROM dual;
SELECT seq_empno.NEXTVAL FROM dual;
```

```
1000
1001
1002
```

INSERT문에서 시퀀스 사용

```
INSERT INTO emp ( empno, ename)
VALUES (seq_empno.nextval, 'TEST');
```

```
SELECT empno, ename FROM emp WHERE ename = 'TEST';
```

```
EMPNO      ENAME
---------- ----------
1004       TEST
```

8.3. 시퀀스 수정

START WITH 값을 제외한 다른 설정들을 수정할 수 있다

```
ALTER SEQUENCE sequence_name
[ INCREMENT BY n ]
[ MAXVALUE n | NOMAXVALUE ]
[ MINVALUE n | NOMINVALUE ]
[ CYCLE | NOCYCLE ]
[ CACHE | NOCACHE ]
```

시퀀스 수정 예

```
ALTER SEQUENCE seq_empno
INCREMENT BY 10
CACHE 40;
```

```
SELECT seq_empno.NEXTVAL FROM dual;
SELECT seq_empno.NEXTVAL FROM dual;
SELECT seq_empno.NEXTVAL FROM dual;
```

```
1010
1024
1034
```

8.4. 시퀀스 삭제

DROP SEQUENCE 문으로 시퀀스를 삭제한다

```
DROP SEQUENCE seq_empno;
```

8.5. 시퀀스 DATA DICTIONARY

사용자의 시퀀스객체 정보를 저장하고 있는 USER_SEQUENCES테이블을 조회해보자.

```
SELECT SEQUENCE_NAME, MIN_VALUE, MAX__VALUE,
INCREMENT_BY, CYCLE_FLAG, LAST_NUMBER
FROM USER_SEQUENCES;
```

결과

	SEQUENCE_NAME	MIN_VALUE	MAX_VALUE	INCREMENT_BY	CYCLE_FLAG	ORDER_FLAG	CACHE_SIZE	LAST_NUMBER
1	SEQ_EMPNO	1	9999999999999999999999999999	1	N	N	20	1020

이중 LAST_NUMBER 컬럼의 값이 마지막 숫자값을 나타내며, 사용자가 CACHE설정을 해두었다면, CACHE에 의해 채번된 마지막번호값을 나타낸다.

9. USER

데이터베이스 관리자인 시스템권한(SYS,SYSTEM)은 사용자계정을 발급할 수 있다.

9.1. 사용자계정 생성

관리자계정으로 접속해서 새로운 사용자 USER_NEW를 생성해보자.

```
--관리자계정 접속
CONN sys/oracle AS SYSDBA;

--USER_NEW 생성
--사용자비밀번호 입력 : USER_NEW_PW
CREATE USER USER_NEW IDENTIFIED BY USER_NEW_PW;

--사용자생성확인
SELECT * FROM DBA_USERS;
```

결과

	USERNAME	USER_ID	PASSWORD	ACCOUNT_STATUS	LOCK_DATE	EXPIRY_DATE	DEFAULT_TABLESPACE	TEMPORARY_TABLESI
1	USER_NEW	50	(null)	OPEN	(null)	18/03/04	SYSTEM	TEMP
2	SCOTT	48	(null)	OPEN	(null)	18/02/24	SYSTEM	TEMP
3	TEST_USER1	49	(null)	OPEN	(null)	18/03/03	SYSTEM	TEMP
4	SYS	0	(null)	OPEN	(null)	18/02/24	SYSTEM	TEMP
5	SYSTEM	5	(null)	OPEN	(null)	18/02/24	SYSTEM	TEMP
6	ANONYMOUS	35	(null)	OPEN	(null)	14/11/25	SYSAUX	TEMP
7	APEX_PUBLIC_USER	45	(null)	LOCKED	14/05/29	14/11/25	SYSTEM	TEMP
8	FLOWS_FILES	44	(null)	LOCKED	14/05/29	14/11/25	SYSAUX	TEMP
9	APEX_040000	47	(null)	LOCKED	14/05/29	14/11/25	SYSAUX	TEMP

사용자를 생성하더라도, 기본권한인 CREATE SESSION이 없으면, 접속할 수 없다. CREATE SESSION을 포함한 기본 롤(권한묶음) CONNECT와 RESOURCE를 부여한다.

```
--기본권한 부여

GRANT CONNECT, RESOURCE TO USER_NEW;


--사용자 USER_NEW로 접속

CONN USER_NEW/USER_NEW_PW;
```

9.2. 기본적인 롤

ROLE	포함하는 권한
CONNECT	ALTER SESSION, CREATE CLUSTER, CREATE DATABASE LINK, CREATE SEQUENCE, CREATE SESSION, CREATE SYNONYM, CREATE TABLE, CREATE VIEW
RESOURCE	CREATE CLUSTER, CREATE PROCEDURE, C CREATE SEQUENCE, CREATE TABLE, CREATE TRIGGER
DBA	DATABASE객체 관리, 사용자계정관련 모든 권한.

9.3. 사용자계정 삭제

```
--관리자계정 접속
CONN sys/oracle AS SYSDBA;

--USER_NEW 계정삭제
DROP USER USER_NEW;
```

사용자계정을 삭제하면, 사용자 관련 권한, 롤 모두 일괄적으로 삭제된다.

10. ROLE

권한의 부여와 회수를 쉽게할 수 있게 해준다
권한의 묶음. 부여하고자 하는 권한들을 ROLE로 묶어 한번에 여러 권한을 부여하고 회수할 수 있다.
CREATE ROLE 권한을 가진 사용자에 의해 생성된다
한 사용자가 여러 ROLE에 접근할 수 있으며, 여러 사용자에게 같은 ROLE을 부여할 수 있다
권한 부여, 회수와 같은 명령어를 사용한다
사용자 역할에 맞도록 ROLE을 설정하고 유저에게 지정만 하면 되므로 여러 유저가 있더라도 권한을 부여하는 작업을 효율적으로 처리할 수 있다

10.1. ROLE 생성 문법

```
CREATE ROLE role_name
```

10.2. ROLE을 이용한 사용자의 권한 처리 예

```
-- ROLE 생성
SQL> CREATE ROLE role_manager;

-- ROLE에 권한 부여
SQL> GRANT create session, create table TO role_manager;

-- scott, test1, test2 사용자에게 ROLE 부여
SQL> GRANT role_manager TO scott, test1, test2;
```

10.3. ROLE을 부여한 사용자의 권한 회수

```
SQL> REVOKE role_manager FROM test1, test2;
```

11. 동의어(SYNONYM)

동의어는 데이터베이스객체에 대한 별칭이다. 공유하는 테이블에 접근하기 위해서는 **[사용자명.테이블명]**의 문법을 사용하는 데, 이때 동의어를 사용해서 간결하게 할 수 있다.

형식

```
CREATE [PUBLIC] SYNONYM synonym_name
FOR user_name.object_name;
```

object_name은 동의어를 만들려는 데이터베이스객체(테이블, 뷰 등)이다.

11.1. 종류

가. 공개동의어

권한을 주는 사용자가 정의한 동의어로 누구나 사용할 수 있습니다. 동개 동의어는 DBA권한을 가진 사용자만이 생성할 수 있습니다. SYNONYM앞에 PUBLIC을 붙여서 정의한다.

나. 비공개동의어

객체에 대한 접근권한을 부여받은 사용자가 정의한 동의어. 해당사용자만 사용할 수 있다.

11.2. DUAL

DUAL은 SYS가 소유한 테이블로, SYS.DUAL로 접근해야 하지만, SYNONYM을 사용해서 모든사용자가 DUAL로 간결하게 접근하고 있다.

11.3. 공개동의어

공개동의어를 생성해보자. 공개동의어는 DBA가 정의하고 모든 사용자가 열람할 수 있다. SYS계정으로 로그인해서 SCOTT계정의 EMP테이블의 공개동의어를 생성한다.

```
--- SYS로 로그인.
CONN SYS/ORACLE AS SYSDBA;
CREATE PUBLIC SYNONYM EMP_PUBLIC
FOR SCOTT.EMP;

--- 새로운 테스트유저를 생성하고, 기본롤과 해당테이블 열람권한을 부여하자.
CREATE USER TEST_USER1 IDENTIFIED BY 1234;
GRANT CONNECT, RESOURCE TO TEST_USER1;
GRANT SELECT ON SCOTT.EMP TO TEST_USER1;

--- TEST_USER1로 접속하고, EMP_PUBLIC에 접근해보자.
CONN TEST_USER1/1234;
SELECT * FROM EMP_PUBLIC;
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	16/12/19	800	(null)	20
2	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
3	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30
4	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
5	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
6	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
7	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
8	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
9	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
10	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
11	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
12	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
13	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
14	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
15	1001	ALICE	CLERK	1003	16/09/08	800	(null)	30
16	1002	MORRIS	CLERK	1003	16/09/08	800	(null)	30
17	1003	MATT	SALEMAN	(null)	16/01/01	1500	100	30
18	1010	BOB	(null)	(null)	(null)	(null)	(null)	(null)
19	1011	EDWARD	MANAGER	(null)	(null)	(null)	(null)	(null)
20	1006	JORDAN	MANAGER	7782	82/02/05	5000	(null)	10

위 예제에서 원칙적으로는 SCOTT.EMP로 접근해야 하지만, SYNONYM인 EMP_PUBLIC을 사용해서 간결하게 접근할 수 있다.

11.4. 비공개동의어

비공개 동의어를 생성해보자. 비공개동의어는 정의한 사용자만 사용가능하다.

```
--- TEST_USER1로 접속하고, 비공개동의 MY_EMP를 만들어보자.
CONN TEST_USER1/1234;
CREATE SYNONYM MY_EMP
FOR SCOTT.EMP;

--MY_EMP 동의어로 테이블접근.
SELECT * FROM MY_EMP;
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	16/12/19	800	(null)	20
2	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
3	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30
4	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
5	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
6	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
7	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
8	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
9	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
10	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
11	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
12	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
13	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
14	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
15	1001	ALICE	CLERK	1003	16/09/08	800	(null)	30
16	1002	MORRIS	CLERK	1003	16/09/08	800	(null)	30
17	1003	MATT	SALEMAN	(null)	16/01/01	1500	100	30
18	1010	BOB	(null)	(null)	(null)	(null)	(null)	(null)
19	1011	EDWARD	MANAGER	(null)	(null)	(null)	(null)	(null)
20	1006	JORDAN	MANAGER	7782	82/02/05	5000	(null)	10

위와 같이 MY_EMP라는 동의어로 SCOTT.EMP라는 테이블에 접근할 수 있다.

MY_EMP라는 동의어를 SCOTT계정에서 사용가능하지 확인해보자.

```
--- SCOTT으로 접속하고, 비공개동의 MY_EMP로 접속해보자.
CONN SCOTT/TIGER;
SELECT * FROM MY_EMP;
```

ORA-00942: table or view does not exist

테이블의 소유주인 SCOTT일지라도, TEST_USER1이 생성한 비공개동의어 MY_EMP로는 본인의 EMP테이블에 접근할 수 없다.

12 권한

사용자가 테이블에 접근할 수 있도록 하거나, 해당 테이블에 사용할 수 있는 SQL문에 대하여 제한을 두는 것을 말한다. 다수의 사용자가 데이터베이스에 접근하여 저장된 정보를 사용하므로 정보에 대한 보안이 이루어져야 한다. 사용자마다 다른 권한을 부여함으로써 보안을 설정한다.

시스템 권한 : 사용자의 생성과 제거, DB접근 및 객체를 생성할 수 있는 권한 등으로 주로 DBA에 부여한다

객체 권한 : 객체를 조작할 수 있는 권한

12.1 시스템 권한

사용자가 데이터베이스에서 특정 작업을 수행할 수 있도록 한다

권한의 ANY 키워드는 사용자가 모든 스키마에서 권한을 가지게 한다

대표적인 시스템 권한

시스템 권한	기능
CREATE SESSION	데이터베이스에 연결할 수 있는 권한
CREATE ROLE	데이터베이스 롤을 생성할 수 있는 권한
CREATE VIEW	뷰의 생성 권한
ALTER USER	생성한 사용자의 정의를 변경할 수 있는 권한
DROP USER	생성한 사용자를 삭제시키는 권한

12.2 시스템 권한 부여

```
GRANT [ system_privilege | role ] TO [ user | role | PUBLIC ]
[ WITH ADMIN OPTION ]
```

system_privilege : 부여할 시스템 권한의 이름

role : 부여할 롤 이름. role은 권한을 묶음으로 관리한다. 다음 챕터 DB 객체에서 자세히 다룬다.

user, role : 부여할 사용자 이름과 다른 데이터 베이스 롤 이름

PUBLIC : 시스템권한 또는 데이터베이스 역할을 모든 사용자에게 부여할 수 있게 한다

WITH ADMIN OPTION : 권한을 부여 받은 사용자도 부여받은 권한을 다른 사용자에게 부여할 수 있게 한다

scott 계정에 시스템 권한 부여

```
SQL> CONN sys/1234 AS SYSDBA
SQL> GRANT CREATE USER, ALTER USER, DROP USER TO scott WITH ADMIN
OPTION
```

scott 계정에 사용자를 생성, 수정, 삭제 할 수 있는 권한과 이를 다른 사용자와 롤에 부여할 수 있도록 권한 설정

12.3 시스템 권한 회수

```
REVOKE [ system_privilege | role ] FROM [ user | role | PUBLIC ]
```

scott 계정의 시스템 권한 회수

```
SQL> REVOKE CREATE USER, ALTER USER, DROP USER FROM scott;
```

scott 계정에 부여했던 사용자를 생성, 수정, 삭제 할 수 있는 권한을 회수

12.4 객체 권한

테이블이나 뷰, 시퀀스, 프로시저, 함수, 또는 패키지 중 지정된 객체에 특별한 작업을 할 수 있는 권한을 말한다. 기본적으로 객체를 소유한 사용자는 해당 객체의 모든 권한이 자동으로 획득된다
객체 소유자는 다른 사용자에게 특정 객체 권한을 부여할 수 있다
PUBLIC으로 권한을 부여하면 회수할 때도 PUBLIC으로 해야 한다

객체에 따른 권한 목록

객체권한	테이블	뷰	시퀀스	프로시저
ALTER	O		O	
DELETE	O	O		
EXECUTE				O
INDEX	O			
INSERT	O	O		
SELECT	O	O	O	

12.5 객체 권한 부여

```
GRANT object_privilege [ column ]
ON object
TO { user [, user] | role | PUBLIC }
[ WITH GRANT OPTION ]
```

object_privilege : 부여할 시스템 권한의 이름

object : 객체명

user, role : 부여할 사용자 이름과 다른 데이터베이스 롤

PUBLIC : 객체 권한 또는 데이터베이스 롤을 모든 사용자에게 부여할 수 있게 한다

WITH GRANT OPTION : 권한을 부여 받은 사용자도 부여받은 권한을 다른 사용자에게 부여할 수 있게 한다

scott 계정에 객체 권한 부여

```
SQL> GRANT SELECT, INSERT
ON emp
TO scott
WITH GRANT OPTION;
```

scott 계정에 emp 테이블에 대한 SELECT, INSERT 객체 권한을 부여하고 다른 사용자에게도 해당 권한을 부여할 수 있게 한다

12.6 객체 권한 회수

```
REVOKE { privilege[, privilege, …] | ALL }
ON object
FROM { user[, user] | role | PUBLIC }
[ CASCADE CONSTRAINTS ]
```

객체 권한을 부여한 부여자만 권한을 회수할 수 있다

CASCADE CONSTRAINTS : 객체 권한에서 사용된 참조 무결성 제한을 같이 삭제할 수 있다

scott 계정의 권한 회수

```
SQL> REVOKE SELECT, INSERT ON emp FROM scott;
```

scott 계정에 부여했던 emp테이블에 대한 SELECT, INSERT 권한 회수

Chapter 8. 공공데이터 테이블 이용하기

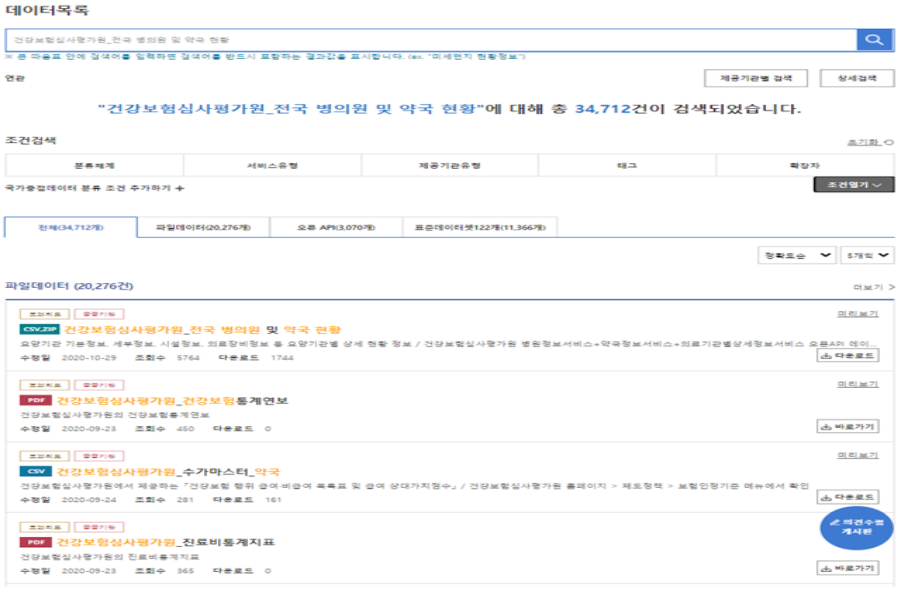
공공데이터 포털에서 데이터를 가져와 데이터 이용하기 공공데이터 내용은 병원정보 데이터를 가지고 실습을 진행하도록 하자. 공공데이터 포털에서 병원에 대한 정보를 가져와 오라클에 저장한뒤 그 데이터를 이용해 보자.

1.1 공공데이터 파일이용해서 테이블 만들기

파일방식으로 제공하는 데이터를 검색하여 오라클 데이터 베이스에 테이블로 생성해보자.

- csv방식의 공공데이터 선정하기

공공데이터 포털에서 csv방식으로 제공하는 데이터를 검색하여 다운로드 하기 검색창에 "건강보험심사평가원_전국 병의원 및 약국현황"입력하여 조회해보자.



결과 화면에서 첫 번째 데이터의 상세화면으로 들어가서 파일을 다운로드 한다. 파일은 ZIP파일로 제공하고 있다.

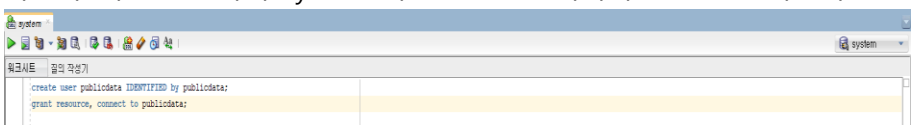


ZIP파일을 다운받아 압축을 해제하면 아래 그림과 같이 여러 가지 csv파일을 볼 수가 있다. 상세화면은 클릭을 하면 엑셀파일로 편하게 볼 수 있고 이 파일을 가지고 간단하게 DB에 테이블로 전환하여 사용할 수 있다.

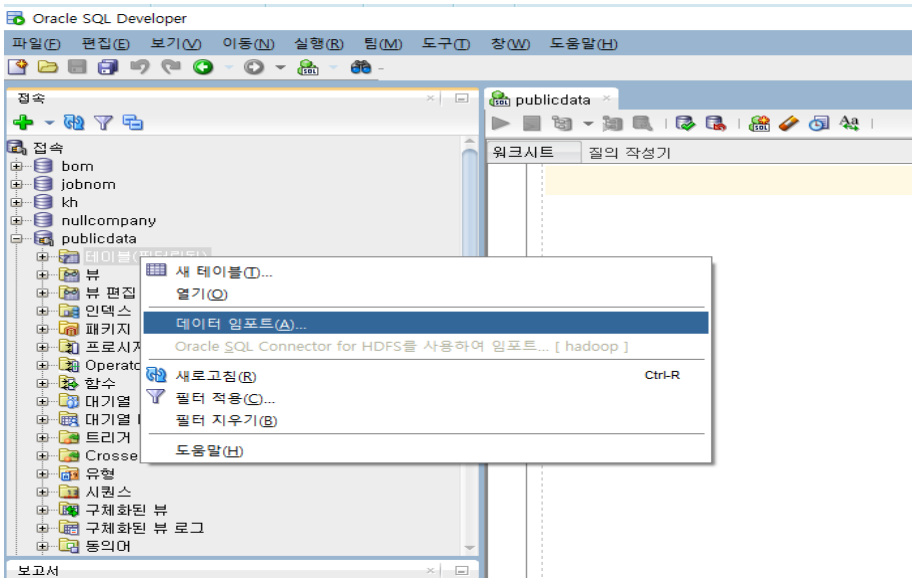


- sql developer이용하여 테이블 만들기

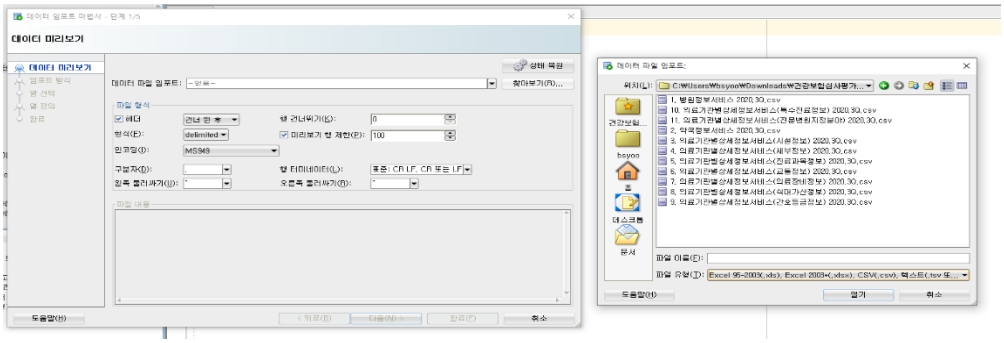
사용자 계정 생성하기 system 계정으로 공공데이터 모델링할 사용자 생성 및 권한부여



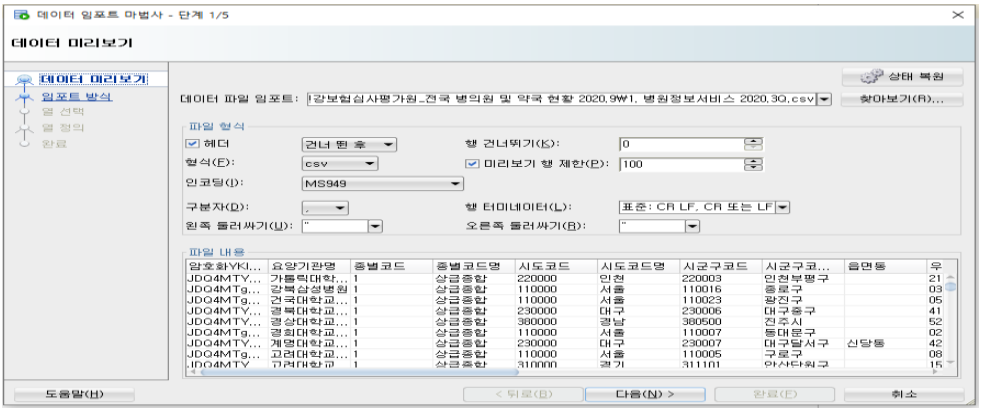
계정으로 접속 후 테이블 폴더에서 데이터 임포트 선택



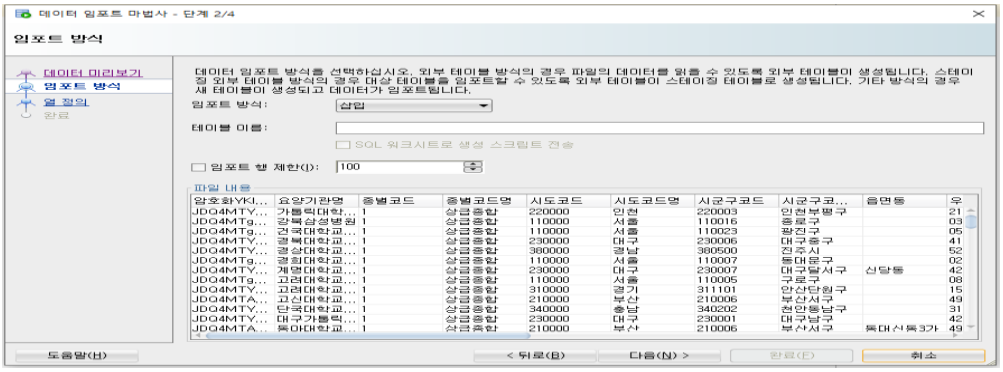
팝업창에서 공공데이터에서 받아온 csv중 병원정보서비스를 테이블로 만들어 보자.



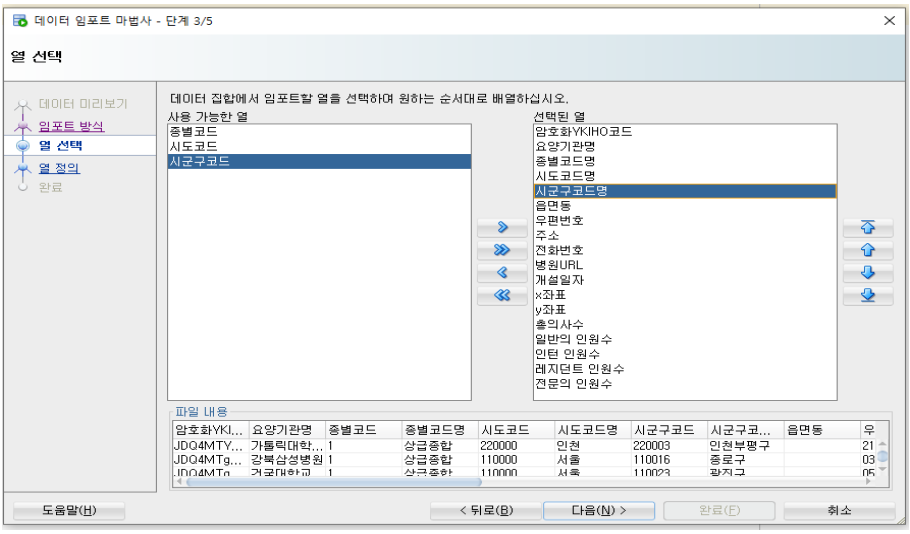
파일을 선택하면 csv의 데이터가 DB의 테이블 형식으로 들어온 것을 확인 할 수가 있다. 다음버튼으로 이동하자.



다음버튼으로 이동을 하게 되면 테이블이름을 설정할 수 있다. 테이블명을 hospital이라고 작성하고 다음으로 이동하자.



다음으로 이동하면 csv에서 필요한 항목만 선택해서 테이블을 생성할 수가 있다. 필요 없는 코드는 빼도록 하자. * 필요에 의해 조정이 가능하다.



항목별 데이터의 자료형, 컬럼명을 설정할 수 있다. 컬럼명을 영어로 변환하여 저장하도록 하자.

1.2 공공데이터 테이블 조회하기

1. 테이블 전체 조회하기

전체 테이블을 조회해서 데이터를 데이터와 테이블들과의 관계를 확인한다.
총 3개의 테이블로 구성했으며 데이터는 다음과 같다.

SELECT * FROM HOSPITAL;

HOSPITAL_CATEGORY_CODE	HOSPITAL_NAME	CITYCODE	STATECODE	POSTNUM	HOSPITAL_ADDRESS
1	가톨릭대학교인천성모병원	220003	220000	21431	인천광역시 부평구 동수로 56 (부평동)
1	강북삼성병원	110016	110000	3181	서울특별시 종로구 새문안로 29 (평동)
1	건국대학교병원	110023	110000	5030	서울특별시 광진구 능동로 120-1 (화양동)
1	경북대학교병원	230006	230000	41944	대구광역시 중구 동덕로 130 (삼덕동2가, 경북대학교병원)
1	경상대학교병원	380500	380000	52727	경상남도 진주시 강남로 79 (칠암동)
1	경희대학교병원	110007	110000	2447	서울특별시 동대문구 경희대로 23 (회기동)
1	계명대학교동산병원	230007	230000	42601	대구광역시 달서구 달구벌대로 1035 (신당동)
1	고려대학교의과대학부속구로병원	110005	110000	8308	서울특별시 구로구 구로동로 148 고려대부속구로병원 (구로동)
1	고려대학교의과대학부속안산병원	311101	310000	15355	경기도 안산시 단원구 적금로 123 (고잔동)
1	고신대학교복음병원	210006	210000	49267	부산광역시 서구 감천로 262 (암남동)
1	단국대학교의과대학부속병원	340202	340000	31116	충청남도 천안시 동남구 망향로 201 (안서동)
1	대구가톨릭대학교병원	230001	230000	42472	대구광역시 남구 두류공원로17길 33 (대명동)
1	동아대학교병원	210006	210000	49201	부산광역시 서구 대신공원로 26 (동대신동3가)
1	부산대학교병원	210006	210000	49241	부산광역시 서구 구덕로 179 (아미동1가)
1	분당서울대학교병원	310403	310000	13620	경기도 성남시 분당구 구미로173번길 82 (구미동, 분당서울대학교병원)
1	삼성서울병원	110001	110000	6351	서울특별시 강남구 일원로 81 (일원동, 삼성의료원)
1	서울대학교병원	110016	110000	3080	서울특별시 종로구 대학로 101 (연건동)
1	순천향대학교부속 천안병원	340202	340000	31151	충청남도 천안시 동남구 순천향6길 31 5동 (풍명동,순천향대학교병원)
1	순천향대학교부속부천병원	310300	310000	14584	경기도 부천시 조마루로 170 (중동)
1	아주대학교병원	310604	310000	16499	경기도 수원시 영통구 월드컵로 164 (원천동)
1	양산부산대학교병원	381100	380000	50612	경상남도 양산시 물금읍 금오로 20 ()
1	연세대학교 원주세브란스기독병원	320400	320000	26426	강원도 원주시 일산로 20-0 (일산동)
1	연세대학교의과대학 강남세브란스병원	110001	110000	6273	서울특별시 강남구 언주로 211 강남세브란스병원 (도곡동)
1	영남대학교병원	230001	230000	42415	대구광역시 남구 현충로 170 (대명동)
1	원광대학교병원	350300	350000	54538	전라북도 익산시 무왕로 895 (신동)
1	의료법인 길의료재단 길병원	220006	220000	21565	인천광역시 남동구 남동대로774번길 21-0 의료법인길의료재단길병원
1	인제대학교부산백병원	210004	210000	47392	부산광역시 부산진구 복지로 75, 진사로83번길 75 진사로83번길 81, 1층(일부)

SELECT * FROM CATEGORY

HOSPITAL_CATEGORY_...	HOSPITAL_CATEGORY_NAME
71	보건소
93	한의원
21	병원
72	보건지소
75	보건의료원
1	상급종합
11	종합병원
28	요양병원
31	의원
51	치과의원
73	보건진료소
41	치과병원
61	조산원
92	한방병원

SELECT * FROM STATE

STATECODE	STATENAME
220000	인천
320000	강원
360000	전남
340000	충남
410000	세종시
310000	경기
390000	제주
350000	전북
250000	대전
330000	충북
260000	울산
240000	광주
110000	서울
230000	대구
380000	경남
210000	부산
370000	경북

```
SELECT * FROM CITY
```

CITYCODE	CITYNAME
310403	성남분당구
220007	인천연수구
240004	광주광산구
110015	은평구
312001	용인기흥구
240003	광주서구
230004	대구서구
370100	경주시
260100	울산울주군
311200	평택시
320200	동해시
340201	천안서북구
340012	예산군
350100	군산시
360018	장흥군
110006	도봉구
360021	해남군
350600	김제시
311300	하남시

2. 전국 병원수 조회하기

COUNT를 이용해서 HOSPITAL에 등록되어 있는 병원을 조회할 수 있다.

```
SELECT COUNT(*) FROM HOSPITAL;
```

	COUNT(*)
1	71467

3. 병원중 상급 종합병원 조회하기

병원중 HOSIPITAL_CATEGORY_NAME이 상급종합병원을 구하는것으로 CATEGORY 테이블의 HOSIPITAL_CATEGORY_NAME항목이 필요하여 조인 후 WHERE을 통해서 조회할 수 있다.

```
SELECT *
FROM HOSPITAL H
JOIN CATEGORY C ON H.HOSPITAL_CATEGORY_CODE=C.HOSPITAL_CATEGORY_CODE
WHERE C.HOSPITAL_CATEGORY_NAME='상급종합';
```

HOSPITAL_CATEGORY_CODE	HOSPITAL_NAME	CITYCODE	STATECODE	POSTNUM	HOSPITAL_ADDRESS
1	경희대학교병원	110007	110000	2447	서울특별시 동대문구 경희대로 23 (회기동)
1	계명대학교동산병원	230007	230000	42601	대구광역시 달서구 달구벌대로 1035 (신당동)
1	고려대학교의과대학부속구로병원	110005	110000	8308	서울특별시 구로구 구로동로 148 고려대부속구로병원 (구로동)
1	고려대학교의과대학부속안산병원	311101	310000	15355	경기도 안산시 단원구 적금로 123 (고잔동)
1	고신대학교복음병원	210006	210000	49267	부산광역시 서구 감천로 262 (암남동)
1	단국대학교의과대학부속병원	340202	340000	31116	충청남도 천안시 동남구 망향로 201 (안서동)
1	대구가톨릭대학교병원	230001	230000	42472	대구광역시 남구 두류공원로17길 33 (대명동)
1	동아대학교병원	210006	210000	49201	부산광역시 서구 대신공원로 26 (동대신동3가)
1	부산대학교병원	210006	210000	49241	부산광역시 서구 구덕로 179 (아미동1가)
1	분당서울대학교병원	310403	310000	13620	경기도 성남시 분당구 구미로173번길 82 (구미동, 분당서울대학교병원)
1	삼성서울병원	110001	110000	6351	서울특별시 강남구 일원로 81 (일원동, 삼성의료원)
1	서울대학교병원	110016	110000	3080	서울특별시 종로구 대학로 101 (연건동)
1	순천향대학교부속 천안병원	340202	340000	31151	충청남도 천안시 동남구 순천향6길 31 5동 (봉명동,순천향대학교병원)
1	순천향대학교부속부천병원	310300	310000	14584	경기도 부천시 조마루로 170 (중동)
1	아주대학교병원	310604	310000	16499	경기도 수원시 영통구 월드컵로 164 (원천동)
1	양산부산대학교병원	381100	380000	50612	경상남도 양산시 물금읍 금오로 20 ()
1	연세대학교 원주세브란스기독병원	320400	320000	26426	강원도 원주시 일산로 20-0 (일산동)
1	연세대학교의과대학 강남세브란스병원	110001	110000	6273	서울특별시 강남구 언주로 211 강남세브란스병원 (도곡동)
1	영남대학교병원	230001	230000	42415	대구광역시 남구 현충로 170 (대명동)
1	원광대학교병원	350300	350000	54538	전라북도 익산시 무왕로 895 (신동)
1	의료법인 길의료재단 길병원	220006	220000	21565	인천광역시 남동구 남동대로774번길 21-0 의료법인길의료재단길병원
1	인제대학교부산백병원	210004	210000	47392	부산진구 복지로 75, 진사로83번길 75 진사로83번길 81, 1층(일부), 3
1	인하대학교의과대학부속병원	220004	220000	22332	인천광역시 중구 인항로 27 (신흥동3가)
1	재단법인아산사회복지재단 서울아산병원	110018	110000	5505	서울특별시 송파구 올림픽로43길 88 서울아산병원 (풍납동)
1	전남대학교병원	240001	240000	61469	광주광역시 동구 제봉로 42 (학동)
1	전북대학교병원	350402	350000	54907	전라북도 전주시 덕진구 건지로 20 (금암동)
1	조선대학교병원	240001	240000	61453	광주광역시 동구 필문대로 365 (학동)
1	중앙대학교병원	110008	110000	6973	서울특별시 동작구 흑석로 102 (흑석동)
1	충남대학교병원	250005	250000	35015	대전광역시 중구 문화로 282 (대사동)
1	충북대학교병원	330104	330000	28644	충청북도 청주시 서원구 1순환로 776-0 충북대학교병원
1	칠곡경북대학교병원	230003	230000	41404	대구광역시 북구 호국로 807 칠곡경북대학교병원 (학정동)
1	학교법인 고려중앙학원 고려대학교의과대학부속병원 (안암병원)	110012	110000	2841	서울특별시 성북구 고려대로 73 고려대병원 (안암동5가)
1	학교법인가톨릭학원가톨릭대학교서울성모병원	110021	110000	6591	서울특별시 서초구 반포대로 222 (반포동)
1	학교법인연세대학교의과대학세브란스병원	110010	110000	3722	서울특별시 서대문구 연세로 50-1 (신촌동)
1	한림대학교성심병원	310702	310000	14068	경기도 안양시 동안구 관평로170번길 22 (평촌동)
1	한양대학교병원	110011	110000	4763	서울특별시 성동구 왕십리로 222-1 (사근동)
1	화순전남대학교병원	360022	360000	58128	전라남도 화순군 화순읍 서양로 322 ()

4. 소재지가 대전인 보건소의 카테고리이름, 병원이름, 시도, 주소, 전화번호 조회하기

소재지와 카테고리이름을 조회해야하므로 STATE테이블, CATEGORY테이블을 HOSPITAL테이블과 JOIN하고 각 항목의 조건을 WHERE절에 사용하여 조회할수 있다.

```
SELECT HOSPITAL_CATEGORY_NAME, HOSPITAL_NAME, STATENAME, H.HOSPITAL_ADDRESS,
H.HOSPITAL_PHONE
FROM HOSPITAL H
JOIN CATEGORY C ON H.HOSPITAL_CATEGORY_CODE=C.HOSPITAL_CATEGORY_CODE
JOIN STATE S ON S.STATECODE=H.STATECODE
WHERE C.HOSPITAL_CATEGORY_NAME='보건소' AND S.STATENAME LIKE '대전';
```

HOSPITAL_CATEGORY_NAME	HOSPITAL_NAME	STATENAME	HOSPITAL_ADDRESS	HOSPITAL_PHONE
보건소	대전광역시대덕구보건소	대전	대전광역시 대덕구 석봉로38번길 55 (석봉동)	042-608-5452
보건소	대전광역시동구보건소	대전	대전광역시 동구 동구청로 147 (가오동)	042-251-6178
보건소	대전광역시서구보건소	대전	대전광역시 서구 만년로 74 (만년동)	042-488-9542
보건소	유성구보건소	대전	대전광역시 유성구 유성대로730번길 51 (장대동)	042-611-5000
보건소	중구보건소	대전	대전광역시 중구 산성로 63 (문화동)	042-288-8000

5. 전국 병원중 의사수가 가장 많은 병원 TOP3 조회하기

서브쿼리와 ROWNUM을 이용하여 상위 병원을 조회할 수 있다.

```
SELECT HOSPITAL_NAME, HOSPITAL_DOCTOR
FROM (SELECT * FROM HOSPITAL ORDER BY HOSPITAL_DOCTOR DESC)
WHERE ROWNUM <= 3;
```

HOSPITAL_NAME	HOSPITAL_DOCTOR
재단법인아산사회복지재단 서울아산병원	1645
서울대학교병원	1586
삼성서울병원	1356

6. 지역별 병원 수 구하기

지역별 병원을 구하기 위해 STATE데이틀과 HOSPITAL테이블을 JOIN하고 조건에 지역이름에 따른 GROUP BY절을 이용하여 조회할 수 있다.

```
SELECT STATENAME, COUNT(*)
FROM HOSPITAL
JOIN STATE USING(STATECODE)
GROUP BY STATENAME;
```

STATENAME	COUNT(*)
서울	17508
충남	2660
부산	5115
인천	3352
울산	1376
제주	935
광주	2105
대전	2229
대구	3753
강원	1840
전북	2803
경북	3290
전남	2507
세종시	386
경기	15510
경남	4034
충북	2064

7. 병원수가 가장 적은 시군구 구하기

지역별, TOPN이필요하여 서브쿼리와 ROWNUM, GURP BY를 이용하여 조회할 수 있다.

```
SELECT * FROM(
SELECT CITY.CITYNAME, COUNT(*)
FROM HOSPITAL
JOIN CITY USING(CITYCODE)
GROUP BY CITY.CITYNAME
ORDER BY COUNT(*))
WHERE ROWNUM <= 3;
```

CITYN...	COUNT(*)
울릉군	8
영양군	20
군위군	21

공공데이터 포털에서 가져온 데이터를 가지고 많은 정보를 조회하여 다양한 정보를 클라이언트에게 전

달할 수 있다. 조회뿐만 아니라 공공데이터의 기본정보에 클라이언트한테 데이터를 입력받으면 폭넓고 양질의 애플리케이션을 구현할 수 있다.